

Dispense di Metodi Numerici

Marco Galletti

A.A. 2024/2025

Sommario

Questa dispensa contiene un riassunto dei principali concetti trattati nel corso di Metodi Numerici con la professoressa **Gabriella Puppo** del corso di laurea di "Scienze matematiche per l'intelligenza artificiale" presso l'Università Roma "La Sapienza".

Indice

1	Ripasso di algebra lineare	2
1.1	Operazioni aritmetiche	3
1.2	Costo computazionale	4
1.3	Matrici	4
1.4	Norme di vettori e matrici	8
1.4.1	Autovalori e autovettori	13
2	Numeri di macchina	16
2.1	Numeri interi	16
2.2	Numeri reali	17
2.3	Operazioni aritmetiche con numeri floating-point	18
3	Sistemi lineari	22
4	Metodi iterativi	34
4.1	Metodo di Jacobi	34
4.2	Metodi preconditionati	36
4.2.1	Metodo di Gauss-Seidel	37
4.2.2	Metodo di Richardson	37
4.3	Velocità di convergenza	39
4.3.1	Approfondimento Richardson	40
4.4	Metodo del gradiente	41
4.5	Metodo Ortomin	43
4.6	Rette e piani	45
4.7	Gradiente coniugato	45
5	Interpolazione polinomiale	48
5.1	Errore di interpolazione Lagrange	50
6	Sistemi non lineari	54
6.1	Equazioni scalari non lineari	55
6.2	Metodo di bisezione	55
6.3	Metodo di Newton	58
6.4	Metodo di punto fisso	60
6.5	Sistemi sovradeterminati	66
6.6	Fattorizzazione QR	68

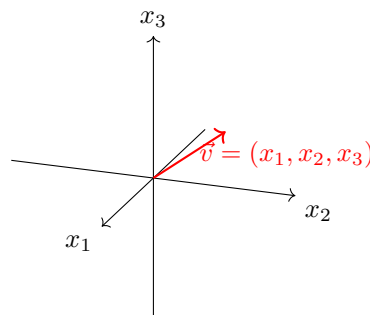
7	Autovalori e autovettori	74
7.1	Teoremi di localizzazione	75
7.2	Calcolo degli autovalori	78
7.3	Calcolo di tutti gli autovalori	80
7.4	SVD	82
7.5	Calcolo della SVD	85
8	Interpolazione Integrali	86
8.1	Interpolazione composita	86
8.2	Errore approssimazione composita	88
8.3	Integrazione numerica	90
8.4	Metodo dei rettangoli	90
8.4.1	Errore	91
8.5	Interpolazione composita	92
8.6	Metodo dei trapezi	92
8.7	Metodo di Cavalieri-Simpson	94
9	Quadratura	96
10	Metodi numerici per ODE	97
10.1	Metodi numerici ad un passo	98
10.2	Differenze finite	100
10.3	Ordine di convergenza	102
10.4	Stabilità assoluta	104
10.5	Metodo Runge-Kutta	107
10.6	Runge-Kutta adattivo	109
10.7	Runge-Kutta adattivo tramite <i>step-doubling</i>	109

1 Ripasso di algebra lineare

Un vettore è un oggetto $\underline{x} \in \mathbb{R}^n$ che ha n componenti. Appare come una matrice colonna:

$$\underline{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

Dove ogni $x_i \in \mathbb{R}$ è la componente del vettore. Ogni tanto potranno apparire anche dei numeri complessi.



Un vettore può anche essere scritto come combinazione lineare dei vettori della base canonica:

$$\underline{x} = x_1 \underline{e}_1 + x_2 \underline{e}_2 + \cdots + x_n \underline{e}_n$$

dove ogni $\underline{e}_i$ è un vettore della base canonica:

$$\underline{e}_i = \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$

e abbiamo che:

$$\underline{e}_i \cdot \underline{e}_j = \delta_{ij}$$

dove δ_{ij} è il delta di Kronecker, che vale 1 se $i = j$ e 0 altrimenti.

1.1 Operazioni aritmetiche

- **Somma:**

$$\underline{x} + \underline{y} = \begin{bmatrix} x_1 + y_1 \\ x_2 + y_2 \\ \vdots \\ x_n + y_n \end{bmatrix} \in \mathbb{R}^n$$

$\mathbb{R}^n$ è uno spazio lineare completo (se sommo due vettori o li moltiplico per uno scalare, ottengo sempre un vettore di $\mathbb{R}^n$).

- **Prodotto interno (o scalare):**

$$\langle \underline{x}, \underline{y} \rangle = \sum_{i=1}^n x_i y_i \in \mathbb{R}$$

Oppure lo si può scrivere come $\underline{x}^T \underline{y}$.

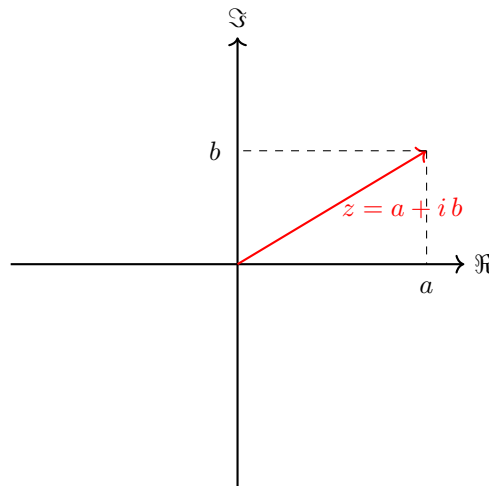
Osservazione 1.1. Se $x, y \in \mathbb{C}^n$, allora:

$$\langle \underline{x}, \underline{y} \rangle = \sum_{i=1}^n \underbrace{x_i}_{\text{complesso coniugato}} \cdot \overline{y_i} \in \mathbb{C}$$

Ricordiamo che $\sqrt{-1} = i$ (unità immaginaria), per cui $i^2 = -1$. Ad esempio:

$$\sqrt{-4} = \sqrt{4 \cdot (-1)} = \sqrt{4} \cdot \sqrt{-1} = 2i$$

I numeri complessi sono formati da una parte reale e una parte immaginaria, per cui $z \in \mathbb{C}$ si può scrivere come $z = a + ib$, dove $a, b \in \mathbb{R}$. a è la parte reale ($a = \operatorname{Re}(z) = \Re(z)$) e b è la parte immaginaria ($b = \operatorname{Im}(z) = \Im(z)$).



La lunghezza di un vettore complesso si calcola come:

$$|z|^2 = a^2 + b^2$$

Osserviamo che:

$$|z|^2 \neq z^2 \iff z^2 = (a + ib)^2 = a^2 + 2aib + (ib)^2 = a^2 - b^2 + 2aib$$

Chiamo $\bar{z}$ il complesso coniugato di z , ossia $\bar{z} = a - ib$. Allora:

$$z \cdot \bar{z} = (a + ib)(a - ib) = a^2 - (ib)^2 = a^2 + b^2 = |z|^2$$

Qualunque numero complesso z si può scrivere come $z = |z|e^{i\theta}$, dove θ è l'argomento di z . Se $\underline{x} \in \mathbb{C}^n$, allora:

$$\langle \underline{x}, \underline{x} \rangle = \sum_{i=1}^n x_i \bar{x}_i = \sum_{i=1}^n |x_i|^2 = \|\underline{x}\|^2$$

Se $\underline{x} \in \mathbb{R}^n$, allora $\bar{x} = x$.

1.2 Costo computazionale

Introduciamo la misura di flop (floating point operation). 1 flop consiste di 1 somma, 1 prodotto e 3 operazioni di memoria.

Esempio 1.1. $\langle \underline{x}, \underline{y} \rangle = \sum_{i=1}^n x_i y_i$ Potremmo scrivere un programma che calcola questa somma:

```
def dot_product(x, y):
    result = 0
    for i in range(len(x)):
        result += x[i] * y[i] # 1 flop
    return result
```

Per cui alla fine result contiene $\langle \underline{x}, \underline{y} \rangle$. Il costo computazionale indica il numero di flops necessario ad arrivare alla fine dell'algoritmo. Il prodotto interno costa n flops, ovvero è $O(n)$.

Quando scriviamo $\cos(x, y)$ intendiamo il coseno dell'angolo tra i due vettori, ed è definito come:

$$\cos(x, y) = \frac{\langle \underline{x}, \underline{y} \rangle}{\|\underline{x}\| \|\underline{y}\|}$$

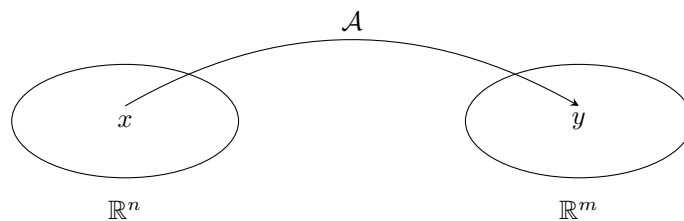
Diciamo che $\underline{x} \perp \underline{y} \iff \cos(x, y) = 0$, ovvero se i due vettori sono ortogonali.

1.3 Matrici

Una matrice $A \in \mathbb{R}^{m \times n}$ è un insieme di $m \times n$ elementi $a_{ij} \in \mathbb{R}$ disposti in m righe e n colonne.

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

Matrice come operatore: $\mathcal{A} : \mathbb{R}^n \rightarrow \mathbb{R}^m$



Se $\mathcal{A}$ è lineare: $\mathcal{A}(\alpha \underline{x}_1 + \beta \underline{x}_2) = \alpha \mathcal{A}(\underline{x}_1) + \beta \mathcal{A}(\underline{x}_2)$

Esempio 1.2. ed è quindi possibile rappresentare $\mathcal{A}$ come una matrice, cioè $\exists A \in \mathbb{R}^{m \times n}$ tale che:

$$y = \mathcal{A}(\underline{x}) \iff y = A\underline{x}$$

Gli elementi di A dipendono dalla base in cui sono rappresentati $\underline{x}$ e $\underline{y}$.

Prodotto matrice-vettore

è definito per $A \in \mathbb{R}^{m \times n}$ e $\underline{x} \in \mathbb{R}^n$:

$$y = A\underline{x} \implies y_i = \langle {}_i a, \underline{x} \rangle \quad \text{dove } {}_i a \text{ è la } i\text{-esima riga di } A$$

Quindi $y_i = \sum_{j=1}^n a_{ij}x_j$. Per calcolare y_i ci vogliono $O(n)$ flops. Per calcolare $\underline{y}$ ci vogliono $O(m \times n)$ flops.

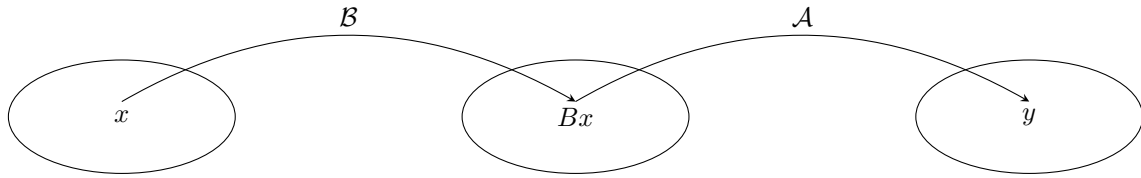
Prodotto matrice-matrice

è definito come:

$$A \cdot B = \begin{bmatrix} \vdots & \vdots & \ddots \\ \vdots & \vdots & \ddots \\ \vdots & \vdots & \ddots \\ \vdots & \vdots & \ddots \end{bmatrix} \begin{bmatrix} \vdots & \vdots & \ddots \\ \vdots & \vdots & \ddots \\ \vdots & \vdots & \ddots \end{bmatrix} = \begin{bmatrix} \vdots & \vdots & \ddots \\ \vdots & \vdots & \ddots \\ \vdots & \vdots & \ddots \end{bmatrix}$$

Quindi $A \in \mathbb{R}^{m \times n_A}$ e $B \in \mathbb{R}^{n_B \times n}$. Deve essere $n_A = n_B = p$ (Inner matrix dimension must agree). Si ottiene una matrice $C \in \mathbb{R}^{m \times n}$. c_{ij} è dato dal prodotto interno della riga i -esima di A : ${}_i \underline{a}$ e della colonna j -esima di B : ${}_j \underline{b}$.

$$y = Cx = ABx = A(Bx) \Leftrightarrow y = \mathcal{A} \circ \mathcal{B}(\underline{x}) \iff y = ABx$$



$$C = \langle {}_i \underline{a}, {}_j \underline{b} \rangle = \sum_{k=1}^p a_{ik} b_{kj}$$

Il costo computazionale è $O(m \times n \times p)$, dato che il costo di c_{ij} è $O(p)$, e ci sono $m \times n$ elementi c_{ij} . Se A e B sono quadrate allora $m = n = p$ e il costo computazionale è $O(n^3)$ flops.

Osservazione 1.2 (BLAS). *Basic Linear Algebra Subprograms sono dei programmi che implementano le operazioni di base tra matrici in maniera efficiente per calcolare quantità del tipo $A \cdot B$*

Sistemi lineari

Data una matrice $A \in \mathbb{R}^{n \times n}$ e un vettore $\underline{b} \in \mathbb{R}^n$, vogliamo trovare un vettore $\underline{x} \in \mathbb{R}^n$ tale che:

$$A\underline{x} = \underline{b}$$

La matrice A si dice matrice dei coefficienti, $\underline{b}$ è il termine noto e $\underline{x}$ è il vettore delle incognite (contiene i coefficienti della combinazione lineare delle colonne di A che danno $\underline{b}$).

$$\underline{b} = x_1 \underline{a}_1 + x_2 \underline{a}_2 + \cdots + x_n \underline{a}_n$$

Per cui x esiste se e solo se $\underline{b} \in \text{span}(\underline{a}_1, \underline{a}_2, \dots, \underline{a}_n)$ (spazio lineare generato dalle colonne di A). Se le colonne di A sono linearmente indipendenti, allora:

- Le colonne di A sono una base di $\mathbb{R}^n$
- Ogni vettore $\underline{b} \in \text{span}(\underline{a}_1, \underline{a}_2, \dots, \underline{a}_n)$
- x esiste sempre ed è unico

Se esistessero x_1 e x_2 tali che $Ax_1 = \underline{b} = Ax_2$, allora:

$$A(x_1 - x_2) = \underline{0} \implies x_1 - x_2 \in \text{Ker}(A)$$

Cioè $x_1 - x_2$ contengono i coefficienti della combinazione lineare delle colonne di A che danno $\underline{0}$. Ma dato che le colonne di A sono linearmente indipendenti, allora $x_1 - x_2 = \underline{0} \implies x_1 = x_2$.

Formalmente:

$$A\underline{x} = \underline{b} \iff \underline{x} = A^{-1}\underline{b}$$

ovvero esiste una matrice A^{-1} tale che $A^{-1}A = \mathbb{I}$. Le seguenti affermazioni sono equivalenti:

- $\forall \underline{b} \in \mathbb{R}^n, \exists! \underline{x} \in \mathbb{R}^n : A\underline{x} = \underline{b}$
- Le colonne di A sono linearmente indipendenti
- A è invertibile
- $\text{Ker}(A) = \{\underline{0}\}$
- $\exists A^{-1} : A^{-1}A = \mathbb{I}$
- $\det A \neq 0$
- $\lambda = 0$ non è autovalore di A

Se λ è autovalore di A , allora $\exists \underline{x} \in \mathbb{R}^n \setminus \{\underline{0}\} : A\underline{x} = \lambda \underline{x}$. Nel caso in cui fosse $\lambda = 0$ vorrebbe dire che $\exists \underline{x} \neq \underline{0} : A\underline{x} = \underline{0}$, ovvero che $\underline{x} \in \text{Ker}(A)$. Inoltre

$$\det(A) = \prod_{i=1}^n \lambda_i \implies \det(A) \neq 0 \iff \lambda_i \neq 0 \quad \forall i$$

Matrice trasposta

Se $A \in \mathbb{R}^{m \times n}$ con coefficienti a_{ij} , chiamo A^T la matrice trasposta di A che appartiene a $\mathbb{R}^{n \times m}$ con coefficienti a_{ji} .

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \longrightarrow A^T = \begin{bmatrix} a_{11} & a_{21} & \cdots & a_{m1} \\ a_{12} & a_{22} & \cdots & a_{m2} \\ \vdots & \vdots & \ddots & \vdots \\ a_{1n} & a_{2n} & \cdots & a_{mn} \end{bmatrix}$$

Ecco alcune proprietà della matrice trasposta:

- $(A^T)^T = A$
- $\langle x, Ay \rangle = \langle A^T x, y \rangle$ (Si può dimostrare con la definizione di prodotto scalare)

Matrici simmetriche

Una matrice $A \in \mathbb{R}^{n \times n}$ si dice simmetrica se $A^T = A$. Per queste matrici vale che:

$$\langle x, Ay \rangle = \langle Ax, y \rangle$$

Matrici ortogonali (importanti)

Una matrice Q si dice ortogonale se $Q^T = Q^{-1}$. Per cui:

$$Q^{-1}Q = \mathbb{I} \implies Q^T Q = \mathbb{I}$$

Ovvero se considero le colonne i, j di Q e calcolo $\langle q_i, q_j \rangle = q_i^T q_j = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases} = \delta_{ij}$.

Esempio 1.3. Si trovino tutte le matrici ortogonali 2×2 .

$$Q = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

Cerco $a, b, c, d \in \mathbb{R}$ tali che:

$$\mathbb{I} = Q^T Q = \begin{bmatrix} a & c \\ b & d \end{bmatrix} \begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Procedo a calcolare:

- $a^2 + c^2 = 1$
- $ab + cd = 0$
- $bc + dc = 0$
- $b^2 + d^2 = 1$

$$a^2 + c^2 = 1 \implies a = \cos \theta \quad c = \sin \theta \quad b^2 + d^2 = 1 \implies b = \cos \alpha \quad d = \sin \alpha$$

Per cui si ottiene:

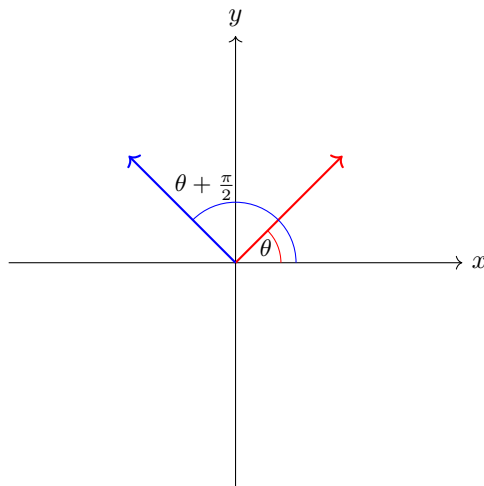
$$ab + cd = \cos \theta \cos \alpha + \sin \theta \sin \alpha = \cos(\theta - \alpha) = 0 \implies \theta - \alpha = \frac{\pi}{2} \vee \theta - \alpha = -\frac{\pi}{2}$$

Da $\theta - \alpha = -\frac{\pi}{2}$ si ottiene:

$$\alpha = \theta + \frac{\pi}{2}$$

Per cui $b = \cos \alpha = \cos(\theta + \frac{\pi}{2}) = -\sin \theta$ e $d = \sin \alpha = \sin(\theta + \frac{\pi}{2}) = \cos \theta$. Allora esistono infinite matrici Q ortogonali:

$$Q = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$



Applicando il vettore $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ alla matrice Q ottengo un vettore che forma un angolo θ con il vettore $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$:

$$Q \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} \cos \theta \\ \sin \theta \end{bmatrix}$$

Per cui Q ruota il vettore di un angolo θ .

1.4 Norme di vettori e matrici

Esempio 1.4. Si consideri un sistema lineare $A\underline{x} = \underline{b}$ con $A \in \mathbb{R}^{n \times n}$. Il calcolatore è in grado di darci una soluzione approssimata $\tilde{x}$. Vogliamo calcolare quanto è grande l'errore che abbiamo commesso.

$$\underline{e} = \underline{x} - \tilde{x} \in \mathbb{R}^n$$

Tuttavia per questo calcolo occorre conoscere x . Possiamo quindi calcolare il residuo:

$$\underline{r} = \underline{b} - A\tilde{x} \in \mathbb{R}^n$$

che è dunque calcolabile, tuttavia potrebbe crescere di dimensioni a seconda di come è fatta la matrice A . Si vorrebbe quindi avere un numero che ci dicesse quanto è grande $\underline{r}$.

Definizione 1.1 (Norma). Una norma su $\mathbb{R}^n$ è una funzione $\|\cdot\| : \mathbb{R}^n \rightarrow [0, +\infty)$ che soddisfa le seguenti proprietà:

- $\|x\| = 0 \iff x = \underline{0}$
- $\|x + y\| \leq \|x\| + \|y\|$ (disuguaglianza triangolare)
- $\|\alpha x\| = |\alpha| \|x\|$ (omogeneità)

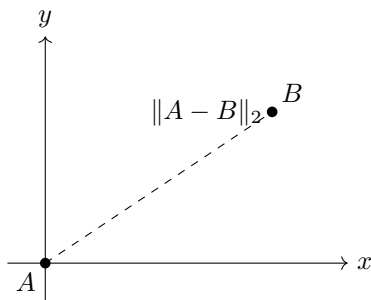
Ci sono infiniti tipi di norme. Noi useremo le norme p con $p \in \mathbb{N}$.

Definizione 1.2 (Norma p di un vettore).

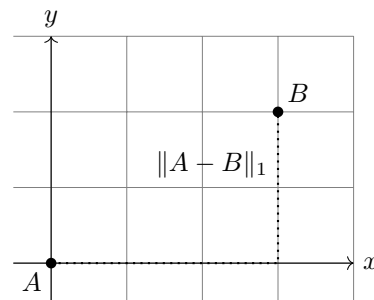
$$\|x\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{\frac{1}{p}}$$

Useremo in particolare le norme p con $p = 1, 2, \infty$.

$$\|x\|_1 = \underbrace{\sum_{i=1}^n |x_i|}_{\text{norma di manhattan}} \quad \|x\|_2 = \underbrace{\sqrt{\sum_{i=1}^n |x_i|^2}}_{\text{norma euclidea}} \quad \|x\|_\infty = \max_{i=1, \dots, n} |x_i|$$



(a) Distanza euclidea



(b) Distanza di Manhattan

Figura 1: Confronto tra distanza euclidea e distanza di Manhattan

Osservazione 1.3. *Tutte le norme sono equivalenti, nel senso che esistono delle costanti $c_1, c_2 > 0$ tali che:*

$$c_1 \|x\|_q \leq \|x\|_p \leq c_2 \|x\|_q \quad \forall x \in \mathbb{R}^n \implies \frac{1}{c_2} \|x\|_p \leq \|x\|_q \leq \frac{1}{c_1} \|x\|_p \quad \forall x \in \mathbb{R}^n$$

Quindi se una successione di vettori $x_n \in \mathbb{R}^n$ converge a $x \in \mathbb{R}^n$ rispetto a una norma, allora converge anche rispetto a tutte le altre norme.

$$\|x_n - x\|_p \rightarrow 0 \quad \forall p$$

Esempio 1.5. Dimostro l'equivalenza della $\|\cdot\|_1$ e della $\|\cdot\|_\infty$.

$$\begin{aligned} \|x\|_1 &= \sum_{i=1}^n |x_i| \leq \sum_{i=1}^n \max |x_i| \\ &= \max_i |x_i| \sum_{i=1}^n 1 = n \|x\|_\infty = n \|x\|_\infty \end{aligned}$$

Inoltre

$$\begin{aligned} \|x\|_1 &= \sum_{i=1}^n |x_i| \geq \max_i |x_i| = \|x\|_\infty \\ c_1 \|x\|_\infty &\leq \|x\|_1 \leq c_2 \|x\|_\infty \end{aligned}$$

Per cui possiamo considerare $c_1 = 1$ e $c_2 = n$.

Definizione 1.3 (Norme di matrici). a) Se considero una matrice $A \in \mathbb{R}^{m \times n}$ come un unico vettore così fatto:

$$a = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} \in \mathbb{R}^{m \times n},$$

definisco $\|A\|$ come $\|a\|$. Per esempio,

$$\|A\|_F = \sqrt{\sum_{i,j} |a_{ij}|^2} \quad (\text{norma di Frobenius}).$$

b) Se considero A come operatore, uso l'azione di A sui vettori:

$$\|A\|_p = \sup_{x \neq 0} \frac{\|Ax\|_p}{\|x\|_p}.$$

Osservo che $\frac{1}{\|x\|_p} = \alpha \in \mathbb{R}$, per cui la posso portare dentro $\|Ax\|$ e ottengo:

$$\|A\|_p = \sup_{x \neq 0} \left\| A \frac{x}{\|x\|_p} \right\|_p.$$

Inoltre, il vettore $\frac{x}{\|x\|_p}$ ha norma 1, quindi:

$$\|A\|_p = \sup_{\|x\|_p=1} \|Ax\|_p.$$

Nel caso $p = 2$, si tratta quindi di prendere tutti i x tali che $\|x\|_2 = 1$. In $\mathbb{R}^2$ questo è un cerchio unitario, e la sua immagine attraverso A risulta in un'ellisse. Il valore di $\|A\|_2$ è il valore del semiasse maggiore dell'ellisse.

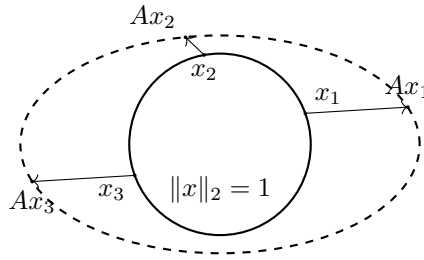


Figura 2: Il cerchio unitario $\|x\|_2 = 1$ viene trasformato in un'ellisse dal mapping $x \mapsto Ax$.

Esempio 1.6. Calcolo $\|A\|_1$. Devo calcolare dunque:

$$\begin{aligned}
 \|Ax\|_1 &= \sum_i |(Ax)_i| = \sum_i \left| \sum_j a_{ij} x_j \right| \\
 &\leq \sum_i \sum_j |a_{ij}| |x_j| \quad (\text{dato che } |a+b| \leq |a| + |b|) \\
 &= \sum_j \sum_i |a_{ij}| |x_j| \\
 &\leq \max_j \sum_i |a_{ij}| \underbrace{\sum_j |x_j|}_{=\|x\|_1}
 \end{aligned}$$

Per cui:

$$\|A\|_1 \leq \max_j \sum_i |a_{ij}| \|x\|_1$$

Cioè

$$\frac{\|Ax\|_1}{\|x\|_1} \leq \max_j \sum_i |a_{ij}| \quad \forall x \neq 0$$

Quindi:

$$\Rightarrow \|A\|_1 = \sup_{x \neq 0} \frac{\|Ax\|_1}{\|x\|_1} \leq \max_j \sum_i |a_{ij}|$$

Se trovo un $\bar{x}$ tale che $\|A\bar{x}\| = \max_j \sum_i |a_{ij}| \|\bar{x}\|$ allora:

$$\|A\|_1 = \max_j \sum_i |a_{ij}|$$

Esempio 1.7. Calcolo $\|A\|_1$ per:

$$A = \begin{bmatrix} 3 & 2 & -1 \\ -2 & 1 & 10 \\ 1 & 0 & 1 \\ 0 & -2 & 2 \end{bmatrix}$$

Calcolo $\sum_i |a_{ij}|$ con $j = 1, 2, 3$:

$$\begin{aligned}
 \sum_i |a_{i1}| &= |3| + |(-2)| + |1| + |0| = 6 \\
 \sum_i |a_{i2}| &= |2| + |1| + |0| + |-2| = 5 \\
 \sum_i |a_{i3}| &= |-1| + |10| + |1| + |2| = 14
 \end{aligned}$$

Per cui:

$$\|A\|_1 = \max\{6, 5, 14\} = 14$$

Verifichiamo che $\bar{x}$ esiste: chiamo k la colonna di A che realizza $\max_j \sum_i |a_{ij}| = \sum_i |a_{ik}|$. Prendo:

$$\bar{x}_i = \begin{cases} 1 & \text{se } a_{ij} \geq 0 \\ -1 & \text{se } a_{ij} < 0 \end{cases} \quad \sum_i a_{ik} \bar{x}_i = \sum_i |a_{ik}| x_k$$

Osservazione 1.4. $\|A\|_p$ è norma di matrice indotta della norma di vettore. Elenchiamo le proprietà:

- $\|Ax\|_p \leq \|A\|_p \|x\|_p \quad \forall x \quad \forall p$ infatti

$$\|A\| = \sup_{x \neq 0} \frac{\|Ax\|}{\|x\|} \implies \|A\|_p \geq \frac{\|Ax\|_p}{\|x\|_p} \quad \forall x \neq 0$$

- $\|AB\| \leq \|A\| \|B\| \quad \forall A, B$. Questa proprietà è conseguenza della prima.

Residuo ed errore (sistemi lineari)

Si ha un sistema lineare $A\underline{x} = \underline{b}$. Calcoliamo $\tilde{x}$ come soluzione approssimata. Si vorrebbe conoscere l'errore $\underline{e} = \underline{x} - \tilde{x}$, ma possiamo solo calcolare il residuo.

$$\|e\| = \|\underline{x} - \tilde{x}\| = \|A^{-1}A(\underline{x} - \tilde{x})\| \leq \|A^{-1}(A\underline{x} - A\tilde{x})\| = \|A^{-1}(b - A\tilde{x})\| = \|-A^{-1}r\| = \|A^{-1}r\| \leq \|A^{-1}\| \|r\|$$

Per cui si è ottenuto che:

$$\|e\| \leq \|A^{-1}\| \|r\|$$

Numero di condizionamento

Si consideri un sistema lineare $A\underline{x} = \underline{b}$. Supponiamo che $\underline{b}$ sia perturbato da degli errori e che quindi il vettore a nostra disposizione è un vettore $\tilde{b}$. Nella migliore delle ipotesi risolvo il sistema:

$$A\tilde{x} = \tilde{b}$$

Ci chiediamo quanto sia grande $\|x - \tilde{x}\|$ rispetto a $b - \tilde{b}$. Vorrei dunque calcolare:

$$\frac{\|x - \tilde{x}\|}{\|x\|} \quad \text{rispetto a} \quad \frac{\|b - \tilde{b}\|}{\|b\|}$$

Per il numeratore:

$$\|x - \tilde{x}\| = \|A^{-1}A(\underline{x} - \tilde{x})\| = \|A^{-1}(b - \tilde{b})\| \implies \|x - \tilde{x}\| \leq \|A^{-1}\| \|b - \tilde{b}\|$$

Per il denominatore:

$$b = Ax \implies \|b\| = \|Ax\| \leq \|A\| \|x\| \implies \frac{1}{\|x\|} \leq \frac{\|A\|}{\|b\|}$$

Per cui:

$$\frac{\|x - \tilde{x}\|}{\|x\|} \leq \underbrace{\|A\| \|A^{-1}\|}_{\text{numero di condizionamento di } A} \frac{\|b - \tilde{b}\|}{\|b\|}$$

Chiamo $K_p(A) = \|A\|_p \|A^{-1}\|_p$ il numero di condizionamento di A rispetto alla norma p .

Esempio 1.8. Vediamo quanto è il numero di condizionamento di $\mathbb{I}$.

$$\|\mathbb{I}\| = \max_{x \neq 0} \frac{\|\mathbb{I}x\|_p}{\|x\|_p} = \max_{x \neq 0} \frac{\|x\|_p}{\|x\|_p} = 1$$

Analogamente:

$$\|\mathbb{I}^{-1}\| = \max_{x \neq 0} \frac{\|\mathbb{I}^{-1}x\|_p}{\|x\|_p} = \max_{x \neq 0} \frac{\|x\|_p}{\|x\|_p} = 1$$

Per cui:

$$K_p(\mathbb{I}) = \|A\|_p \|A^{-1}\|_p = 1$$

Per la matrice A generica (invertibile) si ha:

$$\mathbb{I} = AA^{-1} \implies \|\mathbb{I}\| = \|AA^{-1}\| \leq \|A\|\|A^{-1}\| = \|A\|\|A^{-1}\|$$

Per cui:

$$\forall A \quad K_p(A) \geq 1$$

L'errore nei dati $\|b - \tilde{b}\|$ tipicamente viene amplificato nella differenza $\|x - \tilde{x}\|$ di un fattore $K_p(A)$. Lavorare in doppia precisione ci permetterà di ridurre l'errore relativo di un fattore 10^{-16} . Se ho $\frac{\|b - \tilde{b}\|}{\|b\|} \sim 10^{-16}$ e $K_p(A) \sim 10^8$ allora l'errore sulla soluzione potrà essere di ordine:

$$\frac{\|x - \tilde{x}\|}{\|x\|} \sim 10^{-16} \cdot 10^8 = 10^{-8}$$

Tuttavia se l'errore nei dati è più grande, come nel caso dei dati sperimentali, è normale avere:

$$\frac{\|b - \tilde{b}\|}{\|b\|} \sim 10^{-2}$$

e magari $K(A) \sim 10^8$. In questo caso l'errore sulla soluzione potrà essere di ordine:

$$\frac{\|x - \tilde{x}\|}{\|x\|} \sim 10^{-2} \cdot 10^8 = 10^6$$

Osservazione 1.5. Diremo che A è mal condizionata se $K_p(A) \gtrsim 10^8$. Ma dipende anche da quanto sono affidabili i dati.

Esempio 1.9. Consideriamo la matrice:

$$A = \begin{bmatrix} 10^{20} & 0 \\ 0 & 10^{20} \end{bmatrix} \Rightarrow A^{-1} = \begin{bmatrix} 10^{-20} & 0 \\ 0 & 10^{-20} \end{bmatrix}$$

Per cui abbiamo che:

$$\|A\|_1 = 10^{20} \quad \|A^{-1}\|_1 = 10^{-20} \quad \Rightarrow \quad K_1(A) = 1$$

Vediamo il caso di una matrice mal condizionata:

$$A = \begin{bmatrix} 10^{20} & 0 \\ 0 & 10^{-20} \end{bmatrix} \Rightarrow A^{-1} = \begin{bmatrix} 10^{-20} & 0 \\ 0 & 10^{20} \end{bmatrix}$$

Per cui abbiamo che:

$$\|A\|_1 = 10^{20} \quad \|A^{-1}\|_1 = 10^{20} \quad \Rightarrow \quad K_1(A) = 10^{40}$$

Questa è una matrice mal condizionata: sono fortemente anisotrope.

Esempio 1.10. Calcoliamo il numero di condizionamento delle matrici ortogonali. Utilizzo la norma 2.

$$\|Qx\|_2 = \|x\|_2$$

Le matrici ortogonali non modificano la lunghezza dei vettori.

$$Q = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix}$$

Per cui:

$$\sup_{x \neq 0} \frac{\|Qx\|_2}{\|x\|_2} = 1$$

$$E \quad \|Q^{-1}\|_2 = \|Q^T\|_2 = 1 \implies K_2(Q) = 1.$$

1.4.1 Autovalori e autovettori

Definizione 1.4 (Autovalore). Data una matrice $A \in \mathbb{R}^{n \times n}$, si dice che $\lambda \in \mathbb{R}$ è un autovalore di A se esiste un vettore $\underline{x} \in \mathbb{R}^n \setminus \{0\}$ tale che:

$$A\underline{x} = \lambda\underline{x}$$

Il vettore $\underline{x}$ si dice autovettore di A relativo all'autovalore λ .

Per cui gli autovettori sono direzioni invarianti rispetto all'azione di A .

$$A\underline{x} - \lambda\underline{x} = 0 \implies (A - \lambda I)\underline{x} = 0$$

Questo ci dice che $\ker(A - \lambda I) \neq \{0\}$. Per cui la matrice $A - \lambda I$ non è invertibile e quindi $\det(A - \lambda I) = 0$. Calcolando $\det(A - \lambda I)$ otteniamo un polinomio in λ

$$p(\lambda) = \det(A - \lambda I) = 0$$

Gli autovalori sono le radici di questo polinomio, che si chiama polinomio caratteristico di A . Questo polinomio ha grado n , per cui ha n radici, che non sono necessariamente distinte. Dico che λ_k ha molteplicità algebrica k se λ_k è radice del polinomio caratteristico k volte.

$$\sum_{\lambda_k} k = n$$

$p(\lambda)$ ha coefficienti reali. Se $\lambda \in \mathbb{C}$ anche $\bar{\lambda}$ è autovalore (il suo complesso coniugato), cioè se ci sono autovalori complessi questi vengono sempre a coppie.

Per definizione, ad ogni autovalore corrisponde almeno un autovettore. Elenchiamo le proprietà degli autovalori:

1. Supponiamo che λ_1 e λ_2 siano autovalori distinti di A con autovettori $\underline{x}_1$ e $\underline{x}_2$. Allora $\underline{x}_1$ e $\underline{x}_2$ sono linearmente indipendenti. Cioè se:

$$c_1\underline{x}_1 + c_2\underline{x}_2 = 0 \implies c_1 = c_2 = 0$$

Per assurdo suppongo che $c_1 \neq 0$. Allora

$$\underline{x}_1 = -\frac{c_2}{c_1}\underline{x}_2$$

Applico:

$$\begin{aligned} 0 &= A(c_1\underline{x}_1 + c_2\underline{x}_2) = c_1A\underline{x}_1 + c_2A\underline{x}_2 \\ &= c_1\lambda_1\underline{x}_1 + c_2\lambda_2\underline{x}_2 \\ \text{Sostituisco } \underline{x}_1 &: \left(-\frac{c_2}{c_1}c_1\lambda_1 + c_2\lambda_2\right)\underline{x}_2 = 0 \\ \implies c_2(\lambda_2 - \lambda_1)\underline{x}_2 &= 0 \end{aligned}$$

Se $c_2 = 0$ allora $c_1\underline{x}_1 = 0 \implies c_1 = 0$ e i vettori sono linearmente dipendenti. E dato che abbiamo supposto che $\lambda_1 \neq \lambda_2$ e $\underline{x}_2 \neq 0$ allora non può essere $c_2 \neq 0$.

2. Se una matrice A ha n autovalori distinti allora ha n autovettori $\underline{x}_1, \dots, \underline{x}_n$ che sono linearmente indipendenti. Per cui $\underline{x}_1, \dots, \underline{x}_n$ formano una base di $\mathbb{R}^n$. Chiamo $X = [\underline{x}_1, \dots, \underline{x}_n]$ la matrice che ha per colonne gli autovettori. Allora:

$$\text{rank}(X) = n \implies \det(X) \neq 0 \implies X \text{ è invertibile}$$

So che $A\underline{x}_i = \lambda_i\underline{x}_i$. Allora:

$$AX = XD \quad D = \text{diag}(\lambda_1, \dots, \lambda_n)$$

(Moltiplicazione a destra per una matrice diagonale). E dato che X è invertibile posso moltiplicare a destra per X^{-1} e ottengo:

$$A = XDX^{-1}$$

Questa è la diagonalizzazione di A .

3. Non tutte le matrici sono diagonalizzabili. Se λ ha molteplicità algebrica > 1 allora:

- $\exists$ certamente un autovettore per λ . Tuttavia non è detto che ci sia un altro autovettore linearmente indipendente per λ .

$$\begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} \quad \lambda = 2 \text{ con molteplicità algebrica } 2$$

e dato che A è diagonale è quindi diagonalizzabile.

$$\begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix} \quad \lambda = 2 \text{ con molteplicità algebrica } 2$$

e c'è un autovettore linearmente indipendente.

4. Se A è simmetrica allora:

- Tutti gli autovalori sono reali;
- Tutti gli autovettori sono ortogonali.
- A è diagonalizzabile tramite una matrice ortogonale.

Vediamo (a): suppongo che λ sia autovalore di A con autovettore $\underline{x}$. Calcolo:

$$\langle \underline{x}, A\underline{x} \rangle = \langle \underline{x}, \lambda \underline{x} \rangle = \lambda \langle \underline{x}, \underline{x} \rangle = \lambda \|\underline{x}\|^2$$

Ma anche:

$$\langle A^T \underline{x}, \underline{x} \rangle = \langle A\underline{x}, \underline{x} \rangle = \langle \lambda \underline{x}, \underline{x} \rangle = \bar{\lambda} \|\underline{x}\|^2$$

dove $\bar{\lambda}$ è il complesso coniugato di λ e $\langle x, y \rangle = \sum \bar{x}_i y_i$. Ricordando che:

$$\langle Ax, y \rangle = \langle x, A^T y \rangle$$

quindi:

$$\langle x, \lambda x \rangle = \sum_i \bar{x}_i \lambda x_i = \lambda \sum_i \bar{x}_i x_i = \lambda \sum_i |x_i|^2$$

Mentre:

$$\langle \lambda x, x \rangle = \sum_i \lambda \bar{x}_i x_i = \bar{\lambda} \sum_i \bar{x}_i x_i = \bar{\lambda} \sum_i |x_i|^2$$

Quindi:

$$\lambda \|x\|^2 = \bar{\lambda} \|x\|^2$$

per cui $\lambda = \bar{\lambda}$ e quindi $\lambda \in \mathbb{R}$.

Vediamo (b): Se $\lambda_1 \neq \lambda_2$ (con A simmetrica) allora $\underline{x}_1$ e $\underline{x}_2$ sono ortogonali.

$$\langle x_2, Ax_1 \rangle = \langle x_2, \lambda_1 x_1 \rangle = \lambda_1 \langle x_2, x_1 \rangle$$

Mentre per simmetria:

$$\langle Ax_2, x_1 \rangle = \langle \lambda_2 x_2, x_1 \rangle = \lambda_2 \langle x_2, x_1 \rangle$$

dunque λ_1, λ_2 sono reali. E dato che $\lambda_1 \neq \lambda_2$ allora:

$$(\lambda_1 - \lambda_2) \langle x_2, x_1 \rangle = 0 \implies \langle x_2, x_1 \rangle = 0 \implies x_1 \perp x_2$$

5. Autovalori della matrice inversa: supponiamo che A abbia un autovalore λ con autovettore $\underline{x}$. Allora A^{-1} , se esiste, ha un autovalore $\frac{1}{\lambda}$ con autovettore $\underline{x}$. Infatti:

$$A\underline{x} = \lambda\underline{x} \implies A^{-1}A\underline{x} = A^{-1}\lambda\underline{x} \implies \underline{x} = \lambda A^{-1}\underline{x} \implies A^{-1}\underline{x} = \frac{1}{\lambda}\underline{x}$$

Quindi gli autovalori di A e A^{-1} sono inversi. Se la matrice è invertibile allora gli autovalori sono tutti non nulli.

6. Norma 2 di matrici simmetriche: se A è simmetrica allora:

- Ha n autovalori reali;
- Ha n autovettori ortogonali fra loro; Chiamo $w_1, \dots, w_n$ gli autovettori di A (che sono ortogonali fra loro, quindi indipendenti, e sono una base di $\mathbb{R}^n$). Dunque qualunque vettore $\underline{x} \in \mathbb{R}^n$ si può scrivere come combinazione lineare dei w_i , cioè $\forall \underline{x} \in \mathbb{R}^n \quad \exists \alpha_1, \dots, \alpha_n \in \mathbb{R}$ tale che

$$\underline{x} = \alpha_1 w_1 + \dots + \alpha_n w_n = \sum_{i=1}^n \alpha_i w_i$$

Per calcolare $\|A\|_2$ devo calcolare:

$$\|A\|_2 = \max_{\underline{x} \neq 0} \frac{\|A\underline{x}\|_2}{\|\underline{x}\|_2}$$

$$\text{con } \|A\underline{x}\| = \sqrt{\langle A\underline{x}, A\underline{x} \rangle}$$

$$A\underline{x} = A\left(\sum_i \alpha_i w_i\right) = \underbrace{\sum_i \alpha_i A w_i}_{A \text{ è un operatore lineare}} = \sum_i \alpha_i \lambda_i w_i$$

Per cui:

$$\begin{aligned} \langle A\underline{x}, A\underline{x} \rangle &= \left\langle \sum_i \alpha_i \lambda_i w_i, \sum_j \alpha_j \lambda_j w_j \right\rangle \\ &= \sum_i \sum_j \alpha_i \bar{\alpha}_j \lambda_i \bar{\lambda}_j \underbrace{\langle w_i, w_j \rangle}_{\substack{0 \text{ se } i \neq j \text{ e } 1 \text{ se } i=j}} \\ &= \sum_i \alpha_i \bar{\alpha}_i \lambda_i \bar{\lambda}_i = \sum_i \alpha_i^2 \lambda_i^2 \end{aligned}$$

Ricordando che $\lambda_i \in \mathbb{R}$, per cui $\bar{\lambda}_i = \lambda_i$. E dato che $w_i \in \mathbb{R}^n$ allora $\alpha_i \in \mathbb{R}$. Per cui:

$$\langle A\underline{x}, A\underline{x} \rangle = \sum_i \alpha_i^2 \lambda_i^2$$

$$\|A\underline{x}\|_2 = \sum_i |\alpha_i|^2 \lambda_i^2 \leq \max_i |\lambda_i|^2 \sum_i |\alpha_i|^2 = \max_i |\lambda_i|^2 \|\underline{x}\|_2^2$$

dato che $\|\underline{x}\|_2^2 = \langle \underline{x}, \underline{x} \rangle = \langle \sum_i \alpha_i w_i, \sum_j \alpha_j w_j \rangle = \sum_i \alpha_i^2 \langle w_i, w_i \rangle = \sum_i \alpha_i^2$. Questo vale se gli autovettori sono **ortonormali** tra di loro. Per cui:

$$\frac{\|A\underline{x}\|}{\|\underline{x}\|} \leq \max_i |\lambda_i|$$

Quindi $\|A\|_2 \leq$ del modulo dell' autovalore di **modulo** massimo. Per uguagliare occorre trovare un vettore $\underline{x}$ tale che:

$$\|A\underline{x}\|^2 = \max_i |\lambda_i|^2 \|\underline{x}\|^2$$

Prendo $\underline{x} = w_k$ con $|\lambda_k| = \max_i |\lambda_i|$. Allora il numero di condizionamento di una matrice simmetrica è:

$$K_2(A) = \|A\|_2 \|A^{-1}\|_2 = \max_i |\lambda_i| \max_i \frac{1}{|\lambda_i|} = \max_i |\lambda_i| \frac{1}{\min_i |\lambda_i|} = \frac{\max_i |\lambda_i|}{\min_i |\lambda_i|}$$

2 Numeri di macchina

Noi usiamo un sistema posizionale in base 10.

$$x = 123 = 3 \cdot 10^0 + 2 \cdot 10^1 + 1 \cdot 10^2$$

Un numero in base 10 è così descritto:

$$x = a_1 a_2 \dots a_t$$

dove $a_i = 0, 1, \dots, 9$ In base 16:

$$x = a_1 a_2 \dots a_t$$

dove $a_i = 0, 1, \dots, 9, A, B, C, D, E, F$. Per esempio $x = 3A0B$:

$$x = B \cdot 16^0 + 0 \cdot 16^1 + A \cdot 16^2 + 3 \cdot 16^3$$

In base 2 $a_i = 0, 1$

$$(1111)_2 = 1 \cdot 2^0 + 1 \cdot 2^1 + 1 \cdot 2^2 + 1 \cdot 2^3 = (1 + 2 + 4 + 8)_{10} = 2^4 - 1$$

Osserviamo che 1111 è il numero più grande che si può scrivere in base 2 con 4 posizioni a disposizione.

$$\underbrace{11 \dots 1}_{k \text{ uni}} = 2^k - 1 = 1 \underbrace{00 \dots 0}_{k \text{ zeri}} - 1$$

$$\begin{array}{llll} 123 : 2 = 61 & \text{resto } 1 & \implies & 123 = 2 \cdot 61 + 1 \\ 61 : 2 = 30 & \text{resto } 1 & \implies & = 2 \cdot (2 \cdot 30 + 1) + 1 \\ 30 : 2 = 15 & \text{resto } 0 & \implies & = 2^2 \cdot (2 \cdot 15) + 2 + 1 \\ 15 : 2 = 7 & \text{resto } 1 & \implies & = 2^3 \cdot (2 \cdot 7 + 1) + 2 + 1 \\ 7 : 2 = 3 & \text{resto } 1 & \implies & = 2^4 \cdot (2 \cdot 3 + 1) + 2^3 + 2 + 1 \\ 3 : 2 = 1 & \text{resto } 1 & \implies & = 2^5 \cdot (2 \cdot 1 + 1) + 2^4 + 2^3 + 2 + 1 \end{array}$$

Per cui:

$$123 = 2^6 + 2^5 + 2^4 + 2^3 + 2^1 + 2^0 = (1111011)_2$$

2.1 Numeri interi

$$x = a_1 a_2 \dots a_t$$

Abbiamo visto il caso di `uint8` per le immagini, che sta per unsigned-integers a 8bit=1 byte, dove ciascun bit è una cifra binaria (0 o 1). Il massimo memorizzabile nel sistema `uint8` è:

$$11111111 = 2^8 - 1 = 256 - 1 = 255$$

Per cui in `uint8` posso memorizzare esattamente interi da 0 a 255. I numeri `int` occupano 4 byte = 4*8 bit = 32 bit. Quindi il massimo intero che possono memorizzare è:

$$2^{31} - 1 \simeq 2.15 \cdot 10^9$$

(1 bit è riservato al segno) Vi sono poi gli `short int` che occupano 2 byte e i `long int` che occupano 8 byte (il suo massimo è dell'ordine di 10^{18}). Gli interi possono essere memorizzati esattamente, purché $x \in \mathbb{Z}$ e $|x| < \text{max int}$.

Se $|x| > \text{max int}$ si dice che si va in overflow, per cui x viene memorizzato come `Inf`. Valgono le seguenti proprietà:

- $2 \cdot \text{inf} = \text{inf}$
- $1 + \text{inf} = \text{inf}$
- $\text{inf} - \text{inf} = \text{NaN}$ che è indeterminato

2.2 Numeri reali

Abbiamo infiniti modi equivalenti per scrivere un numero reale:

$$\begin{aligned} x &= 123.02 \\ &= 12.302 \cdot 10 \\ &= 0.12302 \cdot 10^3 \\ &= 12302.0 \cdot 10^{-2} \end{aligned}$$

Tuttavia per evitare ambiguità si usa la notazione detta **standardizzata**

$$x = 0.a_1a_2 \dots \cdot \beta^q$$

dove $a_1 \neq 0$. Un'altra ambiguità si potrebbe avere per:

$$0.999 \dots = 1.0 = 0.1 \cdot \beta^1$$

Per $\beta = 2$ ogni numero in forma standardizzata sarà:

$$x = 0.101 \dots 0 \cdot \beta^q \quad \text{dire che } a_1 \neq 0 \implies a_1 = 1$$

Quindi a_1 non viene memorizzato, risparmiando così un bit.

$x \in \mathbb{R}$ in forma standardizzata:

$$x = \pm 0.a_1a_2 \dots a_t a_{t+1} \dots \cdot \beta^q$$

La parte decimale si chiama mantissa, q si chiama esponente $\in \mathbb{Z}$.

Otteniamo i cosiddetti numeri floating-point (o numeri in virgola mobile) che sono caratterizzati dall'avere un numero massimo di cifre per q e un numero massimo di cifre per t (# cifre per la mantissa).

- **single precision:** 32 bit (4 byte):

$$1\text{bit segno} + 8\text{bit esponente} + 23\text{bit mantissa}$$

- **double precision:** 64 bit (8 byte):

$$1\text{bit segno} + 11\text{bit esponente} + 52\text{bit mantissa}$$

Vediamo i numeri massimi che possiamo rappresentare.

- **single precision:**

$$q_{\max} = 2^7 - 1 = 127 \quad (1 \text{ bit è sempre riservato al segno})$$

per cui il numero massimo è nell'ordine di $2^{127} \simeq 10^{38}$.

- **double precision:**

$$q_{\max} = 2^{1023} \simeq 10^{308}$$

quindi se $|x| > 10^{308}$ si va in overflow. Tuttavia se $|x| < 10^{-308}$ si va in underflow e il numero viene approssimato a 0. Se $10^{-308} \leq |x| \leq 10^{308}$ allora il numero è rappresentabile. Chiamo $\tilde{x} = fl(x)$ il numero floating-point più vicino a x . Supponiamo che:

$$x = 0.a_1a_2 \dots a_t a_{t+1} \dots \cdot \beta^q$$

$\tilde{x}$ può essere quindi uguale a:

$$\tilde{x} = 0.a_1a_2 \dots a_t \cdot \beta^q \quad (\text{Troncamento})$$

oppure:

$$\tilde{x} = 0.a_1a_2 \dots \tilde{a}_t \cdot \beta^q \quad (\text{Arrotondamento})$$

Se $x = 0.1231$ e $t = 3$ allora $\tilde{x} = 0.123$ sia per arrotondamento che per troncamento. Se invece $x = 0.1235$ allora:

$$\tilde{x} = 0.123 \quad (\text{Troncamento})$$

$$\tilde{x} = 0.124 \quad (\text{Arrotondamento})$$

Siamo interessati a valutare **l'errore di rappresentazione**.

- Per troncamento:

$$|x - \tilde{x}| = |0.a_1a_2 \dots a_t a_{t+1} \dots \cdot \beta^q - 0.a_1a_2 \dots a_t \cdot \beta^q| = |00 \dots 0a_{t+1} \dots| \cdot \beta^q$$

Quindi

$$|0.a_{t+1} \dots| \cdot \beta^{q-t} \leq \beta^{q-t}$$

- Per arrotondamento:

$$|x - \tilde{x}| \leq \frac{1}{2} \beta^{q-t}$$

L'errore dipende da quanto è grande il numero attraverso β^q e ci dice anche che i numeri floating-point non sono uniformemente spazati.

L'errore relativo è invece dato da:

$$\left| \frac{x - \tilde{x}}{x} \right| = \frac{|x - \tilde{x}|}{|x|} \leq \frac{\beta^{q-t}}{0.a_1a_2 \dots a_t a_{t+1} \dots \cdot \beta^q}$$

E ricordando che $x = 0.a_1a_2 \dots \geq 0.1 = \beta^{-1}$ allora:

$$\frac{|x - \tilde{x}|}{|x|} \leq \frac{\beta^{-t}}{\beta^{-1}} = \beta^{1-t} \quad (\text{troncamento})$$

$$\frac{|x - \tilde{x}|}{|x|} \leq \frac{\beta^{1-t}}{2} \quad (\text{arrotondamento})$$

Errore relativo nel caso di arrotondamento viene detto **precisione di macchina** (*eps*), ed è caratteristica del sistema floating-point usato.

In precisione singola abbiamo che $\beta = 2$ $t = 23$ per cui:

$$\text{eps} = \frac{1}{2} 2^{1-23} = 2^{-23} \simeq 10^{-7}$$

In precisione doppia abbiamo che $\beta = 2$ $t = 52$ per cui:

$$\text{eps} = \frac{1}{2} 2^{1-52} = 2^{-52} \simeq 2.2 \cdot 10^{-16}$$

Chiamo $\mathcal{F}$ l'insieme dei numeri floating-point. Per cui $\mathcal{F} = \mathcal{F}(\beta, t, q_{\max})$. Questo insieme ha cardinalità finita.

2.3 Operazioni aritmetiche con numeri floating-point

Sia $\mathcal{F} = \mathcal{F}(\beta = 10, t = 3, q_{\max} = 10)$. Abbiamo i numeri:

$$x = 0.432 \cdot 10^3 = \tilde{x} \quad y = 0.710 \cdot 10^{-2} = \tilde{y}$$

Supponiamo di voler calcolare $x \cdot y$. Allora:

$$x \cdot y = \tilde{x} \cdot \tilde{y} = (0.432)(0.710) \cdot 10^{3-2} = 0.30672 \cdot 10^1$$

per cui:

$$fl(\tilde{x} \cdot \tilde{y}) = 0.307 \cdot 10^1$$

Per cui il sistema floating-point non è chiuso rispetto all'operazione di moltiplicazione. Procediamo ora a calcolare la somma $x + y$:

$$x + y = \tilde{x} + \tilde{y} = 0.432 \cdot 10^3 + 0.710 \cdot 10^{-2} = 0.432 \cdot 10^3 + 0.00000710 \cdot 10^3 = 0.43200710 \cdot 10^3$$

per cui:

$$fl(x + y) = 0.432 \cdot 10^3$$

quindi il sistema non è chiuso neanche rispetto all'operazione di somma.

Osservazione 2.1. Se $x, y \in \mathbb{R}$ e $x, y > 0$ allora:

$$x + y > x \quad e \quad x + y > y$$

Ma nell'aritmetica floating-point:

$$fl(x + y) \geq fl(x) \quad e \quad fl(x + y) \geq fl(y)$$

Per cui non vale più nemmeno la proprietà commutativa della somma. Inoltre:

$$fl(x + y) = fl(x) \not\Rightarrow y = 0$$

Il numero più piccolo tale che $1 + x > 1$ è $x = \mathbf{eps}$.

Sia $x \in \mathbb{R}$ e $\tilde{x} \in \mathcal{F}$ t.c. $fl(x) = \tilde{x}$. Solitamente si scrive:

$$\tilde{x} = x(1 + \delta_x) \quad \text{con} \quad \delta_x \in \mathbb{R}$$

E l'errore è:

$$\left| \frac{x - \tilde{x}}{x} \right| = \frac{|x - x - x\delta_x|}{|x|} = |\delta_x| \leq \mathbf{eps}$$

L'errore nel prodotto è dato da:

$$\begin{aligned} \left| \frac{xy - \tilde{x}\tilde{y}}{xy} \right| &= \frac{|xy - x(1 + \delta_x)y(1 + \delta_y)|}{|xy|} = \\ &= \frac{|x\delta_x y\delta_y + xy(\delta_x\delta_y)|}{|xy|} = \\ &= \frac{|xy - (x - x\delta_x)(y - y\delta_y)|}{|xy|} = \\ &= \frac{|xy - xy - xy\delta_x - xy\delta_y + xy\delta_x\delta_y|}{|xy|} = \delta_x + \delta_y + \delta_x\delta_y \leq 3\mathbf{eps} \end{aligned}$$

L'errore nella somma è dato da:

$$\begin{aligned} \left| \frac{x + y - \tilde{x} - \tilde{y}}{x + y} \right| &= \frac{|x + y - x(1 + \delta_x) - y(1 + \delta_y)|}{|x + y|} = \\ &= \frac{|x\delta_x + y\delta_y|}{|x + y|} \leq \frac{|x|}{|x + y|}|\delta_x| + \frac{|y|}{|x + y|}|\delta_y| \end{aligned}$$

Questo risultato non è $O(\mathbf{eps})$ se $|x + y|$ è piccolo. Se sottraggo due numeri quasi uguali l'errore nella sottrazione può essere molto grande $\sim \frac{1}{|x + y|}$. Questo prende il nome di **catastrophic cancellation**.

Esempio 2.1. Si vuole calcolare e^x . Sappiamo che:

$$e^x = \sum_{k=1}^{\infty} a_k \quad a_k = \frac{x^k}{k!}$$

Devi calcolare una somma finita, dato che non possiamo calcolare infiniti termini.

$$S^n = \sum_{k=1}^n a_k \quad \lim_{n \rightarrow \infty} S^n = e^x$$

Ci chiediamo quanto deve essere grande n e come calcolare a_k : occorre chiaramente calcolare x^k e $k!$ per ogni k . Osserviamo che $k!$ cresce molto rapidamente, per cui il termine generale a_k decresce molto rapidamente. Un'ulteriore osservazione è che è molto costoso calcolare $k!$ per ogni k , per cui si preferisce calcolare a_k come:

$$a_k = \frac{x^k}{k!} = \frac{x}{k} \cdot a_{k-1} \quad a_0 = 1$$

Algorithm 1 Calcolo di a^x

```

1: function A_K( $x, k$ )
2:    $a_0 \leftarrow 1$ 
3:   for  $i = 1$  to  $k$  do
4:      $a_i \leftarrow \frac{x}{i} \cdot a_{i-1}$ 
5:   end for
6:   return  $a_k$ 
7: end function

```

Quindi per calcolare e^x possiamo usare il seguente algoritmo:

Algorithm 2 Calcolo di e^x

```

1: function EXP( $x, n$ )
2:    $a_0 \leftarrow 1$ 
3:    $S_n \leftarrow 0$ 
4:   for  $k = 1$  to  $n$  do
5:      $a_k \leftarrow \text{A\_K}(x, k)$ 
6:      $S_n \leftarrow S_n + a_k$ 
7:   end for
8:   return  $S_n$ 
9: end function

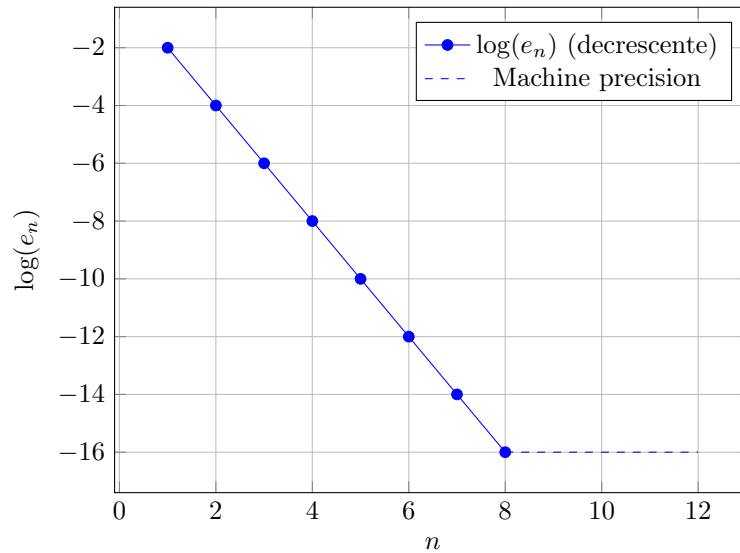
```

Per controllare il codice posso stampare S_n e $\exp(x)$ e valutare $S_n - \exp(x)$. Mi aspetto che:

$$e_n = |S_n - \exp(x)| \rightarrow 0 \quad \text{per } n \rightarrow \infty$$

Osservo che non riesco ad andare sotto la machine precision.

Andamento di $\log(e_n)$ in funzione di n



Decidiamo di fermarci quando $|S_n - S_{n-1}| < \varepsilon$.

$$\forall \varepsilon > 0 \quad \exists N \text{ t.c. } |S_n - S_{n-1}| < \varepsilon \quad \forall n \geq N \quad (1)$$

In analisi 2 avevamo visto la successione di Cauchy:

$$\forall \varepsilon > 0 \quad \exists N \text{ t.c. } |S_n - S_m| < \varepsilon \quad \forall n, m \geq N \quad (2)$$

Se la mia serie converge monotonicamente allora basta (1) per verificare la convergenza anche se è meno generale di (2). Allora possiamo sostituire il ciclo **for** con un ciclo **while**:

```

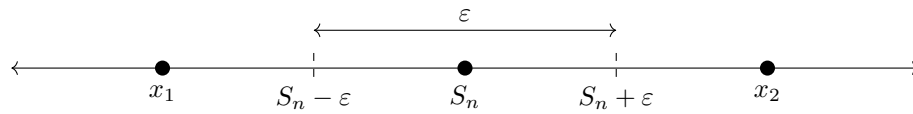
1: while Test == False do
2:   Calcolo  $S_n$  a partire da  $S_{n-1}$  e aggiorniamo il Test
3: end while
4: return  $S_n$ 

```

Dove $\text{Test} = S_n - S_{n-1} < \varepsilon$, con $\varepsilon > \text{eps}$. Tuttavia $S_n - S_{n-1}$ implica una sottrazione tra due numeri quasi uguali, ma notiamo che possiamo evitarlo, infatti:

$$S_n - S_{n-1} = a_n \quad \text{Test} = |a_n| < \varepsilon$$

In aggiunta un test del tipo $|S_n - S_{n-1}| < \varepsilon$ è un test assoluto, quindi se S_n è molto grande allora potremmo avere i numeri in virgola mobile più distanti da S_n rispetto a ε .



Dove x_1, x_2 sono numeri floating-point intorno a S_n . Se $\varepsilon < x_2 - x_1$ potresti non trovare approssimazioni di S_n con numeri floating-point che soddisfano il test. Devo usare un test relativo:

$$\left| \frac{S_n - S_{n-1}}{S_{n-1}} \right| < \varepsilon \quad \text{cioè} \quad |S_n - S_{n-1}| < \varepsilon |S_{n-1}|$$

Inoltre è utile fissare un numero interno di iterazioni massimo **itmax**, in questo modo evitiamo un ciclo infinito.

```

1: while Test == False and it < itmax do
2:   Calcolo  $S_n$  a partire da  $S_{n-1}$  e aggiorniamo il Test
3:   it ← it + 1
4: end while
5: return  $S_n$ 

```

Se $x \in \mathbb{R}$ abbiamo che $\tilde{x} = fl(x)$ dove $\tilde{x}$ è la rappresentazione floating-point di x .

$$x = 0.a_1a_2 \dots a_t a_{t+1} \dots \cdot \beta^q$$

Ricordiamo che:

$$|x - \tilde{x}| \leq \frac{1}{2} \beta^{q-t} \quad \text{dipende da } x$$

mentre:

$$\left| \frac{x - \tilde{x}}{x} \right| \leq \frac{1}{2} \beta^{1-t} \quad \text{non dipende da } x$$

Inoltre abbiamo visto che il prodotto tra numeri floating-point:

$$x \cdot y : \quad \left| \frac{xy - \tilde{x}\tilde{y}}{xy} \right| \leq \text{ceps}$$

è ben condizionato, mentre la somma tra numeri floating-point:

$$x + y : \quad \left| \frac{x + y - \tilde{x} - \tilde{y}}{x + y} \right| \leq \frac{|x|}{|x + y|} |\delta_x| + \frac{|y|}{|x + y|} |\delta_y|$$

e quindi per $x + y \approx 0$ la somma è mal condizionata. Come possiamo quindi evitare problemi mal condizionati?

Esempio 2.2 (Calcolo delle radici di $ax^2 + bx + c = 0$).

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Se $|4ac|$ è piccolo allora $-b \pm \sqrt{b^2 - 4ac}$ può essere mal condizionato. In particolare:

$$\begin{aligned} \text{se } b > 0 \quad & \begin{cases} -b + \sqrt{b^2 - 4ac} & \text{è mal condizionato} \\ -b - \sqrt{b^2 - 4ac} & \text{è ben condizionato} \end{cases} \\ \text{se } b < 0 \quad & \begin{cases} -b + \sqrt{b^2 - 4ac} & \text{è ben condizionato} \\ -b - \sqrt{b^2 - 4ac} & \text{è mal condizionato} \end{cases} \end{aligned}$$

Allora calcolo:

$$x_1 = \begin{cases} \frac{-b + \sqrt{b^2 - 4ac}}{2a} & \text{se } b < 0 \\ \frac{-b - \sqrt{b^2 - 4ac}}{2a} & \text{se } b > 0 \end{cases}$$

Per l'altra radice uso: $x_1 x_2 = \frac{c}{a}$ e quindi:

$$x_2 = \frac{c}{ax_1}$$

che è ben condizionato. In questo modo elimino la necessità di calcolare la radice mal condizionata. Ricordiamo che se x_1 e x_2 sono le due radici di $ax^2 + bx + c = 0$ allora:

$$a \left(x^2 + \frac{b}{a}x + \frac{c}{a} \right) = a(x - x_1)(x - x_2) \implies x_1 x_2 = \frac{c}{a}$$

3 Sistemi lineari

Abbiamo visto che un sistema lineare si può scrivere come:

$$Ax = b$$

dove A è una matrice $n \times n$, $b \in \mathbb{R}^n$ e si suppone che $\det A \neq 0$. Allora esiste un'unica soluzione $x \in \mathbb{R}^n$ che soddisfa il sistema. Se A è triangolare il sistema si risolve per sostituzioni successive.

Sistemi triangolari inferiori li indichiamo con $A = \begin{bmatrix} a_{11} & 0 & 0 & \cdots & 0 \\ a_{21} & a_{22} & 0 & \cdots & 0 \\ a_{31} & a_{32} & a_{33} & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \cdots & a_{nn} \end{bmatrix}$.

$$\det A = \prod_{i=1}^n a_{ii} \quad \det A \neq 0 \iff a_{ii} \neq 0 \quad \forall i$$

Quindi si avrà un sistema del tipo:

$$\begin{aligned} a_{11}x_1 &= b_1 & \implies x_1 &= \frac{b_1}{a_{11}} \\ a_{21}x_1 + a_{22}x_2 &= b_2 & \implies x_2 &= \frac{b_2 - a_{21}x_1}{a_{22}} \\ &\vdots & \implies x_i &= \frac{\left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j\right)}{a_{ii}} \\ a_{n1}x_1 + a_{n2}x_2 + \dots + a_{nn}x_n &= b_n \end{aligned}$$

Partendo dall'equazione 1 per sostituzioni successive trovo le incognite nell'ordine $x_1, x_2, \dots, x_n$. Il costo soluzione del sistema è $\sim \frac{n \cdot n}{2} \Rightarrow \mathcal{O}\left(\frac{n^2}{2}\right)$.

Per i sistemi triangolari superiori ci si comporta in maniera analoga, ma partendo dall'ultima equazione e risalendo.

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n &= b_1 \\ a_{22}x_2 + \dots + a_{2n}x_n &= b_2 \\ &\vdots \\ a_{nn}x_n &= b_n \end{aligned} \quad \begin{aligned} x_{n-1} &= \frac{b_{n-1} - a_{n-1,n}x_n}{a_{n-1,n-1}} \\ &\implies x_n = \frac{b_n}{a_{nn}} \end{aligned}$$

Parto dal calcolo di x_n e per sostituzioni successive calcolo $x_n, x_{n-1}, \dots, x_1$. L'idea è quella di trasformare $Ax = b$ nella soluzione di sistemi triangolari equivalenti.

Uso la **riduzione a scala di Gauss**.

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{bmatrix}$$

Trovo dunque delle trasformazioni lineari che riempiano di zeri la prima colonna della matrice A . Ottengo una nuova matrice $A^{(1)}$ che sarà così fatta:

$$A^{(1)} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ 0 & a_{22}^{(1)} & \dots & a_{2n}^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & a_{n2}^{(1)} & \dots & a_{nn}^{(1)} \end{bmatrix}$$

Questo lo eseguo chiamando A_i la riga i e calcolando:

$$A_i^1 = A_i^0 - m_{i1}A_1^0$$

dove m_{i1} è tale che $(A_i^1)_1 = 0$. Ripeto dunque lo stesso procedimento per la seconda colonna, per cui ottengo $A^{(2)}$ che sarà così fatta:

$$A^{(2)} = \begin{bmatrix} a_{11} & a_{12} & \dots & \dots & a_{1n} \\ 0 & a_{22}^{(1)} & \dots & \dots & a_{2n}^{(1)} \\ 0 & 0 & a_{33}^{(2)} & \dots & a_{3n}^{(2)} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & a_{n3}^{(2)} & \dots & a_{nn}^{(2)} \end{bmatrix}$$

Per calcolare $A^{(2)}$ necessito dei moltiplicatori m_{i2} che azzerino gli elementi della seconda colonna. Poiché voglio che $a_{i1}^1 = 0$ ho:

$$a_{i1} - m_{21}a_{11} = 0 \implies m_{i1} = \frac{a_{i1}}{a_{11}} \quad a_{11} \neq 0$$

Generalizzando:

$$a_{ij}^{(k+1)} = a_{ij}^{(k)} - m_{ik}a_{kj}^{(k)}$$

e occorre fare la stessa cosa anche su b .

$$b_i^{(k+1)} = b_i^{(k)} - m_{ik}b_k^{(k)}$$

Questa operazione è lineare, per cui può essere rappresentata come prodotto per una matrice opportuna M^1 , cioè

$$A^1 = M^1 A$$

Si vede che

$$M_1 = \begin{bmatrix} 1 & 0 & \dots & 0 \\ -m_{21} & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ -m_{n1} & 0 & \dots & 1 \end{bmatrix}$$

Per la seconda colonna parto dalla riga 2 di $A^{(1)}$, la multiplico per un moltiplicatore tale che

$$A_i^{(2)} = A_i^{(1)} - m_{i2}A_2^{(1)} \quad m_{i2} : a_{i2}^{(2)} = a_{i2}^{(1)} - m_{i2}a_{22}^{(1)} = 0 \implies m_{i2} = \frac{a_{i2}^{(1)}}{a_{22}^{(1)}}$$

Necessito che $a_{22}^{(1)} \neq 0$. Notare che $a_{22}^{(1)} \neq a_{22}^{(0)}$.

$$a_{ij}^{(2)} = a_{ij}^{(1)} - m_{i2}a_{2j}^{(1)} \quad i, j = 3, \dots, n$$

e

$$b_i^{(2)} = b_i^{(1)} - m_{i2}b_2^{(1)} \quad i = 3, \dots, n$$

Ora $A^{(2)}$ ha le colonne 1 e 2 piene di zeri sotto la diagonale. Quella che abbiamo fatto è un'operazione lineare, per cui $\exists M^2$ tale che:

$$A^{(2)} = M^2 A^{(1)} = M^2 M^1 A$$

$$M_2 = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 \\ 0 & 1 & 0 & \dots & 0 \\ 0 & -m_{32} & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & -m_{n2} & 0 & \dots & 1 \end{bmatrix}$$

M_2 è una matrice identità con la colonna 2 riempita coi moltiplicatori. Posso quindi generalizzare per tutte le altre colonne fino alla $n-1$ -esima. Quindi a ogni stadio ho:

$$A^{(i)} = M^i A^{(i-1)}$$

terminata questa operazione ho

$$A^{(n-1)} = U$$

è triangolare superiore, purché gli elementi $a_{ii}^{(i-1)} \neq 0 \quad \forall i = 1, \dots, n-1$. Questi elementi sono detti **pivots**.

$$\begin{aligned} U &= A^{(n-1)} = M^{n-1} A^{(n-2)} = \\ &= M^{n-1} M^{n-2} A^{(n-3)} = \\ &= M^{n-1} M^{n-2} \dots M^1 A \end{aligned}$$

Ognuna delle matrici M^i sono invertibili perché hanno determinante 1. Posso riformularlo come:

$$A = (M^{n-1} \dots M^1)^{-1} U = (M^1)^{-1} (M^2)^{-1} \dots (M^{n-1})^{-1} U$$

Si verifica inoltre che:

$$(M^i)^{-1} = \begin{bmatrix} 1 & 0 & \dots & \dots & 0 \\ 0 & 1 & \dots & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & & 1 & 0 \\ 0 & 0 & & 0 & 1 \end{bmatrix}$$

Per cui ha nella colonna i gli elementi m_{ij} . Si ottiene che:

$$L = (M^1)^{-1} (M^2)^{-1} \dots (M^{n-1})^{-1} = \begin{bmatrix} 1 & 0 & \dots & 0 \\ m_{21} & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ m_{n1} & m_{n2} & \dots & 1 \end{bmatrix}$$

ovvero:

$$L_{ij} = \begin{cases} 0 & \text{se } j > i \\ 1 & \text{se } i = j \\ m_{ij} & \text{se } j < i \end{cases}$$

cioè con l'algoritmo di Gauss posso riscrivere:

$$A = LU$$

Esempio 3.1 (Fattorizzazione LU). Calcolare la fattorizzazione LU della matrice:

$$A = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 2 & -1 & 2 \end{bmatrix}$$

e risolvere il sistema:

$$Ax = b \quad b^T = [1 \quad 0 \quad 0]$$

1. Riduzione del sistema con la riduzione a scala

$$\begin{aligned} a_{11} = 2 \neq 0 & \implies \text{calcolo i moltiplicatori della prima colonna} \\ m_{21} &= \frac{a_{21}}{a_{11}} = \frac{1}{2} \\ m_{31} &= \frac{a_{31}}{a_{11}} = 1 \end{aligned}$$

Troviamo II riga di $A^{(1)}$

$$A_2^1 = A_2^0 - m_{21}A_1^0 = [1 \quad 2 \quad 1] - \frac{1}{2}[2 \quad 1 \quad 1] = [0 \quad \frac{3}{2} \quad \frac{1}{2}]$$

Troviamo III riga di $A^{(1)}$

$$A_3^1 = A_3^0 - m_{31}A_1^0 = [2 \quad -1 \quad 2] - 1[2 \quad 1 \quad 1] = [0 \quad -2 \quad 1]$$

Quindi la matrice A^1 e b^1 sono:

$$\begin{aligned} A^1 &= \begin{bmatrix} 2 & 1 & 1 \\ 0 & \frac{3}{2} & \frac{1}{2} \\ 0 & -2 & 1 \end{bmatrix} \\ b^1 &= \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 0 \\ m_{21}b_1^0 \\ m_{31}b_1^0 \end{bmatrix} = \begin{bmatrix} 1 \\ -\frac{1}{2} \\ -1 \end{bmatrix} \end{aligned}$$

Notiamo che $a_{22}^{(1)} = \frac{3}{2} \neq 0$, quindi possiamo proseguire. Calcoliamo i moltiplicatori della seconda colonna:

$$m_{32} = \frac{a_{32}^{(1)}}{a_{22}^{(1)}} = \frac{-2}{\frac{3}{2}} = -\frac{4}{3}$$

Troviamo la III riga di $A^{(2)}$

$$A_3^2 = A_3^1 - m_{32}A_2^1 = [0 \quad -2 \quad 1] - \left(-\frac{4}{3}\right)[0 \quad \frac{3}{2} \quad \frac{1}{2}] = [0 \quad 0 \quad \frac{5}{3}]$$

E troviamo $b_3^{(2)} = b_3^1 - m_{32}b_2^1 = -1 - \left(-\frac{4}{3}\right)\left(-\frac{1}{2}\right) = -1 - \frac{2}{3} = -\frac{5}{3}$.
Quindi il sistema diventa:

$$A^{(2)}x = b^{(2)} \implies \begin{bmatrix} 2 & 1 & 1 \\ 0 & \frac{3}{2} & \frac{1}{2} \\ 0 & 0 & \frac{5}{3} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ -\frac{1}{2} \\ -\frac{5}{3} \end{bmatrix}$$

Abbiamo quindi ridotto a scala e trasformato b . A questo punto possiamo ricorrere alla risoluzione del sistema triangolare superiore (partendo dall'ultima equazione) oppure possiamo usare la fattorizzazione LU . Vediamo la risoluzione con la risoluzione del sistema triangolare superiore:

$$\begin{cases} x_3 = -1 \\ \frac{3}{2}x_2 + \frac{1}{2}x_3 = -\frac{1}{2} \implies x_2 = 0 \\ 2x_1 + x_2 + x_3 = 2x_1 - 1 \implies x_1 = 1 \end{cases}$$

Quindi la soluzione è $x = [1 \ 0 \ -1]$.

Osservazione 3.1. $a_{11} \neq 0$ lo vediamo dalla matrice direttamente, mentre $a_{22}^{(1)} \neq 0$ lo vediamo dai calcoli.

2. Fattorizzazione LU

$$A = LU = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 2 & -1 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ m_{21} & 1 & 0 \\ m_{31} & m_{32} & 1 \end{bmatrix} \begin{bmatrix} 2 & 1 & 1 \\ 0 & \frac{3}{2} & \frac{1}{2} \\ 0 & 0 & \frac{5}{3} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & 1 & 0 \\ 1 & -\frac{4}{3} & 1 \end{bmatrix} \begin{bmatrix} 2 & 1 & 1 \\ 0 & \frac{3}{2} & \frac{1}{2} \\ 0 & 0 & \frac{5}{3} \end{bmatrix}$$

Posso dunque riscrivere il sistema originale

$$Ax = b \implies LUx = b \implies Ly = b \implies Ux = y$$

Quindi risolviamo il sistema in due passi:

(a) Risoluzione di $Ly = b$

$$\begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & 1 & 0 \\ 1 & -\frac{4}{3} & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \implies y = \begin{bmatrix} 1 \\ -\frac{1}{2} \\ -\frac{1}{3} \end{bmatrix}$$

(b) Risoluzione di $Ux = y$

$$\begin{bmatrix} 2 & 1 & 1 \\ 0 & \frac{3}{2} & \frac{1}{2} \\ 0 & 0 & \frac{5}{3} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ -\frac{1}{2} \\ -\frac{1}{3} \end{bmatrix} \implies x = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}$$

Osservazione 3.2. Nelle librerie di calcolo (linpack) la fattorizzazione LU viene memorizzata nella matrice A stessa via via che viene calcolata.

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ m_{21} & [A^{(1)}] & \end{bmatrix}$$

Osservazione 3.3 (In Matlab). Dato un sistema $Ax = b$ si può risolvere in matlab con $x = A \backslash b$ (che calcola riduzione a scala di A e poi risolve il sistema), o ricorrendo alla fattorizzazione $A = LU$ con $[L, U] = \text{lu}(A)$ che calcola la fattorizzazione e memorizza i risultati in L e U , quindi si risolve il sistema con $y = L \backslash b$ (che risolve $Ly = b$) e $x = U \backslash y$ (che risolve $Ux = y$).

Algorithm 3 Fattorizzazione LU

```

1: for  $k = 1 : n - 1$  do ▷ Calcola matrici  $A^k$ 
2:   if  $a_{kk}^{(k-1)} \leq \text{TEST}$  then
3:     exit
4:   end if
5:   for  $i = k + 1 : n$  do
6:      $m_{ik} = \frac{a_{ik}^{(k-1)}}{a_{kk}^{(k-1)}}$ 
7:     for  $j = k + 1 : n$  do
8:        $a_{ij}^{(k)} = a_{ij}^{(k-1)} - m_{ik} a_{kj}^{(k-1)}$ 
9:     end for
10:     $\star$ 
11:   end for
12: end for
13:  $U = A^{(n-1)}$ 

```

Se si vuole risolvere il sistema $Ax = b$ possiamo anche aggiornare b in $\star$ con:

$$b_i^{(k)} = b_i^{(k-1)} - m_{ik} b_k^{(k-1)} \quad i = k + 1, \dots, n$$

Osservazione 3.4. Osserviamo che abbiamo usato la condizione $a_{kk}^{(k-1)} \leq \text{TEST}$ per evitare i difetti dell'aritmetica floating point. Vogliamo che:

$$|a_{kk}^{(k-1)}| > \text{tol} \cdot C$$

dove la costante C serve a rendere il test relativo. Possiamo prendere $C = \max_{i,j} |a_{ij}|$ o $C = \|A\|_1$ o $C = \|A\|_\infty$ o $C = \|A\|_F$.

Vediamo come ci comportiamo per $a_{kk}^{(k-1)} = 0$, dato che questa condizione non implica la non risolubilità del sistema.

Esempio 3.2.

$$A = \begin{bmatrix} 0 & 2 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$$

In questo caso $a_{11} = 0$, ma $\det(A) \neq 0$, per cui il sistema è risolubile. Infatti notiamo che invertendo la prima riga con la seconda otteniamo:

$$A = \begin{bmatrix} 1 & 3 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} b_2 \\ b_1 \end{bmatrix}$$

e quindi il sistema è risolubile.

Quindi se $a_{kk}^{(k-1)} = 0$ cerchiamo i tale che $|a_{ik}^{(k-1)}| > 0$ e scambiamo la riga k -esima con la i -esima sia in A che in b e posso continuare la riduzione a scala. L'esistenza di i è garantita dal fatto che se non esistesse allora la colonna k -esima sarebbe combinazione lineare delle colonne precedenti e quindi A non sarebbe invertibile. Se A è invertibile allora possiamo sempre trovare i tale che $|a_{ik}^{(k-1)}| > 0$.

Valutiamo il **costo computazionale** per la fattorizzazione LU:

- Per il calcolo di A^1 occorre moltiplicare ogni moltiplicatore per $n - 1$ elementi di A e poi per ciascun $a_{ij}^{(1)}$ occorre fare $n - 1$ moltiplicazioni. Il costo di questa operazione è $(n - 1) \times (n - 1)$.
- Per il calcolo di A^2 occorre moltiplicare ogni moltiplicatore per $n - 2$ elementi di A e poi per ciascun $a_{ij}^{(2)}$ occorre fare $n - 2$ moltiplicazioni. Il costo di questa operazione è $(n - 2) \times (n - 2)$.
- In generale per il calcolo di A^k occorre moltiplicare ogni moltiplicatore per $n - k$ elementi di A e poi per ciascun $a_{ij}^{(k)}$ occorre fare $n - k$ moltiplicazioni.

Possiamo porre questa costruzione come una piramide in cui ogni cubetto è un'operazione di moltiplicazione. Quindi il costo computazionale è dato dal volume di questa piramide.

$$\frac{1}{3}(n-1)(n-1)^2 = O\left(\frac{1}{3}n^3\right)$$

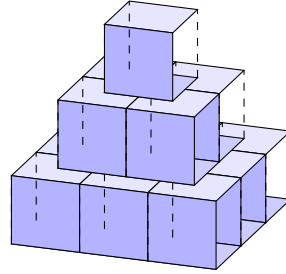


Figura 3: Ogni cubetto rappresenta un'operazione

Esempio 3.3. Vediamo dunque perché lo scambio di righe può essere utile anche se $a_{kk}^{(k-1)} \neq 0$.

$$A = \begin{bmatrix} \varepsilon & 1 \\ 1 & 1 \end{bmatrix} \quad \det A \neq 0 \text{ se } \varepsilon \neq 1 \text{ e } \varepsilon > 0$$

Consideriamo il sistema:

$$\begin{bmatrix} \varepsilon & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Risolvendo il sistema “a mano” trovo:

$$x = \frac{1}{\varepsilon - 1} \quad y = \frac{\varepsilon}{1 - \varepsilon} \quad \text{per } \varepsilon \rightarrow 1 \implies \begin{bmatrix} x \\ y \end{bmatrix} \rightarrow \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

Risolvero con la fattorizzazione LU: $m_{21} = \frac{a_{21}}{a_{11}} = \frac{1}{\varepsilon}$ Per la riga 2 dunque ci troviamo:

$$\begin{bmatrix} 1 & 1 \end{bmatrix} - \frac{1}{\varepsilon} \begin{bmatrix} \varepsilon & 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 - \frac{1}{\varepsilon} \end{bmatrix}$$

quindi la fattorizzazione diventa:

$$A = LU = \begin{bmatrix} 1 & 0 \\ \frac{1}{\varepsilon} & 1 \end{bmatrix} \begin{bmatrix} \varepsilon & 1 \\ 0 & 1 - \frac{1}{\varepsilon} \end{bmatrix}$$

Se considero l'errore di macchina e suppongo ε piccolo trovo:

$$\tilde{L}\tilde{U} = \begin{bmatrix} 1 & 0 \\ \frac{1}{\varepsilon} & 1 \end{bmatrix} \begin{bmatrix} \varepsilon & 1 \\ 0 & -\frac{1}{\varepsilon} \end{bmatrix}$$

perché $(1 - \frac{1}{\varepsilon}) = -\frac{1}{\varepsilon}$ Quindi risolvo $\tilde{L}\tilde{U}x = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$

$$\tilde{L}z = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \implies \begin{bmatrix} 1 & 0 \\ \frac{1}{\varepsilon} & 1 \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Per cui trovo che $z_1 = 1$ e $\frac{1}{\varepsilon}z_1 + z_2 = 0 \implies z_2 = -\frac{1}{\varepsilon}$. Risolvo dunque:

$$\tilde{U} \begin{bmatrix} x \\ y \end{bmatrix} = z \implies \begin{bmatrix} \varepsilon & 1 \\ 0 & -\frac{1}{\varepsilon} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 1 \\ -\frac{1}{\varepsilon} \end{bmatrix}$$

Per cui trovo che $\varepsilon x + 1 = 1 \implies x = 0$ e $y = 1$. Quindi $\begin{bmatrix} x \\ y \end{bmatrix}$ trovato con la fattorizzazione $\tilde{L}\tilde{U}$ è molto diverso dalla soluzione esatta. Se invece scambiamo la prima riga con la seconda otteniamo:

$$\begin{bmatrix} 1 & 1 \\ \varepsilon & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Applicando Gauss $m_{21} = \frac{a_{21}}{a_{11}} = \frac{\varepsilon}{1} = \varepsilon$ e dunque:

$$LU = \begin{bmatrix} 1 & 0 \\ \varepsilon & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 1 - \varepsilon \end{bmatrix}$$

L'approssimazione floating point diventa dunque:

$$\tilde{L}\tilde{U} = \begin{bmatrix} 1 & 0 \\ \varepsilon & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$$

e il sistema che risolvo diviene:

$$\begin{bmatrix} 1 & 0 \\ \varepsilon & 1 \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Per cui trovo che $z_2 = 1$ e $z_1 = 0$, dunque da

$$U \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix}$$

si ottiene $x = -1$ e $y = 1$, che è molto vicino alla soluzione esatta.

Osservazione 3.5. *Dall'esempio precedente osserviamo che per avere una fattorizzazione accurata è una buona pratica evitare moltiplicatori grandi, ovvero evitare elementi pivot $a_{kk}^{(k-1)}$ piccoli.*

Gauss con pivoting: applico l'algoritmo cercando elementi pivot più grandi possibili. L'algoritmo dunque diviene:

Algorithm 4 Gauss con pivoting

```

1: for  $k = 1 : n - 1$  do
2:   scelgo l'elemento pivot più grande possibile  $l = \arg \max_{i=k:n} |a_{ik}^{(k-1)}|$  ovvero  $|a_{lk}^{(k-1)}| = \max_{k \leq i \leq n} |a_{ik}^{(k-1)}|$ 
3:   if  $|a_{lk}^{(k-1)}| \leq \text{To1} \|A\|_\infty$  then
4:     exit (matrice quasi singolare)
5:   else
6:     scambio la riga  $k$ -esima con la riga  $l$ -esima sia in  $A$  che in  $b$ 
7:   end if
8:   l'algoritmo prosegue dunque come abbiamo già definito
9: end for

```

Per evitare di copiare davvero una riga sull'altra posso ricordare lo scambio da effettuare con un vettore:

$$p(k) = l$$

ovvero allo stadio k scambia la riga l con la riga k .

Gauss con pivoting sulle colonne ottengo un algoritmo più stabile. Ricordiamo che per risolvere sistemi $Ax = b$ in realtà, per gli errori di arrotondamento, risolviamo qualcosa del tipo $A\tilde{x} = \tilde{b}$ dove $\tilde{b}$ è ottenuto da b con un errore di arrotondamento e

$$\frac{\|x - \tilde{x}\|}{\|x\|} \leq K(A) \frac{\|b - \tilde{b}\|}{\|b\|} \quad \star$$

cioè risolvo esattamente un problema vicino a quello dato, ottenendo una soluzione approssimata $\tilde{x}$ che ha un'accuratezza data da $\star$. Se risolvo il sistema applicando l'algoritmo di Gauss, introduco ulteriori errori e ottengo una soluzione approssimata y . Quindi l'errore è

$$x - y = x - \tilde{x} + \tilde{x} - y$$

per cui $x - \tilde{x}$ è legato al condizionamento del problema, mentre $\tilde{x} - y$ è legato all'errore introdotto dall'algoritmo:

$$\|x - y\| \leq \|x - \tilde{x}\| + \|\tilde{x} - y\|$$

quindi implementare Gauss con pivoting risulta in un algoritmo più stabile di Gauss senza pivoting. Se A è simmetrica e definita positiva (SPD) allora non è necessario fare il pivoting. Si può dimostrare che (Golub - Matrix Computations) che se $\tilde{L}\tilde{U}$ è la fattorizzazione ottenuta con Gauss allora

$$\tilde{L}\tilde{U} = A + \varepsilon$$

cioè $\tilde{L}\tilde{U}$ è la fattorizzazione esatta di un problema diverso con

$$\|E\| \leq \mathbf{eps} \cdot C \quad (\|A\| + \|\tilde{L}\| \cdot \|\tilde{U}\|)$$

Ricordiamo il problema del filo elastico:

$$\begin{cases} -u''(x) = f(x) \\ u(0) = u(1) = 0 \end{cases} \quad x \in (0, 1)$$

Abbiamo dunque scelto un intero n , definito una griglia:

$$x_0 = 0, x_1, \dots, x_n = b$$

su $[0, 1]$ con $x_i = ib$ e $b = \frac{1}{n}$. Chiamo $u(x)$ la soluzione esatta e cerco $u(x_1) \dots u(x_{n-1})$ nei nodi interni. Devo dunque approssimare la derivata seconda.

$$\frac{du}{dx}|_y = \lim_{\varepsilon \rightarrow 0} \frac{u(y + \varepsilon) - u(y)}{\varepsilon} \approx \frac{u(y + \varepsilon) - u(y)}{\varepsilon} \quad (\text{purché } \varepsilon \text{ sia piccolo})$$

Dato un punto x_j troviamo x_{j-1} e x_{j+1} e possiamo approssimare la derivata seconda

$$\frac{d^2u}{dx^2} \approx \left[\frac{du}{dx}|_{x_{j+\frac{1}{2}}} - \frac{du}{dx}|_{x_{j-\frac{1}{2}}} \right] \frac{1}{h}$$

Per cui:

$$\frac{du}{dx}|_{x_{j+\frac{1}{2}}} \approx \frac{u(x_{j+1}) - u(x_j)}{h}$$

e

$$\frac{du}{dx}|_{x_{j-\frac{1}{2}}} \approx \frac{u(x_j) - u(x_{j-1})}{h}$$

quindi:

$$\frac{d^2u}{dx^2} \approx \frac{1}{h^2} [u(x_{j+1}) - 2u(x_j) + u(x_{j-1})]$$

calcolo la soluzione solo nei punti di griglia e ottengo un vettore

$$U = [U_1 \quad \dots \quad U_{n-1}]$$

Con $U_0 = U_n = 0$, dunque l'equazione differenziale viene approssimata da:

$$-u''|_j = \frac{1}{h^2} [-U_{j+1} + 2U_j - U_{j-1}] = f(x_j)$$

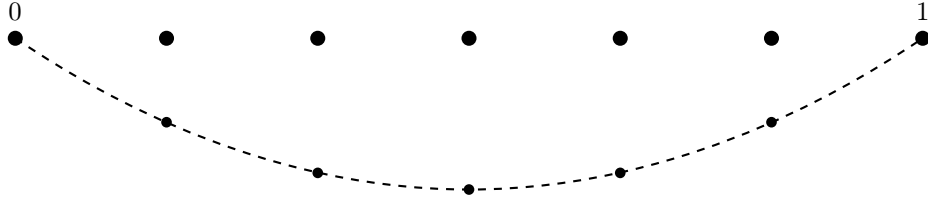
per $j = 1, \dots, n-1$. Ottengo il sistema lineare:

$$A_h U = b$$

con

$$A_h = \frac{1}{h^2} \begin{bmatrix} 2 & -1 & 0 & \dots & 0 \\ -1 & 2 & -1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & -1 & 2 & -1 \\ 0 & \dots & 0 & -1 & 2 \end{bmatrix} \quad b = \begin{bmatrix} f(x_1) \\ \vdots \\ f(x_{n-1}) \end{bmatrix}$$

per cui $A_h \in \mathbb{R}^{n-1 \times n-1}$. Risolvendo il sistema troviamo la deformazione in ogni punto della griglia.



Definisco $U(x)$ unendo i punti (U_j, x_j) , l'errore è dato da

$$u(x) - \tilde{U}(x) = \underbrace{u(x) - U(x)}_{(1)} + \underbrace{U(x) - \tilde{U}(x)}_{(2)}$$

(1) si riferisce all'errore di discretizzazione del modello, cioè l'errore dovuto alla sostituzione delle derivate con i rapporti incrementali. ($\rightarrow 0$ per $h \rightarrow 0$)

(2) è l'errore nella risoluzione del sistema. È l'errore floating-point accumulato nell'algoritmo di Gauss.

Se diminuisco h per avere una soluzione più precisa allora A_h diviene più grande (dato che $n \sim \frac{1}{h}$).

Vediamo come risponde la fattorizzazione LU all'utilizzo del pivoting. Innanzitutto occorre capire quale è la matrice che permette di scambiare la riga l con la riga k .

$$\begin{bmatrix} 1 & 0 & 0 & 0 & \cdots & \cdots & 0 \\ 0 & 1 & 0 & 0 & \cdots & \cdots & 0 \\ \vdots & \ddots & \ddots & \vdots & \vdots & \vdots & 0 \\ 0 & \ddots & 0 & 0 & 1 & \cdots & 0 \\ 0 & \ddots & 0 & 1 & 0 & \ddots & 0 \\ 0 & \ddots & 0 & 0 & 0 & 1 & 0 \\ 0 & \cdots & 0 & 0 & \cdots & 0 & 1 \end{bmatrix}$$

$$(P_{lk})_{ll} = 0 \quad (P_{lk})_{kk} = 0 \quad (P_{lk})_{lk} = 1 \quad (P_{lk})_{kj} = 1$$

Per A ottengo A con la riga l e la riga k scambiate.

$$P_{lk} P_{lk} = \mathbb{I} \quad P_{lk}^{-1} = P_{lk}$$

queste matrici sono dette di **permutazione**. Posso dunque riscrivere l'algoritmo con pivoting:

$$M^1 P^1 A$$

con P^1 il pivoting sulla prima colonna e M^1 i moltiplicatori della prima u . Continuando:

$$M^{n-1} P^{n-1} M^{n-2} P^{n-2} \dots M^2 P^2 M^1 P^1 A = U$$

Riduzione a scala in forma matriciale:

$$M^{n-1} P^{n-1} M^{n-2} P^{n-2} \dots M^1 P^1 A x = M^{n-1} P^{n-1} M^{n-2} P^{n-2} \dots M^1 P^1 b$$

dove

$$P^i = \begin{cases} \mathbb{I} & \text{se non faccio scambi di righe} \\ P & \text{definita dal vettore } p(i) = l \text{ che memorizza gli scambi altrimenti} \end{cases}$$

per cui:

$$A = \underbrace{(P^1)^{-1}(M^1)^{-1} \dots (P^{n-1})^{-1}(M^{n-1})^{-1}}_{L_p} U$$

L_p è una matrice triangolare inferiore con le righe scambiate. È quello che ottengo in Matlab con $[l, u] = \text{lu}(A)$.

Osservazione 3.6. È possibile dimostrare che questo risultato può essere scritto

$$PA = LU$$

dove $P = \Pi P^{lk}$. L è una matrice triangolare inferiore ma con i moltiplicatori permutati. La ottengo con $[l, u, p] = \text{lu}(A)$.

Filo elastico

$$\begin{cases} -u''(x) = f(x) \\ u(0) = u(1) = 0 \end{cases} \quad x \in (0, 1)$$

$u(x)$ rappresenta la deviazione del filo dovuta alla forza $f(x)$, rispetto alla posizione di equilibrio che si ha per $f(x) = 0 \Rightarrow u(x) = 0$.

Membrana elastica (immagina una bolla di sapone che viene soffiata dal bastone per le bolle). Dominio $\Omega \subset \mathbb{R}^2$. La membrana è vincolata sul bordo $\partial\Omega$ e

$$u(x, y) = 0 \quad \forall (x, y) \in \partial\Omega$$

L'equazione diventa:

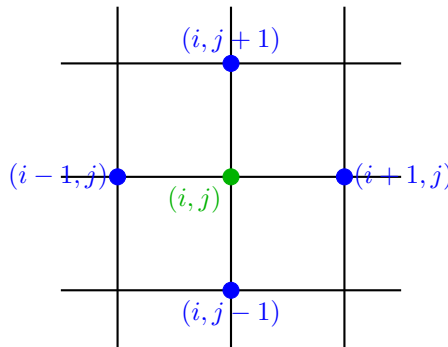
$$-\Delta u = f(x, y) \quad \text{in } \Omega$$

dove Δ è l'operatore Laplaciano di $u = \partial_{xx}^2 u + \partial_{yy}^2 u$. Se $f(x, y) = 0$ all'equazione ho una posizione di equilibrio che è $u(x, y) = 0$. Considero $\Omega = (0, 1) \times (0, 1)$:

Chiamo $n = \#$ intervalli per lato. $h = \frac{1}{n}$ la spaziatura tra i nodi. Abbiamo dunque $(n-1)^2$ nodi interni:

$$x_{i,j} = (ih, jh) \quad i, j = 1, \dots, n-1 \quad \bar{\Omega} = [0, 1]^2 = \Omega \cup \partial\Omega$$

Con lo schema numerico voglio calcolare $u_{i,j} = u(ih, jh)$ e $-\Delta u|_{i,j} = f(ih, jh) \quad \forall i, j = 1, \dots, n-1$



Quindi il laplaciano diventa:

$$\begin{aligned} -\Delta u|_{i,j} &= -\partial_x^2 u|_{i,j} - \partial_y^2 u|_{i,j} \\ &\approx -\frac{1}{h^2} (u_{i+1,j} - 2u_{i,j} + u_{i-1,j}) - \frac{1}{h^2} (u_{i,j+1} - 2u_{i,j} + u_{i,j-1}) \end{aligned}$$

Dunque:

$$-\Delta u|_{i,j} \approx \frac{1}{h^2} (-u_{i+1,j} - u_{i-1,j} - u_{i,j+1} - u_{i,j-1} + 4u_{i,j}) = f(ih, jh)$$

Definisco un sistema lineare nelle incognite $u_{i,j}$ per ogni $i, j = 1, \dots, n-1$. Le incognite possono essere allineate in un unico vettore che avrà $(n-1)^2$ elementi “impilando” un $u_{i,j}$ su un’unica colonna:

$$U = [u_{1,1} \quad u_{1,2} \quad \dots \quad u_{1,n-1} \quad u_{2,1} \quad \dots \quad u_{n-1,1} \quad \dots \quad u_{n-1,n-1}]$$

Otengo il sistema lineare

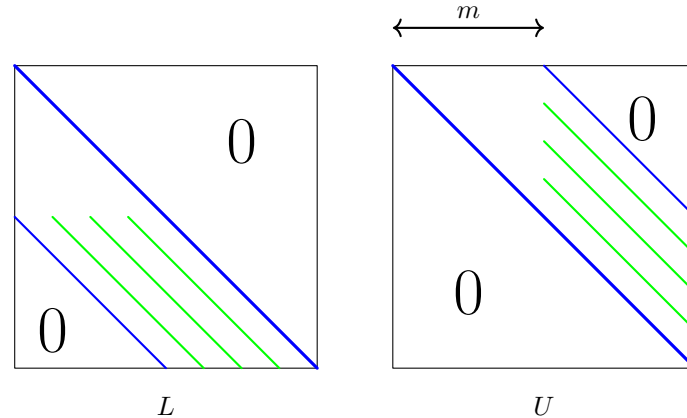
$$A_h U = f$$

dove A_h si chiama matrice di Poisson.

$$(A_h)_{l,k} = \begin{cases} 4 & \text{se } l = k \\ -1 & \text{se } l = k \pm 1, \text{ oppure } l = k \pm (n-1) \\ 0 & \text{altrimenti} \end{cases}$$

Si osservano 5 elementi non nulli per riga. Chiamo $m = n-1$ per cui $A_h \in \mathbb{R}^{m^2 \times m^2}$. A_h ha m^4 elementi di cui solo $5m^2$ non nulli e riesco a trovarli: A_h è una matrice sparsa, simmetrica (tutti gli autovalori sono reali) e positiva definita (autovalori positivi). Questa matrice viene anche detta a banda (con larghezza di banda m).

Per risolvere il sistema $A_h U = f$ possiamo costruire la fattorizzazione LU, per cui trovo:



La fattorizzazione LU rispetta la struttura a banda, ma riempie (fill-in) di elementi non nulli la banda. In matlab con $A_h = \text{gallery('poisson',m)}$ ottengo la matrice di Poisson in forma sparsa. Se poi faccio $A_h = \text{full}(A_h)$ ottengo la matrice piena. $[l,u] = \text{lu}(A_h)$ mi dà la fattorizzazione LU di A_h , mentre $\text{spy}(A_h)$ mi dà un grafico con un $\cdot$ dove c'è un elemento non nullo. Devo calcolare tutti gli elementi $\neq 0$ di L e U e memorizzarli.

Matrici sparse sono matrici A con pochi elementi $\neq 0$ di cui conosco l'indirizzo. Pertanto posso memorizzare solo gli $a_{i,j} \neq 0$ con i relativi indici i e j colonna per colonna.

Esempio 3.4.

$$A = \begin{bmatrix} 3 & 0 & -5 \\ 7 & 1 & 0 \\ 0 & 0 & 2 \end{bmatrix}$$

- 3: (1,1)
- 7: (2,1)
- -5: (1,3)
- 1: (2,2)

- 2: (3,3)

Applicazioni di LU:

1. Calcolo del determinante: si è calcolato $PA = LU$ dove LU calcolato con pivoting. Costruisco dunque P con il vettore degli scambi $p(k) = l$. Ho scambiato la riga l con la riga k se $p(k) = l$.

$$\det(PA) = \det(LU) \implies \det(P) \det(A) = \det(L) \det(U) = 1 \cdot \prod_{i=1}^n U_{i,i}$$

($\det(L) = 1$ dato che L è una matrice triangolare inferiore con elementi 1 sulla diagonale). Il determinante di $P = \prod \det(\text{tutte le matrici di permutazione } P_k \text{ che ho usato})$. Ma $\det(P_{lk}) = -1$ vale per qualunque matrice di permutazione. Dunque:

- Se faccio uno scambio $P_i = P_{lk}$ allora $\det(P_i) = -1$
- Se allo stadio i non faccio scambi $P_i = \mathbb{I}$ e $\det(P_i) = 1$

Dunque:

$$\det(P) = (-1)^r \quad \text{dove } r \text{ è il numero delle volte in cui } P(k) \neq k$$

Quindi $\det(A) = \prod_{i=1}^n U_{i,i} \cdot (-1)^r$. Il costo di questa operazione è $O\left(\frac{n^3}{3}\right)$. Se uso la formula coi cofattori il costo del $\det A$ è $O(n!)$.

2. Calcolo dell'inversa: $AA^{-1} = \mathbb{I}$. Considero colonna per colonna:
Colonna j -esima di A^{-1} , $\underline{A}_j^{-1}$ soddisfa il sistema:

$$A \cdot \underline{A}_j^{-1} = \underline{e}_j \quad \text{dove } \underline{e}_j = \begin{cases} 1 & \text{se } j = i \\ 0 & \text{altrimenti} \end{cases}$$

Risolvero dunque il sistema e trovo la colonna di $\underline{A}_j^{-1}$. Rifaccio per ogni colonna e ottengo la matrice inversa A^{-1} . Di fatto calcolo la fattorizzazione LU e risolvo

$$LU \cdot \underline{A}_j^{-1} = \underline{e}_j \quad \forall j = 1, \dots, n$$

Il costo è dato dal costo di LU + n volte il costo di risoluzione dei due sistemi triangolari. Quindi $\sim O\left(\frac{n^3}{3}\right) + 2nO\left(\frac{n^2}{2}\right) = O\left(\frac{4n^3}{3}\right)$. Col metodo dei cofattori il costo è $O(n!)$.

4 Metodi iterativi

Si vogliono risolvere sistemi lineari $Ax = b$ cercando una successione di vettori $x^{(k)} \in \mathbb{R}^n$ tale che

$$x^{(k)} \rightarrow x \quad \text{per } k \rightarrow \infty$$

4.1 Metodo di Jacobi

Scrivo la matrice A come $A = D + E$ dove $D = \text{diag}(A)$. Il sistema diventa:

$$Dx = b - Ex$$

Definisco la mia successione iterativa dato $x^{(0)}$ (totalmente arbitraria):

$$x^{(k+1)} = \underbrace{b - Ex^{(k)}}_{\text{termine noto}}$$

per trovare $x^{(k+1)}$ devo risolvere un sistema diagonale. Vediamo componente per componente:

$$x_i^{(k+1)} = \left(b_i - \sum_{j=1, j \neq i}^n a_{ij} x_j^{(k)} \right) \frac{1}{a_{ii}} \quad \forall i = 1, \dots, n$$

Dato x^0 (ad esempio $x^0 = b$):

```

1: while TEST do
2:   Risolvi  $Dx^{(k+1)} = b - Ex^{(k)}$ 
3:   TEST
4: end while

```

Il TEST può essere:

1. Test sulle iterazioni successive:

$$\text{TEST} = \|x^{(k+1)} - x^{(k)}\| > c \cdot \text{tol} \quad c = \|x^k\|$$

2. Test sul residuo:

$$\text{TEST} = \|Ax^{(k+1)} - b\| > c \cdot \text{tol} \quad c = \|b\|$$

Se il residuo è ancora grande calcola una nuova iterazione.

Osservazione 4.1. *In generale si completa il test con un contatore sul # di iterazioni fino a un intero itmax e esco se il numero di iterazioni è uguale a itmax . In questo modo prevengo un ciclo infinito.*

quindi

```

1:  $x^{(0)} = b$   $it = 0$ 
2: while TEST do
3:    $x^{(k+1)} = b - Ex^{(k)}$ 
4:    $it = it + 1$ 
5: end while

```

Per cui rimaniamo dentro al ciclo se:

$$\|x^{(k+1)} - x^{(k)}\| \geq c \cdot \text{tol} \quad \wedge \quad it < \text{itmax}$$

Convergenza di Jacobi

$$Dx^{k+1} = b - Ex^k$$

quindi se $x^k \rightarrow y$ allora y è soluzione di $Ax = b$. Per sapere se x^k converge valutiamo l'errore. Sia y la soluzione esatta di $Ax = b$, per cui $Ay = b$. Allora:

$$e_{k+1} = y - x^{k+1} \quad (\text{errore al passo } k+1)$$

Ma $x^{k+1} = D^{-1}(b - Ex^k)$ per cui, sottraendo y :

$$\begin{aligned}
 e_{k+1} &= y - x^{k+1} \\
 &= y - D^{-1}(b - Ex^k) \\
 &= D^{-1}(Dy - Ay + Ex^k) \quad \text{Ma } E = A - D \\
 &= D^{-1}((D - A)(y - x^k)) \\
 &= -D^{-1}(D - A)e_k
 \end{aligned}$$

quindi:

$$e_{k+1} = -D^{-1}Ee_k$$

quindi l'errore $e_{k+1} \sim e_k$. Passando in norma, vale:

$$\|e_{k+1}\| = \|D^{-1}Ee_k\| \leq \underbrace{\|D^{-1}E\|}_\varrho \|e_k\|$$

dato che

$$\|A\| = \sup_{x \neq 0} \frac{\|Ax\|}{\|x\|} \Rightarrow \frac{\|Ax\|}{\|x\|} \leq \|A\| \Rightarrow \|Ax\| \leq \|A\|\|x\|$$

dunque:

$$\|e_{k+1}\| \leq \varrho \|e_k\|$$

e vale

$$\begin{aligned} \|e_k\| &\leq \varrho^2 \|e_{k-1}\| \\ \|e_{k-1}\| &\leq \varrho^2 \|e_{k-2}\| \\ &\vdots \end{aligned}$$

quindi:

$$\|e_k\| \leq \varrho^k \|e_0\|$$

risulta che se $\varrho < 1$ allora Jacobi converge $\forall x^0$, quindi $\|D^{-1}E\|$ deve essere minore di 1 per la condizione di convergenza. Usiamo la norma $\|\cdot\|_\infty$ (sono tutte equivalenti e vale che se $\|x^{k+1} - y\|_\infty \rightarrow 0$ allora va a zero anche nelle altre norme).

$$\|D^{-1}E\|_\infty = \max_i \sum_{j=1}^n |D^{-1}E|_{i,j}$$

E

$$\sum_j \|D^{-1}E\|_j = \sum_j \frac{1}{|d_{ii}|} |e_{ij}| = \frac{1}{|d_{ii}|} \sum_{j \neq i} |e_{ij}|$$

imponendo $\frac{1}{|d_{ii}|} \sum_{j \neq i} |e_{ij}| < 1 \forall i$ ottengo:

$$\sum_{j \neq i} |e_{ij}| < |d_{ii}| \forall i \implies \|D^{-1}E\|_\infty = \varrho < 1$$

questa matrice si dice a **dominanza diagonale** (matrici per cui conviene il metodo di Jacobi).

Definizione 4.1 (Matrice a dominanza diagonale). Una matrice A si dice a dominanza diagonale se:

$$|a_{ii}| \geq \sum_{j \neq i} |a_{ij}| \quad \forall i$$

In altre parole per ogni riga l'elemento sulla diagonale è maggiore o uguale alla somma degli altri elementi della riga. Si dice anche che A è una matrice a dominanza diagonale stretta se vale la disuguaglianza stretta $|a_{ii}| > \sum_{j \neq i} |a_{ij}| \forall i$.

Teorema 4.1. Se A è una matrice a dominanza diagonale stretta, allora Jacobi converge per ogni x^0 .

In realtà per la convergenza di Jacobi è sufficiente che A sia a dominanza diagonale e che abbia dominanza stretta su almeno una riga (e.g. filo elastico).

4.2 Metodi preconditionati

Metodi basati su preconditionatori. Si scrive il residuo $r^k = b - Ax^k$

$$x^{k+1} = x^k + M^{-1}r^k \quad r \text{ matrice invertibile}$$

Si verifica che Jacobi è di questo tipo con $M = D$ (diagonale di A). M si dice preconditionatore. Osservo che se $M = A$ allora:

$$x^{k+1} = x^k (b - Ax^k) = A^{-1}b = x$$

soluzione esatta. Se scelgo $M = 1$, convergo in una iterazione. Ad ogni iterazione, devo risolvere sistemi lineari del tipo:

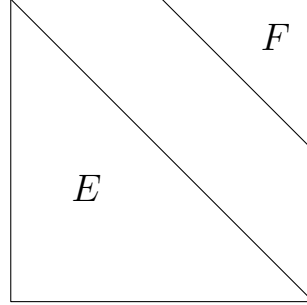
$$Mx^{k+1} = Mx^k + r^k$$

Un buon condizionatore deve soddisfare:

1. $M \approx A$
2. $Mx^{k+1} = Mx^k + r^k$ è facile da risolvere

per il primo punto abbiamo che più $M \approx A$ e meno iterazioni occorre fare. Per il secondo punto più è facile da risolvere più ogni iterazione è veloce.

4.2.1 Metodo di Gauss-Seidel



$$e_{i,j} = \begin{cases} a_{i,j} & \text{se } i \geq j \\ 0 & \text{altrimenti} \end{cases} \quad f_{i,j} = \begin{cases} a_{i,j} & \text{se } i < j \\ 0 & \text{altrimenti} \end{cases}$$

E dunque scrivo $F = A - E$ e ad ogni iterazione risolvo:

$$Ex^{k+1} = Ex^k + r^k \quad (M = E)$$

è veloce perché $\forall k$ devi risolvere un sistema triangolare inferiore.

$$Ex^{k+1} = Ex^k + b - Ax^k \implies Ex^{k+1} = b - Fx^k$$

Componente per componente:

$$a_{ii}x_i^{k+1} = b_i - \sum_{j=i+1}^n a_{ij}x_j^k - \sum_{j=1}^{i-1} a_{ij}x_j^{k+1}$$

Osservazione 4.2. Se Jacobi converge, allora converge anche Gauss-Seidel (di solito più velocemente).

4.2.2 Metodo di Richardson

Matrice del preconditionatore è $M = \frac{1}{\alpha}\mathbb{I}$, per cui l'iterazione che si ottiene partendo da:

$$\frac{1}{\alpha}\mathbb{I}x^{k+1} = \frac{1}{\alpha}\mathbb{I}x^k + r^k \quad Mx^{k+1} = Mx^k + r^k$$

è la seguente:

$$x^{k+1} = x^k + \alpha r^k$$

Il metodo converge a $e^k = x - x^k$ per cui:

$$e^{k+1} = x - x^{k+1} = x - x^k - \alpha r^k = e^k - \alpha(b - Ax^k) = e^k - \alpha Ae^k = (\mathbb{I} - \alpha A)e^k$$

ed essendo $b = Ax$ (ricordando che $r^k = b - Ax^k$) si ottiene:

$$e^{k+1} = e^k - \alpha Ae^k \implies \|e_{k+1}\| \leq \|\mathbb{I} - \alpha A\| \|e^k\|$$

Studio la convergenza per A SPD (simmetrica definita positiva), per cui gli autovalori di A sono reali e positivi. Li ordino in modo che $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n > 0 \implies \max_\lambda = \lambda_1$. Uso la norma $\|\cdot\|_2$ che è basata sugli autovalori e che per A SPD è

$$\|A\|_2 = \max_i |\lambda_i| = \lambda_1$$

Dunque:

$$\|\mathbb{I} - \alpha A\|_2 = \max_i |1 - \alpha \lambda_i|$$

per convergere devo avere:

$$\begin{aligned} -1 &< 1 - \alpha \lambda_i < 1 \quad \forall i \\ -2 &< -\alpha \lambda_i < 0 \\ \implies \alpha \lambda_i &< 2 \end{aligned}$$

cioè $\alpha < \frac{2}{\lambda_i} \quad \forall i$ per cui $\alpha < \frac{2}{\lambda_1}$. Vediamo come ottenere un valore ottimale per α . Consideriamo la funzione

$$p(\alpha) = \|\mathbb{I} - \alpha A\|,$$

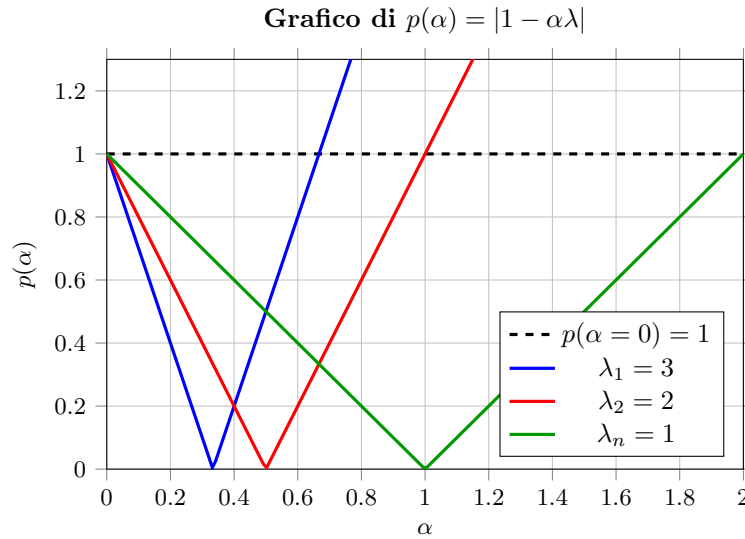
che rappresenta il fattore di contrazione dell'errore per un metodo iterativo applicato al sistema $Ax = b$.

Poiché A è simmetrica definita positiva, possiamo diagonalizzarla e scrivere

$$\|\mathbb{I} - \alpha A\| = \max_i |1 - \alpha \lambda_i|,$$

dove λ_1 e λ_n sono rispettivamente il massimo e il minimo autovalore di A .

Possiamo fare un'analisi grafica di questa funzione. Se disegniamo $p(\alpha) = |1 - \alpha \lambda_i|$, otteniamo delle rette con pendenza λ_i .



Il valore ottimale di α si ottiene per

$$|1 - \alpha \lambda_1| = |1 - \alpha \lambda_n|,$$

da cui si ricava

$$\alpha = \left(\frac{\lambda_1 + \lambda_n}{2} \right)^{-1}.$$

Con questa scelta, l'errore relativo in direzione di ciascun autovalore diventa

$$P_{\alpha_{\text{ott}}}(\lambda_i) = \left| 1 - \frac{2}{\lambda_1 + \lambda_n} \lambda_i \right|,$$

e, in particolare, l'errore massimo è dato da

$$P_{\alpha_{\text{ott}}} = \max_i P_{\alpha_{\text{ott}}}(\lambda_i) = \left| 1 - \frac{2}{\lambda_1 + \lambda_n} \lambda_n \right| = \left| \frac{\lambda_1 - \lambda_n}{\lambda_1 + \lambda_n} \right|.$$

Raccogliendo λ_n a numeratore e denominatore, si ottiene:

$$P_{\alpha_{\text{ott}}} = \frac{\frac{\lambda_1}{\lambda_n} - 1}{\frac{\lambda_1}{\lambda_n} + 1} = \frac{K(A) - 1}{K(A) + 1},$$

dato che $K(A) = \frac{\lambda_1}{\lambda_n}$ rappresenta il numero di condizionamento per matrici SPD.

Questo risultato evidenzia come la velocità di convergenza dei metodi iterativi dipenda fortemente dal numero di condizionamento della matrice. In particolare, se $K(A)$ è vicino a 1 (matrice ben condizionata), allora $P_{\alpha_{\text{ott}}}$ è basso e la convergenza sarà rapida. Al contrario, per matrici mal condizionate, con $K(A)$ elevato, il fattore di contrazione $P_{\alpha_{\text{ott}}}$ si avvicina a 1, determinando una convergenza lenta del metodo.

4.3 Velocità di convergenza

Supponiamo di aver dimostrato che, per un certo metodo iterativo, l'errore al passo k sia stimato da:

$$\|e^k\| \leq \varrho^k \|e^0\|$$

e converge per $0 \leq \varrho < 1$. Vediamo dunque quanto effettivamente converge rapidamente. Ossia per quale k l'errore al passo k è sufficientemente piccolo

$$\|e^k\| \leq \varepsilon \|e^0\| \quad \left(\text{tipicamente } \varepsilon = \frac{1}{10} \right)$$

Vogliamo k tale che $\|e^k\| \leq \varepsilon \|e^0\|$.

$$\begin{aligned} \implies \varrho^k &\leq \varepsilon \\ k \ln \varrho &\leq \ln \varepsilon \\ k &\geq \frac{\ln \varepsilon}{\ln \varrho} \quad \left(\frac{\ln \varepsilon}{\ln \varrho} > 0 \text{ se } \varepsilon < 1 \right) \end{aligned}$$

Esempio 4.1 (Metodo di Richardson).

$$\varrho = \frac{K(A) - 1}{K(A) + 1}$$

dove $K(A) = \frac{\lambda_{\max}}{\lambda_{\min}}$ (condizionamento di A SPD). Per cui:

$$k \geq \frac{\ln \varepsilon}{\ln \left(\frac{K(A) - 1}{K(A) + 1} \right)}$$

e ricordando che $\ln(1+x) \approx x$ per $x \ll 1$:

$$\ln \left(\frac{K(A) - 1}{K(A) + 1} \right) = \ln \left(\underbrace{\frac{K(A) + 1}{K(A) + 1}}_{=1} - \frac{2}{K(A) + 1} \right) \approx -\frac{2}{K(A) + 1} \quad \text{per } K(A) \gg 1$$

Quindi:

$$\implies k \geq \frac{\ln \varepsilon}{-\frac{2}{K(A)+1}} \geq \underbrace{K(A) \left(\frac{\ln \varepsilon}{2} \right)}_{\# \text{ di iterazioni necessarie per ottenere una precisione } \varepsilon}$$

per $\varepsilon = \frac{1}{10}$ si ha:

$$k \geq K(A) \left(\frac{\ln 10}{2} \right)$$

Numero di iterazioni è quindi proporzionale al numero di condizionamento.

4.3.1 Approfondimento Richardson

$$\begin{aligned} x^{k+1} &= x^k + \alpha(b - Ax^k) \\ &= x^k + \alpha r^k \quad r \text{ residuo} \end{aligned}$$

Voglio risolvere $b - Ax = 0$. Un modo per risolvere questo problema è riscriverlo come:

$$\partial_t x = b - Ax$$

con $x \in \mathbb{R}^n$ e $x(t) : \mathbb{R} \rightarrow \mathbb{R}^n$. Se per esempio A è una matrice di poisson per $b - Ax = 0$, x contiene la deformazione della membrana sotto il carico memorizzato in b . Il problema $\partial_t x = b - Ax$ con dato iniziale $x(0) = x^0$ descrive l'evoluzione della membrana dal dato iniziale x^0 fino a raggiungere l'equilibrio (quando $b - Ax = 0 \Rightarrow \partial_t x = 0$). Intuitivamente diciamo che quando raggiunge la posizione di equilibrio, la membrana non si muove più.

Il metodo di Richardson si può interpretare come una **discretizzazione** del problema:

$$\begin{cases} \partial_t x = b - Ax \\ x(0) = x^0 \end{cases}$$

con

$$\partial_t x \simeq \frac{x(t_{k+1}) - x(t_k)}{\underbrace{t_{k+1} - t_k}_{\Delta t}}$$

Ponendo $x^{k+1} = x(t_{k+1}) \rightarrow \partial_t x \simeq \frac{x^{k+1} - x^k}{\Delta t}$ si ha:

$$x^{k+1} = x^k + \underbrace{\Delta t}_{\text{passo di tempo}} \underbrace{(b - Ax^k)}_{\partial_t x} \quad \text{con } \Delta t = \alpha$$

Se anziché scegliere Δt sempre uguale a ogni iterazione lo scegliamo ogni volta. Questo metodo è detto **del gradiente**.

Osservazione 4.3. Se A è SPD, il problema di trovare x tale che $Ax = b$ è equivalente al problema di trovare x tale che $\varphi(x) \leq \varphi(y) \quad \forall y \in \mathbb{R}^n$ con $\varphi : \mathbb{R}^n \rightarrow \mathbb{R}$ (funzionale) definito come:

$$\varphi(x) = \frac{1}{2} \langle x, Ax \rangle - \langle b, x \rangle \quad \langle b, x \rangle = b^T x = \sum_{i=1}^n b_i x_i$$

Ossia risolvere $Ax = b$ è equivalente a risolvere il problema di minimo di $\varphi(x)$, che rappresenta l'energia della membrana con:

$$\begin{aligned} &\underbrace{\frac{1}{2} \langle x, Ax \rangle}_{\geq 0} \quad \text{energia elastica} \\ &-\langle b, x \rangle \quad \text{energia potenziale} \end{aligned}$$

Proviamo dunque che $Ax = b \Rightarrow \varphi(x)$ è un minimo. Vogliamo dunque che $Ax = b \Rightarrow \nabla \varphi = 0$. Calcoliamo la derivata direzionale di $\varphi(x)$ in una direzione fissata $h \in \mathbb{R}^n$:

$$D_h \varphi \Big|_x = \lim_{\varepsilon \rightarrow 0} \frac{\varphi(x + \varepsilon h) - \varphi(x)}{\varepsilon}$$

Vediamo il termine $\varphi(x + \varepsilon h)$:

$$\begin{aligned} \varphi(x + \varepsilon h) &= \frac{1}{2} \langle x + \varepsilon h, A(x + \varepsilon h) \rangle - \langle b, x + \varepsilon h \rangle \\ &= \frac{1}{2} \langle x, Ax \rangle + \frac{1}{2} \varepsilon \langle h, Ax \rangle + \frac{1}{2} \varepsilon \langle x, Ah \rangle + \frac{1}{2} \varepsilon^2 \langle h, Ah \rangle - \langle b, x \rangle - \varepsilon \langle b, h \rangle \end{aligned}$$

Mentre il termine $\varphi(x) = \frac{1}{2}\langle x, Ax \rangle - \langle b, x \rangle$, dunque:

$$\varphi(x + \varepsilon h) - \varphi(x) = \varepsilon \langle h, Ax \rangle - \varepsilon \langle b, h \rangle + \frac{1}{2} \varepsilon^2 \langle h, Ah \rangle$$

e si ottiene:

$$D_h \varphi \Big|_x = \lim_{\varepsilon \rightarrow 0} \frac{\varepsilon \langle h, Ax \rangle - \varepsilon \langle b, h \rangle + \overbrace{\frac{1}{2} \varepsilon^2 \langle h, Ah \rangle}^{\rightarrow 0}}{\varepsilon} = \langle h, Ax \rangle - \langle h, b \rangle = \langle h, Ax - b \rangle$$

e dato che $D_h \varphi = \nabla \varphi \cdot h$ ($\langle h, \nabla \varphi \rangle$), si ha:

$$Ax = b \implies \nabla \varphi \Big|_x = 0$$

L'hessiano $\mathcal{H}(\varphi) = A$ è una matrice simmetrica, quindi è diagonalizzabile e ha autovalori tutti positivi, per cui è SPD. E dunque x è un minimo (A definita positiva). Infatti:

$$\varphi(x + \varepsilon h) = \varphi(x) + \underbrace{\varepsilon \langle h, Ax - b \rangle}_{\rightarrow 0} + \underbrace{\frac{1}{2} \varepsilon^2 \langle h, Ah \rangle}_{> 0 \forall h \in \mathbb{R}^n \setminus \{0\} (A \text{ SPD})}$$

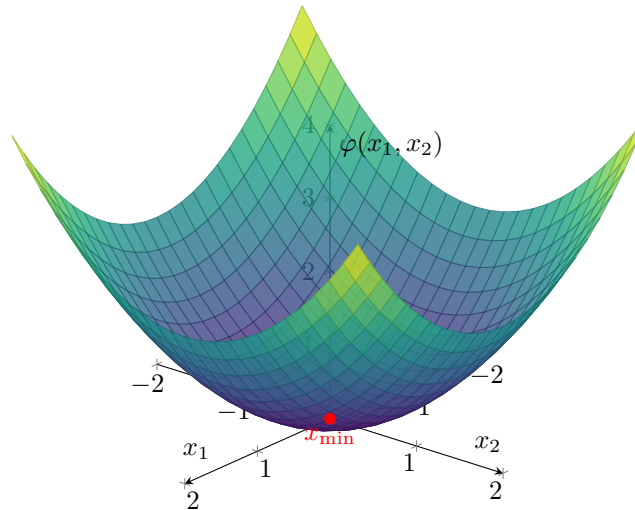
perciò $\varphi(x + \varepsilon h) > \varphi(x) \forall h \in \mathbb{R}^n \setminus \{0\} \forall \varepsilon > 0$. □

Omettiamo di dimostrare il viceversa, ovvero che $\varphi(x)$ è un minimo $\implies Ax = b$.

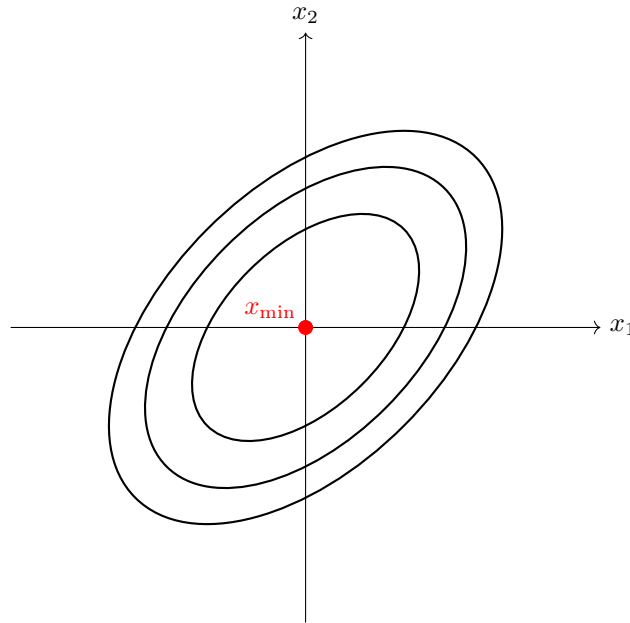
4.4 Metodo del gradiente

$$\varphi(x) = \frac{1}{2} \langle x, Ax \rangle - \langle b, x \rangle$$

dove A è SPD $n \times n$. Cerchiamo x tale da minimizzare $\varphi(x)$. Se $x \in \mathbb{R}^2$ possiamo visualizzare la situazione con un grafico:



E le curve di livello sono:



Dato x^0 , mi posso muovere nella **direzione di massima discesa** verso il minimo x grazie all'**antigradiente**

$$-\nabla\varphi = b - Ax = r \quad (\nabla\varphi = Ax - b)$$

Quando scriviamo

$$x^{k+1} = x^k + \alpha_k r^k$$

si procede, a partire da x^k , lungo la direzione r^k (di massima discesa) con un passo α_k opportunamente scelto. Il punto x^{k+1} ottimale lungo questa direzione si ottiene quando il vettore di spostamento è tangente alla curva di livello, cioè quando il nuovo gradiente è ortogonale al passo effettuato.

Vogliamo cioè α tale che $\underbrace{\varphi(x^k + \alpha r^k)}_{\text{minimo lungo } r} = g(\alpha)$, funzione scalare $g : \mathbb{R} \rightarrow \mathbb{R}$. Ovvero α tale che

$g'(\alpha) = 0$ (direzione di discesa che minimizza il funzionale φ). Calcoliamo:

$$\begin{aligned} g'(\alpha) &= \nabla\varphi \Big|_{x^k + \alpha r^k} \cdot r^k \\ &= \langle A(x^k + \alpha r^k) - b, r^k \rangle \\ &= \langle Ax^k - b, r^k \rangle + \alpha \langle Ar^k, r^k \rangle \\ &= -\langle r^k, r^k \rangle + \alpha \langle Ar^k, r^k \rangle \end{aligned}$$

Ponendo $g'(\alpha) = 0$ si ha:

$$-\langle r^k, r^k \rangle + \alpha \langle Ar^k, r^k \rangle = 0,$$

da cui il **passo ottimale**

$$\alpha_k = \frac{\langle r^k, r^k \rangle}{\langle Ar^k, r^k \rangle}.$$

Algorithm 5 Metodo del gradiente

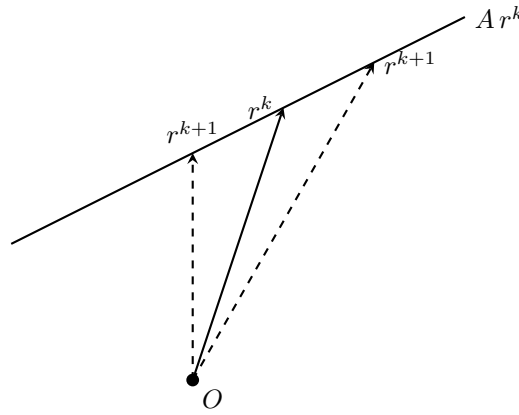
```

1:  $x^0 \leftarrow b$  (stima iniziale)
2:  $r^0 \leftarrow b - Ax^0$ 
3: for  $k = 0, 1, 2, \dots$  do
4:   Calcolo di  $Ar^k$ 
5:    $\alpha_k \leftarrow \frac{\langle r^k, r^k \rangle}{\langle Ar^k, r^k \rangle}$ 
6:    $x^{k+1} \leftarrow x^k + \alpha_k r^k$ 
7:    $r^{k+1} \leftarrow b - Ax^{k+1} (= r^k - \alpha_k Ar^k)$ 
8:   Test di arresto
9: end for

```

L'operazione più costosa è il calcolo di Ar^k .

Osservazione 4.4. Quando scriviamo $r^{k+1} = r^k - \alpha_k Ar^k$ geometricamente possiamo vederlo come:



Dove i possibili $r^{k+1} \in r^k + \text{span}(Ar^k)$. Il valore α_{ott} rende r^{k+1} ortogonale a Ar^k . Pongo $r^{k+1} \perp Ar^k$ e ottengo:

$$\begin{aligned}
 \langle r^{k+1}, Ar^k \rangle &= 0 \\
 \langle r^k - \alpha_k Ar^k, Ar^k \rangle &= 0 \\
 \langle r^k, Ar^k \rangle - \alpha_k \langle Ar^k, Ar^k \rangle &= 0 \\
 \alpha_k &= \frac{\langle r^k, Ar^k \rangle}{\langle Ar^k, Ar^k \rangle}
 \end{aligned}$$

Osservazione 4.5. Non è richiesto che A sia SPD, ma è richiesto che A sia *simmetrica*.

4.5 Metodo Ortomin

Dato x^0 , $r^0 = b - Ax^0$ si calcola:

$$x^{k+1} = x^k + \alpha_k r^k$$

Scegliendo α_k in modo che $r^{k+1} \perp Ar^k$ si ha:

$$\alpha_k = \frac{\langle r^k, Ar^k \rangle}{\langle Ar^k, Ar^k \rangle}$$

Dato che A non è SPD non è detto che r^k sia una direzione di discesa, perché per matrici non SPD non vale l'equivalenza:

$$Ax - b = 0 \iff \text{si trova minimo di } \varphi(x)$$

Calcolo $\|r^{k+1}\|^2$ per vedere se $\|r^{k+1}\| < \|r^k\|$.

$$\begin{aligned}\|r^{k+1}\|^2 &= \langle r^{k+1}, r^{k+1} \rangle = \\ &= \langle r^k - \alpha_k Ar^k, r^k - \alpha_k Ar^k \rangle \\ &= \langle r^k, r^k \rangle - \alpha_k \langle Ar^k, r^k \rangle - \alpha_k \langle r^k, Ar^k \rangle + \alpha_k^2 \langle Ar^k, Ar^k \rangle\end{aligned}$$

Infatti $\langle r^k, \alpha_k Ar^k \rangle = \sum_{i=1}^n r_i^k (\alpha_k (Ar^k)_i) = \alpha_k \sum_{i=1}^n r_i^k (Ar^k)_i$. Siccome tutte le componenti sono reali il prodotto interno è simmetrico, quindi:

$$\begin{aligned}\|r^{k+1}\|^2 &= \|r^k\|^2 - 2\alpha_k \langle r^k, Ar^k \rangle + \alpha_k^2 \langle Ar^k, Ar^k \rangle \\ \text{Sostituisco } \alpha_k & \\ &= \|r^k\|^2 - 2 \frac{\langle r^k, Ar^k \rangle}{\langle Ar^k, Ar^k \rangle} \langle r^k, Ar^k \rangle + \left(\frac{\langle r^k, Ar^k \rangle}{\langle Ar^k, Ar^k \rangle} \right)^2 \langle Ar^k, Ar^k \rangle \\ &= \|r^k\|^2 - \frac{\langle r^k, Ar^k \rangle^2}{\langle Ar^k, Ar^k \rangle} \\ &= \|r^k\|^2 - \underbrace{\frac{\langle r^k, Ar^k \rangle^2}{\|Ar^k\|^2}}_{\geq 0}\end{aligned}$$

Osservazione 4.6. *Ortomin converge se $\langle r^k, Ar^k \rangle \neq 0$. Se $\langle r^k, Ar^k \rangle$ diventa zero per qualche k allora il residuo non diminuisce più e il metodo non converge.*

Se A è SPD allora $\langle x, Ax \rangle > 0 \forall x \neq 0$ e Ortomin converge.

Se A non è SPD, devo garantire che $\langle x, Ax \rangle \neq 0 \forall x \neq 0$. Questo può essere garantito se

$$0 \notin \Lambda = \{\lambda_i \in \mathbb{C}\} \text{ autovalori di } A$$

cioè se la convex hull (un insieme convesso che racchiude tutti i λ_i) non contiene 0, per cui Ortomin converge.

Metodo del gradiente si applica su matrici SPD per le quali so $\langle x, Ax \rangle > 0 \forall x \neq 0$. Posso definire una norma basata sulla matrice A :

$$\|x\|_A = \langle x, Ax \rangle \quad \text{norma dell'energia}$$

è detta norma dell'energia in quanto $\frac{1}{2} \langle x, Ax \rangle$ è l'energia elastica del sistema. Definisco il prodotto interno secondo A come:

$$\langle x, y \rangle_A = \langle x, Ay \rangle = \langle Ax, y \rangle \quad A \text{ simmetrica}$$

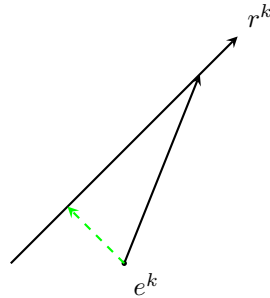
x e y sono A -ortogonali ($\perp_A$) se $\langle x, Ay \rangle = 0$.

Nel metodo del gradiente

$$x^{k+1} = x^k + \alpha_k r^k$$

definisco l'errore $e^k = x - x^k$ dove x è la soluzione esatta.

$$e^{k+1} = x - x^{k+1} = x - x^k - \alpha_k r^k = e^k - \alpha_k r^k$$



$$e^{k+1} = e^k + \text{span}(r^k)$$

Si vuole che (la freccia tratteggiata verde) sia e^{k+1} . Imponendo che $e^{k+1} \perp_A r^k$ ottengo:

$$\begin{aligned} \langle e^{k+1}, Ar^k \rangle &= 0 \\ &= \langle e^k - \alpha_k r^k, Ar^k \rangle \\ &= \langle e^k, Ar^k \rangle - \alpha_k \langle r^k, Ar^k \rangle \quad A \text{ simmetrica} \\ &= \langle Ae^k, r^k \rangle - \alpha_k \langle r^k, Ar^k \rangle \end{aligned}$$

Noto che $Ae^k = A(x - x^k) = b - Ax^k = r^k$. Quindi:

$$\langle e^{k+1}, Ar^k \rangle = \langle r^k, r^k \rangle - \alpha_k \langle r^k, Ar^k \rangle = 0$$

per $\alpha_k = \frac{\langle r^k, r^k \rangle}{\langle r^k, Ar^k \rangle}$, che è proprio il metodo del gradiente.

4.6 Rette e piani

Rette

$$ax + by + c = 0 \quad \text{retta nel piano}$$

Nello spazio una retta ha direzione $\underline{r} \in \mathbb{R}^n$. Definisco una retta parallela a $\underline{r}$ come:

$$\{tv\} \quad t \in \mathbb{R}$$

che passa per l'origine. Mentre una retta passante per $\underline{a}$ ma non per l'origine è:

$$\{a + tv\} \quad t \in \mathbb{R}$$

Se x sta sulla retta: $x \in \underline{a} + \text{span}(\underline{r})$.

Piani Piano nello spazio:

$$ax + by + cz + d = 0$$

Piano in $\mathbb{R}^n$ definito da 2 direzioni v, w linearmente indipendenti:

$$\{t_1 v + t_2 w\} \quad t_1, t_2 \in \mathbb{R} \quad v, w \in \mathbb{R}^n$$

Questo piano passa per l'origine, ma posso definirlo passante per un punto $\underline{x} \in \mathbb{R}^n \in \Pi$ (piano) come:

$$\underline{x} \in \underline{z} + \text{span}(\underline{v}, \underline{w})$$

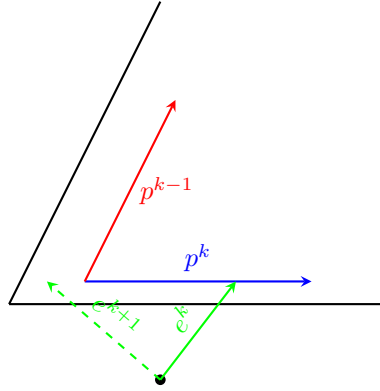
4.7 Gradiente coniugato

Si cerca la soluzione ottimale “muovendosi” su un piano che sia “di discesa ottimale”. Data A SPD:

$$x^{k+1} = x^k + \alpha_k p^k \quad p^k \text{ direzione di discesa}$$

Prendo $p^k = r^k - \beta_k p^{k-1}$ come nuova direzione di discesa, che considera sia la discesa ottimale (r^k) e una di “memoria” lungo la direzione precedente (p^{k-1}). Quindi si hanno 2 parametri liberi α_k e β_k , allora posso imporre 2 condizioni.

1. $e^{k+1} \perp_A p^k$ e
2. $e^{k+1} \perp_A p^{k-1}$



dove $e^{k+1} \perp_A$ al piano Π e $\Pi = e^k + \text{span}(p^k, p^{k-1})$.

Verifichiamo se è possibile trovare α_k e β_k che soddisfano queste 2 condizioni:

1. $e^{k+1} \perp_A p^k$

$$\begin{aligned}
 \langle e^{k+1}, Ap^k \rangle &= 0 \\
 &= \langle x - x^{k+1}, Ap^k \rangle \\
 &= \langle x - x^k - \alpha_k p^k, Ap^k \rangle \\
 &= \langle e^k - \alpha_k p^k, Ap^k \rangle \quad \text{linearità} \\
 &= \langle e^k, Ap^k \rangle - \alpha_k \langle p^k, Ap^k \rangle \quad A^T = A \\
 &= \langle Ae^k, p^k \rangle - \alpha_k \langle p^k, Ap^k \rangle \quad Ae^k = r^k \\
 &\implies \alpha_k = \frac{\langle r^k, p^k \rangle}{\langle p^k, Ap^k \rangle}
 \end{aligned}$$

Osservazione 4.7. Se scegliamo $p^k = r^k$ allora avremmo ottenuto il metodo del gradiente e avrei $\beta_k = 0$.

2. $e^{k+1} \perp_A p^{k-1}$

$$\begin{aligned}
 \langle e^{k+1}, Ap^{k-1} \rangle &= 0 \\
 &= \langle x - x^{k+1}, Ap^{k-1} \rangle \\
 &= \langle x - x^k - \alpha_k p^k, Ap^{k-1} \rangle \\
 &= \langle e^k, Ap^{k-1} \rangle - \langle \alpha_k p^k, Ap^{k-1} \rangle \quad \text{dall'iterazione precedente so che } \langle e^k, Ap^{k-1} \rangle = 0 \\
 &= \alpha_k \langle p^k, Ap^{k-1} \rangle \quad \text{sostituisco } p^k = r^k - \beta_k p^{k-1} \\
 &= \langle r^k - \beta_k p^{k-1}, Ap^{k-1} \rangle \\
 &= \langle r^k, Ap^{k-1} \rangle - \beta_k \langle p^{k-1}, Ap^{k-1} \rangle \\
 &= \beta_k = \frac{\langle r^k, Ap^{k-1} \rangle}{\langle p^{k-1}, Ap^{k-1} \rangle}
 \end{aligned}$$

Osservazione 4.8. $\langle x, Ax \rangle > 0 \quad \forall x \neq 0$ dato che A è SPD.

Quindi riepilogando:

$$\alpha_k = \frac{\langle r^k, p^k \rangle}{\langle p^k, Ap^k \rangle} \implies e^{k+1} \perp_A p^k$$

e

$$\beta_k = \frac{\langle r^k, Ap^{k-1} \rangle}{\langle p^{k-1}, Ap^{k-1} \rangle} \implies e^{k+1} \perp_A p^{k-1}$$

per cui dato x^0 , $r^0 = b - Ax^0$, $p^0 = r^0$:

Algorithm 6 Metodo del gradiente coniugato

```

1:  $x^0 \leftarrow b$  (stima iniziale)
2:  $r^0 \leftarrow b - Ax^0$ 
3:  $p^0 \leftarrow r^0$ 
4: for  $k = 0, 1, 2, \dots$  do
5:   Calcolo di  $Ap^k$ 
6:    $\alpha_k \leftarrow \frac{\langle r^k, p^k \rangle}{\langle p^k, Ap^k \rangle}$ 
7:    $x^{k+1} \leftarrow x^k + \alpha_k p^k$ 
8:    $r^{k+1} \leftarrow r^k - \alpha_k Ap^k$ 
9:    $\beta_{k+1} \leftarrow \frac{\langle r^{k+1}, Ap^k \rangle}{\langle p^k, Ap^k \rangle}$ 
10:   $p^{k+1} \leftarrow r^{k+1} - \beta_{k+1} p^k$ 
11:  Test di arresto
12: end for

```

Il test di arresto può essere:

$$\|r^{k+1}\| < \text{tol} \|r^0\| \quad \text{sul residuo}$$

$$\|x^{k+1} - x^k\| < \text{tol} \|x^k\| \quad \text{sulle iterazioni successive}$$

Il costo di 1 iterazione è $\simeq$ al costo per calcolare Ap^k . Gli altri costi ($\alpha_k, \beta_k, x^{k+1}, p^{k+1}$) sono $O(n)$ (prodotti scalari). Quindi il costo del gradiente e del gradiente coniugato è comparabile, ma il gradiente coniugato converge in $\ll$ iterazioni. Vediamo dunque perché questo metodo funziona. Abbiamo che:

$$\begin{aligned}
p^0 &= r^0 & p^0 &\in \text{span}(r^0) \\
p^1 &= r^1 - \beta_1 p^0 & p^1 &\in \text{span}(r^0, r^1) \\
p^2 &= r^2 - \beta_2 p^1 & p^2 &\in \text{span}(r^0, r^1, r^2) \\
&\vdots & & \\
p^k &= r^k - \beta_k p^{k-1} & p^k &\in \text{span}(r^0, r^1, \dots, r^k)
\end{aligned}$$

Si dimostra, per induzione, che gli r^k sono tutti ortogonali, per cui tutte le direzioni p^k sono mutuamente A -ortogonali. L'errore e^{k+1} è A -ortogonale a $p^k, p^{k-1}, \dots, p^0$. Se A è $n \times n$ allora $e^{k+1} \perp_A \text{span}(p^0, p^1, \dots, p^k) = \mathbb{R}^n$, per cui $e^{k+1} = 0$ (0 è ortogonale a tutti i vettori dello spazio). Cioè il metodo GC (gradiente coniugato) dà la soluzione esatta dopo $n + 1$ iterazioni.

Se applico la stessa logica a **ortomin** calcolo:

$$x^{k+1} = x^k + \alpha_k p^k$$

e considero α_k tale che $r^{k+1} \perp Ar^k$ (ortomin 1). Posso anche calcolare

$$p^k = r^k - \beta_k r^{k-1}$$

e considero β_k tale che $r^{k+1} \perp Ar^{k-1}$ (ortomin 2).

Poiché non eredito l'ortogonalità rispetto alle direzioni precedenti, se A non è SPD. Per accelerare ortomin posso imporre a mano l'ortogonalità di p^k rispetto a tutte le direzioni precedenti.

$$p^k = r^k - \beta_k r^{k-1} - \beta_{k-1} r^{k-2} - \dots - \beta_1 r^0$$

Otengo così il metodo **GMRES** (Generalized Minimal Residual), che è uno dei pochi metodi veloci per matrici non simmetriche.

Si dimostra che ϱ :

$$\|e^k\| \leq \varrho^k \|e^0\|$$

per Richardson 4.2.2:

$$\varrho = \frac{K(A) - 1}{K(A) + 1}$$

per GC 4.7:

$$\varrho = \frac{\sqrt{K(A)} - 1}{\sqrt{K(A)} + 1}$$

Se quindi ho ad esempio $K(A) = 10^6$ allora:

$$\varrho_{GC} = \frac{1000 - 1}{1000 + 1}$$

$$\varrho_{Richardson} = \frac{10^6 - 1}{10^6 + 1}$$

5 Interpolazione polinomiale

Polinomi di Taylor sono un esempio di approssimazione per una funzione, tuttavia ha il limite di approssimare la funzione centrandola in un punto. L'Interpolazione polinomiale vuole che data una funzione $f \in C^\infty([a, b])$ scelgo dei nodi $a \leq x_0 \leq \dots \leq x_n \leq b$. L'obiettivo è trovare un polinomio $p(x) \in \mathbb{R}^n$ con $\mathbb{R}^n$ spazio (vettoriale) dei polinomi di grado al più n .

$$p(x) = a_0 + a_1x + \dots + a_nx^n$$

tale che $p(x_i) = f(x_i)$ per $i = 0 \dots n$

Il polinomio $p(x)$ è univocamente determinato dai coefficienti della **combinazione lineare** $[a_0 \dots a_n] \in \mathbb{R}^{n+1}$ ($\mathbb{R}^{n+1} \approx \mathbb{P}^n$). Per trovare gli a_i devo risolvere il sistema lineare $Va = f$

$$\underbrace{\begin{bmatrix} 1 & x_0 & x_0^2 & \dots & x_0^n \\ 1 & x_1 & x_1^2 & \dots & x_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^n \end{bmatrix}}_{V \in \mathbb{R}^{(n+1) \times (n+1)}} \underbrace{\begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{bmatrix}}_{a \in \mathbb{R}^{n+1}} = \underbrace{\begin{bmatrix} f(x_0) \\ f(x_1) \\ \vdots \\ f(x_n) \end{bmatrix}}_{f \in \mathbb{R}^{n+1}}$$

Caso particolare in cui ho $n + 1$ nodi e cerco polinomio di grado n , V è dunque una matrice quadrata (altrimenti rettangolare). Il sistema è risolvibile se il determinante di V è diverso da zero. Dimosteremo che V è invertibile se e solo se $x_i \neq x_j \quad i \neq j$. V è detta matrice di Vandermonde.

Osservazione 5.1. Se $x_i \approx x_j$ e $i \neq j$ allora V è mal condizionata (vicina a essere singolare). Condizionamento fornisce un'indicazione di quanto una matrice sia "vicina" alla singolarità.

Se $\exists V^{-1}$ allora $\exists!$ la soluzione del sistema lineare e quindi $\exists! p(x) \in \mathbb{P}^n$ che interpola f nei nodi $x_0 \dots x_n$. Se voglio vedere il grafico di $p(x)$ e confrontarlo con la funzione $f(x)$ non posso basarmi esclusivamente sui valori di p ed f sui nodi di interpolazione, dato che $p(x_i) = f(x_i)$ per costruzione. Devo quindi utilizzare una griglia Y (un insieme di punti in $[a, b]$) molto più fitta di $X = \{x_0 \dots x_n\}$.

Osservazione 5.2. La valutazione di $p(x)$ sulla griglia Y può essere efficientata utilizzando il seguente algoritmo:

Algorithm 7 Metodo di Horner

```

1:  $p \leftarrow a_n$ 
2: for  $i = n - 1, n - 2, \dots, 0$  do
3:    $p \leftarrow a_i + y \cdot p$ 
4: end for
```

Il metodo di Horner richiede $O(2n)$ operazioni.

Dimostriamo dunque che V è invertibile.

Dimostrazione. Costruisco p utilizzando i polinomi di Lagrange:

$$l_i(x_j) = \begin{cases} 1 & \text{se } i = j \\ 0 & \text{se } i \neq j \end{cases}$$

Scrivo l_i mettendo in evidenza le sue radici.

$$l_i(x) = c \prod_{j=0, j \neq i}^n (x - x_j) \quad x_j \in X \text{ insieme dei nodi}$$

Sapendo che vogliamo $1 = l_i(x_i) = c \prod_{j=0, j \neq i}^n (x_i - x_j)$ allora $c = \frac{1}{\prod_{j=0, j \neq i}^n (x_i - x_j)}$. Dunque:

$$l_i(x) = \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j}$$

Osservazione 5.3. • $l_i(x)$ soddisfa la condizione di $l_i(x)$

$$l_i(x_j) = \prod_{j=0, j \neq i}^n \frac{x_j - x_j}{x_i - x_j} = 0 \quad l_i(x_i) = \prod_{j=0, j \neq i}^n \frac{x_i - x_j}{x_i - x_j} = 1$$

• $l_i(x) \in \mathbb{P}^n$ (produttoria ha n prodotti)

Dimostriamo che gli $\{l_i\}$ sono linearmente indipendenti. Scrivo una combinazione lineare degli l_i

$$\forall x \sum_{i=0}^n c_i l_i(x) = 0$$

Vorrei che $c_0 = \dots = c_n = 0$. In particolare deve valere per $x = x_j$

$$0 = \sum_{i=0}^n c_i l_i(x_j) = c_j \underbrace{l_j(x_j)}_{=1} \implies c_j = 0$$

Se si applica lo stesso ragionamento $\forall j = 0 \dots n$ si ha che $c_0 = \dots = c_n = 0$. Quindi $\{l_i\}$ sono linearmente indipendenti e sono $n + 1 = \dim(\mathbb{P}^n)$ e dunque sono una base di $\mathbb{P}^n$. Ogni $p \in \mathbb{P}^n$ è una combinazione lineare degli l_i . Per il polinomio interpolatore

$$p(x) = \sum_{i=0}^n c_i l_i(x) \quad \text{con la condizione di interpolazione che } p(x_i) = f(x_i)$$

e dunque:

$$f(x_j) = p(x_j) = \sum_{i=0}^n c_i l_i(x_j) = c_j l_j(x_j) = c_j = f(x_j)$$

Dunque il polinomio interpolatore è dato da:

$$p(x) = \sum_{i=0}^n f(x_i) l_i(x)$$

Questa è detta **interpolazione di Lagrange**. Abbiamo dunque mostrato l'esistenza di questo polinomio interpolatore. Occorre mostrare che è unico.

Supponiamo per assurdo che esistano $p, q \in \mathbb{P}^n$ entrambi polinomi di interpolazione. Dunque

$$p(x_i) = f(x_i) \quad q(x_i) = f(x_i) \quad \forall i = 0 \dots n$$

Considero $p(x) - q(x) \in \mathbb{P}^n$ (spazio lineare). Questo polinomio ha n radici distinte. Per costruzione

$$p(x_i) - q(x_i) = 0 \implies p(x_i) = q(x_i)$$

siccome un polinomio viene identificato dalle sue radici allora $p(x) = q(x)$. Per cui il polinomio interpolatore è unico. $\square$

Esempio 5.1. Interpolare la funzione $f(x) = \cos(\pi x)$ con $x \in [0, 1]$ e vogliamo un polinomio di grado 2 ($n + 1$ nodi necessari). Scelgo dunque i 3 nodi $x_0 = 0$, $x_1 = \frac{1}{2}$, $x_2 = 1$. Dunque $f_0 = 1$, $f_1 = 0$, $f_2 = -1$. Dunque:

$$\begin{bmatrix} 1 & 0 & 0 \\ 1 & \frac{1}{2} & \frac{1}{4} \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}$$

e troviamo che $a_0 = 1$, $a_1 = -2$, $a_2 = 0$. Dunque il polinomio interpolatore è:

$$p(x) = 1 - 2x \quad p(0) = 1 \quad p\left(\frac{1}{2}\right) = 0 \quad p(1) = -1$$

Un altro metodo per interpolare è quello di utilizzare i polinomi di Lagrange.

$$p(x) = \sum_{i=0}^n f(x_i) \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j}$$

Calcolo dunque le basi:

$$\begin{aligned} l_0(x) &= \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} = \frac{(x - \frac{1}{2})(x - 1)}{(-\frac{1}{2})(-1)} \\ l_1(x) &= \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} = \frac{x(x - 1)}{(\frac{1}{2})(-\frac{1}{2})} \\ l_2(x) &= \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)} = \frac{x(x - \frac{1}{2})}{1(\frac{1}{2})} \end{aligned}$$

Dunque:

$$p(x) = f_0 l_0(x) + f_1 l_1(x) + f_2 l_2(x)$$

Osservazione 5.4. La base di Lagrange dipende dalla griglia X scelta.

5.1 Errore di interpolazione Lagrange

Possiamo scrivere l'errore di interpolazione come:

$$e_n(x) = f(x) - p(x)$$

per costruzione $e_n(x_i) = 0$ per gli x_i scelti come nodi. Posso fattorizzare $e_n(x)$ come:

$$e_n(x) = \prod_{i=0}^n (x - x_i) \cdot R(x)$$

dove $R(x)$ è un resto che dobbiamo stimare. Definiamo una funzione ausiliaria $g(t, x)$ come:

$$g(t, x) = f(t) - p(t) - w_n(t)R(x)$$

dove $w_n(t) = \prod_{i=0}^n (t - x_i)$ (detto **polinomio nodale**).

$$g(x_i, x) = f(x_i) - p(x_i) - w_n(x_i)R(x) \quad \text{ha } n + 1 \text{ zeri } \forall i = 0, \dots, n$$

E:

$$g(x, x) = \underbrace{f(x) - p(x) - w_n(x)R(x)}_{e_n(x) - e_n(x)} = 0$$

dunque $g(t, x)$ ha $n + 2$ zeri distinti. Quindi g è regolare C^∞ se $f \in C^\infty([a, b])$ per il teorema di Rolle avremo almeno $n + 1$ punti in cui la derivata di g si annulla. Iterando il ragionamento avremo che:

$$\frac{d^{n+1}}{dt^{n+1}} g \quad \text{ha 1 zero}$$

Sia ξ tale che

$$\left. \frac{d^{n+1}g(t, x)}{dt^{n+1}} \right|_{\xi} = 0$$

calcolo dunque:

$$\frac{d^{n+1}g(t, x)}{dt^{n+1}} = f^{(n+1)}(t) - 0 - (n+1)!R(x)$$

e dato che in ξ si annulla la derivata $n+1$ -esima allora:

$$0 = f^{(n+1)}(\xi) - (n+1)!R(x) \implies R(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!}$$

E moltiplicando per $w_n(x)$ ottengo:

$$e_n(x) = \frac{1}{(n+1)!} f^{(n+1)}(\xi) w_n(x)$$

Proposizione 5.1. Sia $A \in \mathbb{R}^{n \times n}$ non singolare.

$$\frac{1}{K(A)} = \min \left\{ \frac{\|\Delta A\|}{\|A\|} \mid A + \Delta A \text{ è singolare} \right\}$$

Distanza da A dall'essere una matrice singolare (dove ΔA è una perturbazione di A).

Se per l'interpolazione polinomiale uso una base canonica $\{1, x, x^2, \dots, x^n\}$ allora:

$$p(x) = a_0 + a_1x + \dots + a_nx^n$$

e posso trovare gli a_i risolvendo il sistema lineare con la matrice di Vandermonde. Abbiamo anche osservato che per vedere il polinomio occorre costruire una griglia più fitta di X (detta **griglia grafica**). Si possono anche utilizzare altre basi, come la base di Lagrange

$$l_i(x) = \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j} \in \mathbb{P}^n \implies p(x) = \sum_{i=0}^n f(x_i) l_i(x)$$

cui l'errore di interpolazione è:

$$e_n(x) = f(x) - p(x) = w_{n+1}(x) \cdot \frac{f^{(n+1)}(\xi)}{(n+1)!} \quad \xi \in (a, b)$$

e il polinomio nodale $w_{n+1}(x) = \prod_{i=0}^n (x - x_i)$.

Osservazione 5.5. Quando costruiamo il polinomio interpolante su $n+1$ nodi, per costruzione abbiamo:

$$e_n(x_i) = f(x_i) - p_n(x_i) = 0 \quad \text{per } i = 0, 1, \dots, n,$$

dato che $p_n(x_i)$ coincide con $f(x_i)$ in corrispondenza dei nodi. Tuttavia, l'errore tra due nodi consecutivi può diventare grande se la distanza fra i nodi è ampia, soprattutto verso gli estremi dell'intervallo. Per ridurre queste oscillazioni e limitare l'errore, è spesso conveniente **addensare i nodi verso i bordi** dell'intervallo (ad esempio tramite nodi di Gauss-Lobatto), piuttosto che usare nodi equispaziati. In questo modo si contiene la crescita del polinomio nodale alle estremità e si riduce notevolmente il fenomeno di Runge, ovvero l'oscillazione violentemente ai bordi dell'intervallo.

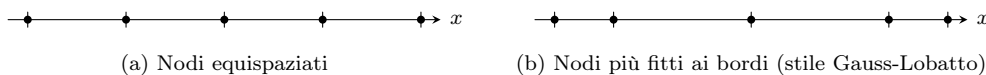


Figura 4: Confronto tra nodi equispaziati (a) e nodi più densi verso gli estremi (b).

$$x_j = \frac{a+b}{2} - \frac{b-a}{2} \cos\left(\frac{j\pi}{n}\right) \quad j = 0 \dots n$$

$$j = 0 \implies x_0 = \frac{a+b}{2} - \frac{b-a}{2} \cos(0) = a$$

$$j = n \implies x_n = \frac{a+b}{2} - \frac{b-a}{2} \cos(\pi) = b$$

Per $n = 5$ e $[a, b] = [-2, 2]$ si ha:

$$x_0 = -2 \quad x_1 = -1.618 \quad x_2 = -0.618 \quad x_3 = -x_2 \quad x_4 = -x_1 \quad x_5 = 2$$

Estrapolazione

Costruisco $p(x)$ su una griglia $X \subset [a, b]$ e uso $p(x)$ per stimare $f(x)$ fuori da $[a, b]$. La formula dell'errore non cambia, ma la funzione nodale diventa più grande via via che x si allontana da $[a, b]$. Per cui è naturale che si ottengano errori maggiori.

Teorema 5.1 (Weierstrass). $\forall f \in C([a, b])$, $\forall \varepsilon > 0$, $\exists n(\varepsilon)$ e un polinomio $p \in \mathbb{P}^n$ tale che:

$$\|f - p\|_\infty < \varepsilon \quad (\|f\|_\infty = \max_{x \in [a, b]} |f(x)|)$$

Inoltre $\exists$ un polinomio $p^* \in \mathbb{P}^n$ ottimale tale che:

$$E_n^* = \|f - p^*\|_\infty \leq \|f - q\|_\infty \quad \forall q \in \mathbb{P}^n$$

Si dimostra pure che il polinomio di interpolazione p sulla griglia X soddisfa:

$$\|f - p\|_\infty \leq E_n^*(1 + \Lambda_n(X))$$

dove $\Lambda_n(X) > 0$ è detta costante di Lebesgue e dipende dalla griglia X :

$$\Lambda_n(X) = \max_{x \in [a, b]} \sum_{i=0}^n |l_i(x)|$$

Osservazione 5.6. Scegliere una griglia vuol dire scegliere dei nodi di interpolazione $x_0 \dots x_n$.

Si dimostra che $\Lambda_n(X) \sim \log(n+1)$ se utilizzo una griglia di Gauss-Lobatto oppure $\Lambda_n(X) \sim \frac{2^{n+1}}{n \log n}$ se utilizzo una griglia uniforme.

Condizionamento nell'interpolazione polinomiale

Suppongo di avere dei dati $f_0 \dots f_n$ ($f_i = f(x_i)$) ma utilizzo dei dati con errore $\tilde{f}_0 \dots \tilde{f}_n$ ($\tilde{f}_i = f(x_i) + \delta_i$) con δ_i un errore (di misura o di tipo floating). Se $p \in \mathbb{P}^n$ su una griglia X per i nodi $f_0 \dots f_n$ e chiamo $\tilde{p} \in \mathbb{P}^n$ il polinomio interpolatore sui dati perturbati $\tilde{f}_0 \dots \tilde{f}_n$ su X (stessa griglia) vogliamo stimare $p - \tilde{p}$. Calcoliamo:

$$p(x) - \tilde{p}(x) = \sum_{i=0}^n f(x_i) l_i(x) - \sum_{i=0}^n \tilde{f}_i l_i(x)$$

Osservazione 5.7. Gli l_i sono gli stessi dato che X è la stessa.

dunque raccogliendo:

$$\begin{aligned}
 p(x) - \tilde{p}(x) &= \sum_{i=0}^n (f(x_i) - \tilde{f}_i) l_i(x) \\
 |p(x) - \tilde{p}(x)| &= \left| \sum_{i=0}^n (f(x_i) - \tilde{f}_i) l_i(x) \right| \\
 &\leq \sum_{i=0}^n |f(x_i) - \tilde{f}_i| |l_i(x)| \\
 &\leq \max_i |f_i - \tilde{f}_i| \sum_{i=0}^n |l_i(x)| \\
 &\leq \max_i |f_i - \tilde{f}_i| \underbrace{\max_x \sum_{i=0}^n |l_i(x)|}_{\text{costante di Lebesgue } \Lambda_n(X)}
 \end{aligned}$$

Dunque:

$$\|p(x) - \tilde{p}(x)\|_{\infty} \leq \|f - \tilde{f}\|_{\infty} \Lambda_n(X)$$

Veniamo ora allo **pseudocodice**:

Il nostro programma prende in input a, b, n (grado del polinomio) e una funzione.

1. Griglia dei nodi:

$X = \text{linspace}(a, b, n+1) \leftarrow$ griglia uniforme

prendiamo poi i valori della funzione sui nodi:

$$Y = f(X)$$

e la griglia grafica:

$$\bar{X} = \text{linspace}(a, b, 101)$$

2. Costruiamo basi di Lagrange:

$$l_i(x) = \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j}$$

posso dunque costruire una funzione:

$$\text{function}[\text{base}] = \text{lagrange}(X, \bar{X})$$

```

1: for i = 1 : length(X) do
2:   tmp=1
3:   for j = 1 : length(X) do
4:     if abs(i - j) > 0 then
5:       tmp = tmp * ( (X(i) - X(j)) / (X(i) - X(j)) )
6:     end if
7:   end for
8:   base(i, :) = tmp
9: end for

```

▷ Sfrutto calcolo vettorizzato

E dunque:

```

1: for i = 1 : length(X̄) do
2:     pesi = lagrange(X, X̄(i))
3:     value(i) = sum(Y.*pesi)
4: end for
5: err = max(abs(value-f(X̄)))
6: figure
7: plot(X̄, f(X̄), X̄, value)

```

6 Sistemi non lineari

Al momento abbiamo lavorato su sistemi lineari: $x \in \mathbb{R}^n$ tale che $Ax + b = 0$ con A matrice $n \times n$ e invertibile. Tutte le volte che qualche parametro dipende dalla soluzione tipicamente ho un problema **non lineare**. Ad esempio nel caso della membrana elastica:

$$\begin{cases} -\Delta u = f' & \text{in } \Omega \\ u = 0 & \text{su } \partial\Omega \end{cases}$$

Problema lineare rappresenta un caso particolare di

$$\begin{cases} -\nabla(\nu \nabla u) = f & \text{in } \Omega \\ u = 0 & \text{su } \partial\Omega \end{cases}$$

Noi abbiamo considerato ν (costante di elasticità della membrana) uguale a 1.

Osservazione 6.1.

$$\Delta = \partial_{xx}^2 + \partial_{yy}^2 \implies \Delta u = \nabla \cdot \nabla u = \partial_x(\partial_x u) + \partial_y(\partial_y u)$$

Se considero $\nu = \nu(u)$, ovvero l'elasticità della membrana dipende dalla deformazione ottengo l'equazione non lineare:

$$\begin{cases} -\nabla(\nu(u) \nabla u) = f & \text{in } \Omega \\ u = 0 & \text{su } \partial\Omega \end{cases}$$

Solitamente localmente un problema non lineare può essere approssimato con un problema lineare. Per esempio:

$$f(x) = \underbrace{f(x_0) + f'(x_0)(x - x_0)}_{\text{parte lineare}} + \underbrace{\frac{1}{2}f''(x_0)(x - x_0)^2}_{\text{parte non lineare}}$$

Se $|x - x_0|$ è piccolo allora $f(x) \simeq f(x_0) + f'(x_0)(x - x_0)$. Lo stesso approccio vale per funzioni $f : \mathbb{R}^n \rightarrow \mathbb{R}$.

$$f(x) = f(x_0) + Jf \Big|_{x_0} (x - x_0) + \frac{1}{2}(x - x_0)^T Hf \Big|_{x_0} (x - x_0) + \dots$$

dove J è lo Jacobiano e H è la Hessiana:

$$(Jf)_{i,j} = \frac{\partial f_i}{\partial x_j}$$

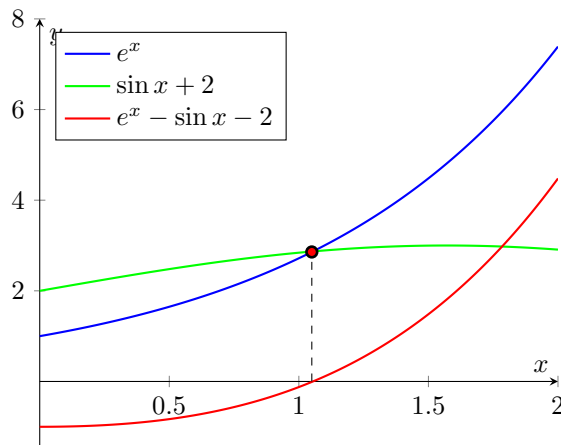
Osservazione 6.2. Se $f : \mathbb{R}^n \rightarrow \mathbb{R}$ ha una sola componente di f e in quel caso:

$$(Jf)_{1,j} = \frac{\partial f}{\partial x_j}$$

6.1 Equazioni scalari non lineari

Trovare y tale che $f(y) = 0$. $f : C([a, b]) \rightarrow \mathbb{R}$ continua.

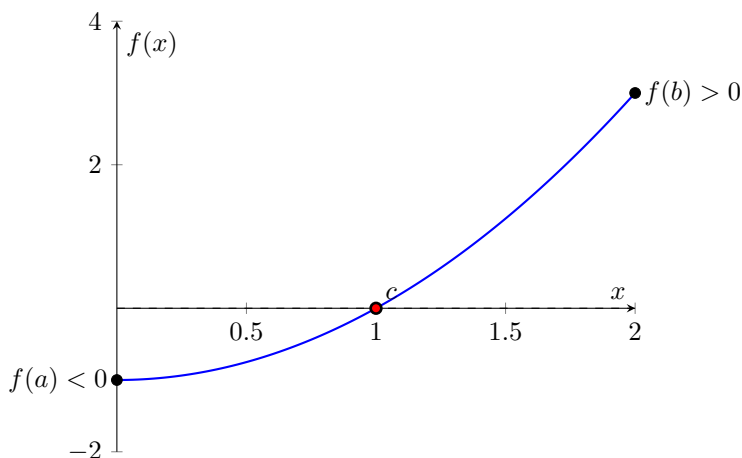
Esempio 6.1. Si trovi intersezione tra $y = e^x$ e $y = \sin x + 2$. Dunque $f(x) = e^x - \sin x - 2$.



Per sapere se c'è uno zero:

$$f \in C([a, b]) \quad \text{e} \quad f(a)f(b) < 0$$

Infatti se f è continua su $[a, b]$ e $f(a)$ e $f(b)$ hanno segno opposto allora per il teorema degli zeri esiste $y \in (a, b)$ tale che $f(y) = 0$.



Teorema degli zeri: se f è continua in $[a, b]$ e $f(a)f(b) < 0$, allora $\exists c \in (a, b)$ tale che $f(c) = 0$.

Considereremo solo le radici semplici, ovvero $f'(y) \neq 0$ (Ad esempio per $f(x) = x^2$ ho una radice doppia in y). Le radici multiple sono tipicamente mal condizionate.

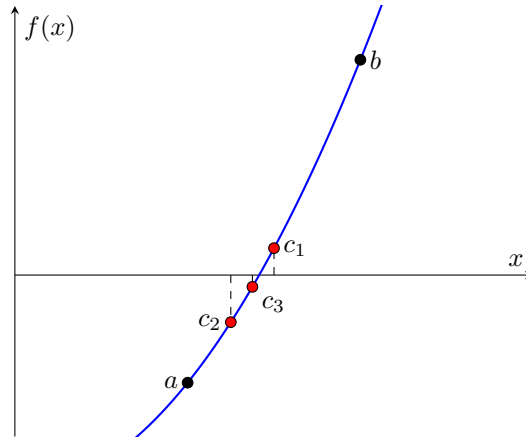
L'idea è quella di trovare una successione $\{x^k\}$ tale che $x^k \rightarrow y$ per $k \rightarrow \infty$. Si tratta sempre di un metodo iterativo.

6.2 Metodo di bisezione

Sia $f : C([a, b])$ con $f(a)f(b) < 0$. Quello che si fa è costruire una successione $\{x^k\}$ tale che:

$$x^k = \frac{a^k + b^k}{2}$$

e si rimpiazza l'intervallo $[a, b]$ con $[a^k, b^k]$. In altre parole considero sempre il punto medio dell'intervallo e calcolo $f(x^k)$. Ad esempio considero $c = \frac{a+b}{2}$ posso calcolare $f(c)$ e se $f(a)f(c) < 0$ allora $y \in (a, c)$, altrimenti $y \in (c, b)$.



Si ripete dunque il procedimento.

```

1:  $[a_0, b_0] = [a, b]$ 
2: for  $k = 0, 1, \dots$  do
3:    $c_k = \frac{a_k + b_k}{2}$ 
4:   if  $f(a_k)f(c_k) < 0$  then
5:      $a_{k+1} = a_k$ 
6:      $b_{k+1} = c_k$ 
7:   else
8:      $a_{k+1} = c_k$ 
9:      $b_{k+1} = b_k$ 
10:  end if
11:  TEST di arresto
12: end for

```

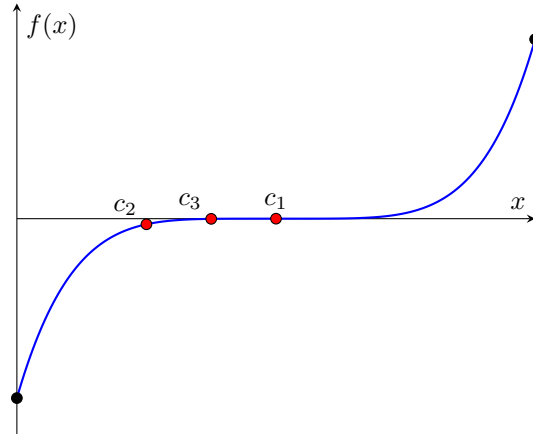
Osservazione 6.3. Come per i sistemi lineari, il **TEST di arresto** può essere:

- *Sul residuo: esci se $|f(x^k)| < c \cdot \text{tol}$. Ricordando che c deve riflettere la grandezza di f , ad esempio:*

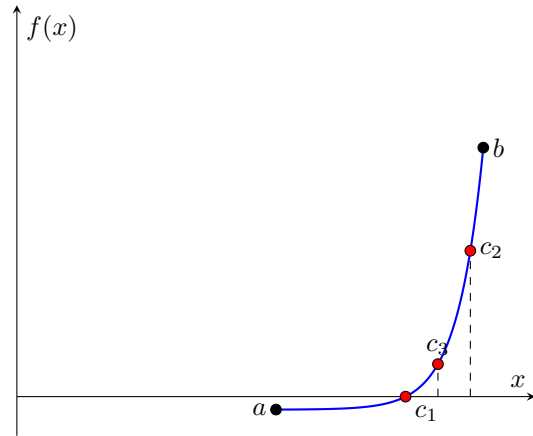
$$c = \frac{|f(a)| + |f(b)|}{2}$$

- *Sulle iterazioni successive: esci se $|b_k - a_k| < c \cdot \text{tol}$. Dove $c = b - a$.*

Solitamente si utilizza una combinazione dei due, dato che il test sul residuo è inaffidabile se $|f(x)| \ll 1$ in un intorno grande della soluzione y che si cerca. Succede quando la funzione è molto "piatta"



Mentre il test sulle iterazioni successive è inaffidabile nel caso opposto, posso avere un residuo grande anche se sono già vicino alla soluzione.



La bisezione genera una successione di intervalli tali:

$$y \in (a_k, b_k) \subset (a_{k-1}, b_{k-1}) \subset \dots \subset (a, b)$$

Dunque vediamo la **convergenza**. Chiamiamo errore al passo k

$$e_k = |y - c_k|$$

Quindi

$$e_k = |y - c_k| \leq (b_k - a_k) = \frac{b_{k-1} - a_{k-1}}{2} = \frac{1}{2^k} (b_0 - a_0) = \frac{1}{2^k} (b - a)$$

Per cui se valgono le ipotesi di partenza $f \in C([a, b])$ e $f(a)f(b) < 0$ allora il metodo di bisezione converge sempre con velocità di convergenza che è indipendente da f . Calcolo k tale che:

$$e_k \leq \varepsilon e_0 \quad \varepsilon = \frac{1}{10} \quad \text{e} \quad e_0 = b - a$$

$$\frac{1}{2^k} (b - a) \leq \varepsilon (b - a)$$

$$\frac{1}{2^k} \leq \varepsilon$$

$$-\ln 2^k \leq \ln \varepsilon$$

$$-k \ln 2 \leq \ln \varepsilon$$

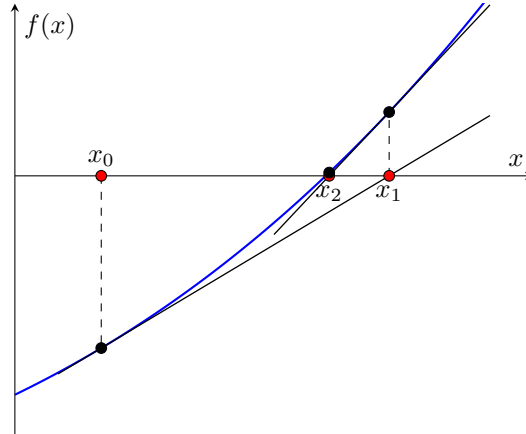
$$k \geq -\frac{\ln \varepsilon}{\ln 2}$$

$$k \geq \frac{\ln 10}{\ln 2} \approx 3.32$$

Ovvero guadagno una cifra decimale ogni 3-4 iterazioni.

6.3 Metodo di Newton

Parto da un punto x_0 , traccio la retta tangente a questo punto e quindi trovo il punto di intersezione della retta tangente con l'asse delle ascisse. Trovo quindi x_1 , su cui posso valutare la nuova tangente e trovare il nuovo punto di intersezione con l'asse delle ascisse.



In formule:

```

1: dato  $x_0$ 
2: for  $k = 0, 1, \dots$  do
3:   calcolo  $x_{k+1}$  tale che:  $0 = f(x_k) + f'(x_k)(x_{k+1} - x_k)$ 
4: end for

```

cioè x_{k+1} è il punto di intersezione della retta $0 = f(x_k) + f'(x_k)(z - x_k)$ con l'asse delle ascisse.

Osservazione 6.4. In questo caso (f scalare di 1 variabile scalare) abbiamo che $Jf|_{x_0}$ è 1×1 e coincide con $f'(x_0)$. Nel caso scalare posso risolvere l'equazione del metodo di Newton:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

Osserviamo inoltre che è necessario avere un punto di partenza $x_0 \in (a, b)$ dove $f(a)f(b) < 0$.

Osservazione 6.5. Non è detto che $x_{k+1} \in (a, b)$ e in particolare **non è detto che il metodo converga**.

L'equazione dell'errore $e_k = y - x_k$ per $f(y) = 0$ diventa:

$$\begin{aligned} e_{k+1} &= y - x_{k+1} = y - x_k + \frac{f(x_k)}{f'(x_k)} \\ &= y - x_k + \frac{f(x_k) - f(y)}{f'(x_k)} \end{aligned}$$

Ricordiamo che $f(y) = 0$, quindi possiamo sviluppare:

$$f(y) = f(x_k) + f'(x_k)(y - x_k) + \frac{1}{2}f''(\xi_k)(y - x_k)^2$$

necessitiamo dunque $f \in C^2([a, b])$

$$\begin{aligned} e_{k+1} &= y - x_k + \frac{f(x_k) - f(x_k) - f'(x_k)(y - x_k) - \frac{1}{2}f''(\xi_k)(y - x_k)^2}{f'(x_k)} \\ &= y - x_k - (y - x_k) - \frac{1}{2} \frac{f''(\xi_k)}{f'(x_k)} (y - x_k)^2 \end{aligned}$$

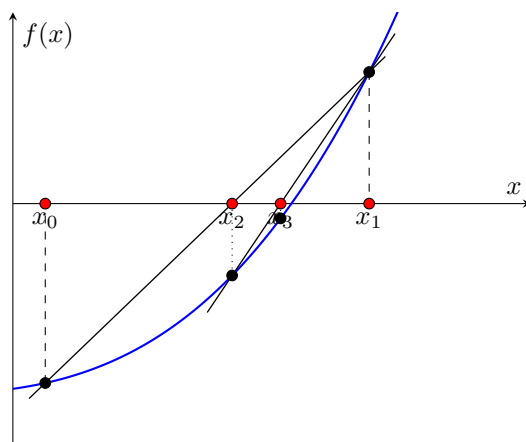
Trovo dunque:

$$e_{k+1} = -\frac{1}{2} \frac{f''(\xi_k)}{f'(x_k)} e_k^2$$

Chiamo $M = \max_{x,z} \left| \frac{f''(z)}{f'(x)} \right|$ dunque $|e_{k+1}| \leq M|e_k|^2$. Se e_0 è piccolo e M è limitato allora il metodo di Newton **converge quadraticamente**. Per avere M limitato devo essere lontano da un punto di estremo (in modo da avere $|f'(x)|$ abbastanza grande). Un altro ostacolo al metodo di Newton è che richiede il calcolo di $f'(x_k)$. Si applica dunque il **metodo delle secanti**, per cui si approssima $f'(x_k)$ con rapporto incrementale, utilizzando x_{k-1} e x_k :

$$f'(x_k) \simeq \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}$$

Otengo così il metodo delle secanti:



```

1: dato  $x_0$  e  $x_1$ 
2: for  $k = 1, 2, \dots$  do
3:    $x_{k+1} = x_k - \frac{f(x_k)}{\frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}}$ 
4: end for

```

Osservazione 6.6. Per evitare che il metodo di Newton (o delle secanti) non converga, si usa un metodo ibrido:

- Uso bisezione quando le iterazioni di Newton si allontanano dalla soluzione.
- Tengo il valore calcolato da Newton quando è “affidabile” (iterazione di Newton resta in (a, b))

Praticamente se un iterazione di Newton fallisce uscendo da (a, b) allora applico la bisezione.

In Matlab $x = \text{fzero}(f, x_0)$ con input f e x_0 stima iniziale. Questo comando utilizza l’approccio ibrido:

- Metodo lento: bisezione
- Metodo veloce: interpolazione quadratica inversa (interpola con una parabola che ha asse parallelo all’asse delle ascisse $x = ay^2 + by + c$ invece che con una retta secante). Scelgo $y = x_{k+1}$ tale che $ay^2 + by + c = 0$. Trovo a, b, c facendo passare la parabola per i punti:

$$(x_k, f(x_k)) \quad (x_{k-1}, f(x_{k-1})) \quad (x_{k-2}, f(x_{k-2}))$$

6.4 Metodo di punto fisso

Sia $g : [a, b] \rightarrow \mathbb{R}$ continua e supponiamo $\exists y \in [a, b]$ tale che $g(y) = y$. Il valore y è detto **punto fisso** per la funzione g . Dato un valore iniziale $x_0 \in [a, b]$, costruiamo la successione definita per ricorrenza:

$$x_{k+1} = g(x_k), \quad k = 0, 1, 2, \dots$$

Se la successione $\{x_k\}_{k=0}^{\infty}$ converge a un limite y , e se g è continua in y , allora tale limite è un punto fisso. Infatti:

$$y = \lim_{k \rightarrow \infty} x_{k+1} = \lim_{k \rightarrow \infty} g(x_k) = g\left(\lim_{k \rightarrow \infty} x_k\right) = g(y)$$

L'ultima uguaglianza segue dalla continuità di g .

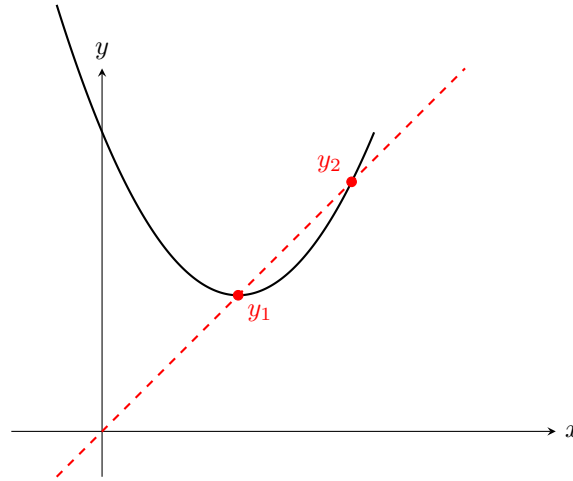


Figura 5: Illustrazione grafica di punti fissi come intersezioni tra $y = g(x)$ e $y = x$.

Quindi se la successione x_k converge, allora converge necessariamente a un punto fisso di g . Vediamo ora sotto quali condizioni la successione x_k converge. Supponiamo che g abbia un punto fisso $y \in [a, b]$, cioè $y = g(y)$. Valutiamo l'errore al passo k :

$$e_k = |y - x_k|$$

Se x_{k-1} è anch'esso in $[a, b]$, possiamo scrivere:

$$e_k = |y - x_k| = |g(y) - g(x_{k-1})|$$

Se g è derivabile nell'intervallo di estremi y e x_{k-1} , per il teorema di Lagrange esiste ξ_k compreso tra y e x_{k-1} tale che:

$$g(y) - g(x_{k-1}) = g'(\xi_k)(y - x_{k-1})$$

Prendendo il valore assoluto:

$$|g(y) - g(x_{k-1})| = |g'(\xi_k)| |y - x_{k-1}|$$

Quindi:

$$e_k = |g'(\xi_k)| e_{k-1}$$

Se supponiamo che esista una costante M tale che $|g'(x)| \leq M$ per tutti gli x di interesse (ad esempio in $[a, b]$), allora:

$$e_k \leq M e_{k-1}$$

Applicando ricorsivamente questa disuguaglianza:

$$e_k \leq M e_{k-1} \leq M^2 e_{k-2} \leq \dots \leq M^k e_0 = M^k |y - x_0|$$

Se $x_0, y \in [a, b]$, allora $|y - x_0| \leq (b - a)$, quindi $e_k \leq M^k (b - a)$. Affinché l'errore tenda a zero per $k \rightarrow \infty$, cioè $x_k \rightarrow y$, è sufficiente che $M < 1$. Quindi, se esiste una costante $M < 1$ tale che $|g'(x)| \leq M$ nell'intervallo $[a, b]$, il metodo converge.

Teorema 6.1 (Convergenza globale nell'intervallo). *Sia $g : [a, b] \rightarrow [a, b]$ una funzione tale che:*

1. g è continua in $[a, b]$.
2. g è derivabile in (a, b) .
3. Esiste una costante $M < 1$ tale che $|g'(x)| \leq M$ per ogni $x \in (a, b)$.

Allora esiste un unico punto fisso $y \in [a, b]$ e la successione $x_{k+1} = g(x_k)$ converge a y per ogni scelta del punto iniziale $x_0 \in [a, b]$.

Dimostrazione (Cenno). L'ipotesi $g : [a, b] \rightarrow [a, b]$ assicura che tutti gli iterati x_k rimangano nell'intervallo $[a, b]$. La condizione $|g'(x)| \leq M < 1$ (detta condizione di contrazione Lipschitziana sulla derivata) implica, tramite il Teorema di Lagrange come visto sopra, che $|g(u) - g(v)| \leq M|u - v|$ per ogni $u, v \in [a, b]$. Questo rende g una contrazione. Il Teorema delle Contrazioni (o di Banach-Caccioppoli) in uno spazio metrico completo (come $[a, b]$) garantisce l'esistenza e l'unicità del punto fisso y e la convergenza dell'iterazione $x_{k+1} = g(x_k)$ a y per ogni $x_0 \in [a, b]$. L'unicità si dimostra anche per assurdo come mostrato più avanti. La stima dell'errore è $|y - x_k| \leq M^k|y - x_0|$, che tende a 0 poiché $M < 1$. $\square$

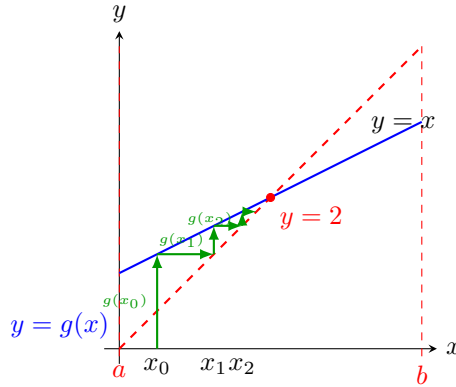


Figura 6: Illustrazione grafica dell'iterazione di punto fisso (metodo della ragnatela o a scala) per $g(x) = 0.5x + 1$ con $x_0 = 0.5$. La successione $x_0, x_1, x_2, \dots$ converge al punto fisso $y = 2$.

Spesso non è facile verificare le ipotesi del teorema precedente (in particolare $g : [a, b] \rightarrow [a, b]$) su un intero intervallo $[a, b]$. Risultati locali, validi in un intorno del punto fisso, sono più comuni.

Teorema 6.2 (Risultato locale (1) - Raffinato). *Sia y un punto fisso per g , cioè $g(y) = y$. Supponiamo che g sia derivabile in un intorno $I_\delta(y) = (y - \delta, y + \delta)$ e che esista $M < 1$ tale che $|g'(x)| \leq M < 1$ per ogni $x \in I_\delta(y)$. Se esiste un sottointervallo $J \subseteq I_\delta(y)$ tale che $y \in J$ e $g(J) \subseteq J$, allora l'iterazione $x_{k+1} = g(x_k)$ converge a y per ogni $x_0 \in J$.*

Osservazione 6.7. La difficoltà pratica di questo teorema è verificare l'esistenza dell'intervallo J che sia mappato in sé stesso. Il prossimo teorema supera questo problema usando la continuità della derivata.

Teorema 6.3 (Risultato locale (2) - Standard). *Sia y un punto fisso per g , cioè $g(y) = y$. Supponiamo che g sia derivabile con derivata g' continua in un intorno $I_\delta(y) = (y - \delta, y + \delta)$ del punto fisso y . Se $|g'(y)| < 1$, allora esiste un intorno $I_\lambda(y) = (y - \lambda, y + \lambda)$ con $0 < \lambda \leq \delta$ tale che l'iterazione:*

$$x_{k+1} = g(x_k) \quad \text{converge a } y \quad \forall x_0 \in I_\lambda(y).$$

Dimostrazione (Idea). Poiché g' è continua in y e $|g'(y)| = c < 1$, per la permanenza del segno (applicata a $|g'(x)| - M$), possiamo trovare un numero M tale che $c < M < 1$ e un valore $\lambda > 0$ (con

$\lambda \leq \delta$) tali che per ogni $x \in I_\lambda(y) = [y - \lambda, y + \lambda]$ si abbia $|g'(x)| \leq M$. Ora, per ogni $x \in I_\lambda(y)$, usando il Teorema di Lagrange:

$$|g(x) - y| = |g(x) - g(y)| = |g'(\xi)||x - y|$$

dove ξ è compreso tra x e y , e quindi $\xi \in I_\lambda(y)$. Ne segue che $|g'(\xi)| \leq M$.

$$|g(x) - y| \leq M|x - y|$$

Se $x \in I_\lambda(y)$, allora $|x - y| \leq \lambda$. Quindi $|g(x) - y| \leq M\lambda$. Poiché $M < 1$, abbiamo $M\lambda < \lambda$. Questo significa che $g(x)$ è ancora nell'intervallo $I_\lambda(y)$. Dunque, g mappa l'intervallo chiuso $I_\lambda(y)$ in sé stesso ($g : I_\lambda(y) \rightarrow I_\lambda(y)$) e soddisfa $|g'(x)| \leq M < 1$ in tale intervallo. Per il Teorema 6.1 (applicato all'intervallo $I_\lambda(y)$), l'iterazione converge a y per ogni $x_0 \in I_\lambda(y)$. $\square$

Osservazione 6.8. *Nei teoremi locali 6.3 abbiamo supposto l'esistenza del punto fisso y .*

Il seguente teorema, noto come Teorema delle Contrazioni o di Banach-Caccioppoli (in $\mathbb{R}$ con la condizione sulla derivata), garantisce sia l'esistenza che l'unicità, oltre alla convergenza.

Teorema 6.4 (Teorema delle Contrazioni (via derivata)). *Sia $g : [a, b] \rightarrow [a, b]$ una funzione tale che:*

1. *g è continua in $[a, b]$.*
2. *g è derivabile in (a, b) .*
3. *Esiste una costante $M < 1$ tale che $|g'(x)| \leq M$ per ogni $x \in (a, b)$.*

Allora esiste ed è unico il punto fisso $y \in [a, b]$ tale che $g(y) = y$, e l'iterazione $x_{k+1} = g(x_k)$ converge a y per ogni $x_0 \in [a, b]$.

Osservazione 6.9. *Questo è essenzialmente lo stesso enunciato del Teorema 6.1, ma qui enfatizziamo che le ipotesi garantiscono automaticamente l'esistenza e l'unicità del punto fisso, non solo la convergenza assumendone l'esistenza. Nota: La condizione 3 implica che g è una contrazione: $|g(u) - g(v)| \leq M|u - v|$ per $u, v \in [a, b]$. Il teorema vale più in generale per funzioni che sono contrazioni, anche se non sono derivabili.*

Dimostrazione dell'unicità. Supponiamo per assurdo che esistano $y, z \in [a, b]$, con $y \neq z$, entrambi punti fissi di g . Allora $g(y) = y$ e $g(z) = z$. Per il teorema di Lagrange, esiste ξ compreso tra y e z (e quindi $\xi \in (a, b)$) tale che:

$$g(y) - g(z) = g'(\xi)(y - z)$$

Prendendo i moduli e usando le proprietà del punto fisso:

$$|y - z| = |g(y) - g(z)| = |g'(\xi)||y - z|$$

Poiché $\xi \in (a, b)$, abbiamo $|g'(\xi)| \leq M < 1$. Quindi:

$$|y - z| \leq M|y - z|$$

Dato che $y \neq z$, abbiamo $|y - z| > 0$. Possiamo dividere per $|y - z|$, ottenendo $1 \leq M$. Ma questo contraddice l'ipotesi $M < 1$. Dunque, l'assunzione che esistano due punti fissi distinti è falsa. Deve essere $y = z$.

Applicazione alla ricerca di zeri

Ricordiamo che il nostro obiettivo originale è spesso trovare una radice (o zero) di una funzione f , cioè un valore y tale che $f(y) = 0$. Possiamo trasformare questo problema in un problema di punto fisso. Un modo semplice è definire:

$$g(x) = f(x) + x$$

Se $f(y) = 0$, allora $g(y) = f(y) + y = 0 + y = y$, quindi y è un punto fisso per g . Tuttavia, questa non è l'unica scelta, e potrebbe non portare a un metodo convergente (cioè, potrebbe non soddisfare $|g'(y)| < 1$).

Esistono infinite funzioni g tali che $f(y) = 0 \iff g(y) = y$. Una famiglia comune è data da:

$$g(x) = x + \alpha(x)f(x)$$

dove $\alpha(x)$ è una funzione (possibilmente costante, $\alpha(x) = \lambda$) non nulla vicino alla radice.

Metodo di Newton come metodo di punto fisso. Scegliendo $\alpha(x) = -1/f'(x)$, otteniamo:

$$g(x) = x - \frac{f(x)}{f'(x)}$$

L'iterazione di punto fisso $x_{k+1} = g(x_k)$ diventa:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

che è esattamente il **metodo di Newton**. Vediamo la condizione di convergenza locale. Calcoliamo $g'(x)$:

$$\begin{aligned} g'(x) &= \frac{d}{dx} \left(x - \frac{f(x)}{f'(x)} \right) \\ &= 1 - \frac{f'(x) \cdot f'(x) - f(x) \cdot f''(x)}{[f'(x)]^2} \\ &= 1 - \left(1 - \frac{f(x)f''(x)}{[f'(x)]^2} \right) \\ &= \frac{f(x)f''(x)}{[f'(x)]^2} \end{aligned}$$

Ora valutiamo $g'(y)$ dove y è una radice di f , cioè $f(y) = 0$. Supponiamo che sia una radice semplice, quindi $f'(y) \neq 0$.

$$g'(y) = \frac{f(y)f''(y)}{[f'(y)]^2} = \frac{0 \cdot f''(y)}{[f'(y)]^2} = 0$$

Poiché $g'(y) = 0$, abbiamo $|g'(y)| < 1$. Quindi, per il Teorema 6.3, il metodo di Newton converge localmente, purché f sia sufficientemente regolare (ad esempio $f \in C^2$ in un intorno di y affinché g' sia continua) e x_0 sia scelto abbastanza vicino a y . La condizione $g'(y) = 0$ implica una convergenza particolarmente veloce (quadratica).

Metodi di punto fisso del tipo $g(x) = x + \lambda f(x)$. Consideriamo la famiglia più semplice $g(x) = x + \lambda f(x)$ con $\lambda \in \mathbb{R}$, $\lambda \neq 0$. Affinché il metodo $x_{k+1} = g(x_k)$ converga localmente a una radice y di f , dobbiamo avere $|g'(y)| < 1$. Calcoliamo la derivata:

$$g'(x) = 1 + \lambda f'(x)$$

La condizione di convergenza locale è quindi:

$$|1 + \lambda f'(y)| < 1$$

Questa disuguaglianza equivale a:

$$-1 < 1 + \lambda f'(y) < 1$$

Sottraendo 1 da tutti i membri:

$$-2 < \lambda f'(y) < 0$$

Questa condizione ci dice due cose su λ (supponendo $f'(y) \neq 0$):

1. $\lambda f'(y)$ deve essere negativo. Quindi, λ deve avere segno opposto a quello di $f'(y)$.

2. La grandezza $\lambda f'(y)$ deve essere compresa tra -2 e 0. Questo significa $|\lambda f'(y)| < 2$, ovvero $|\lambda| |f'(y)| < 2$.

Quindi, possiamo scegliere λ tale che:

$$\text{segno}(\lambda) = -\text{segno}(f'(y)) \quad \text{e} \quad 0 < |\lambda| < \frac{2}{|f'(y)|}$$

Se scegliamo un λ che soddisfa queste condizioni, il metodo iterativo $x_{k+1} = x_k + \lambda f(x_k)$ converge a y se x_0 è sufficientemente vicino a y e se f è sufficientemente regolare (es. $f \in C^1$ con f' continua vicino a y).

Metodi per equazioni non lineari scalari

- Metodo di bisezione
- Metodo di Newton
- Metodo delle secanti
- Metodo del punto fisso

Gli ultimi 3 possono essere generalizzati al caso di sistemi. Il metodo di bisezione non è generalizzabile perché basato su disuguaglianze:

$$f(a) \cdot f(c) < 0$$

Nel caso di sistemi sapere che:

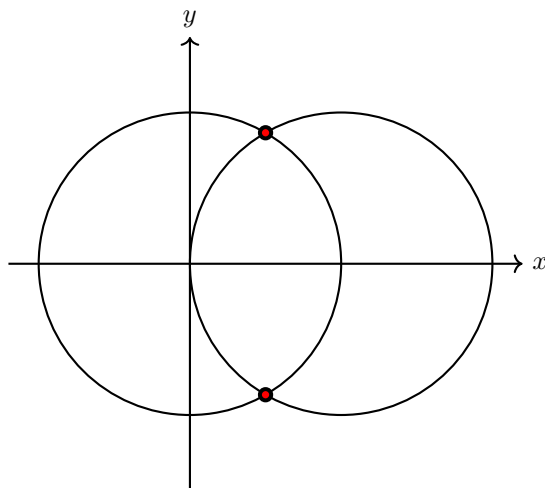
$$\begin{bmatrix} \dots & f(a) & \dots \end{bmatrix}^T \cdot \begin{bmatrix} \vdots \\ f(c) \\ \vdots \end{bmatrix} < 0$$

non dice nulla sull'esistenza di uno zero all'interno di un intervallo $[a] \ [c]$

Per i sistemi cercare $\underline{x}$ tale che $f(\underline{x}) = 0$ significa che tutte le componenti di f devono essere uguali a zero nello stesso punto. Parleremo poi di implementazione di metodi numerici per ODE.

Esempio 6.2. Trovare i punti di intersezione tra le circonferenze:

$$\begin{cases} x^2 + y^2 - 1 = 0 \\ (x-1)^2 + y^2 - 1 = 0 \end{cases}$$



In questo caso stiamo cercando il punto $\underline{x} = \begin{bmatrix} x \\ y \end{bmatrix}$ tale che:

$$f(\underline{x}) = \begin{bmatrix} x^2 + y^2 - 1 \\ (x-1)^2 + y^2 - 1 \end{bmatrix} = 0$$

Metodo di Newton uso Taylor fermandomi al primo ordine.

$$f(y) = f(x_0) + Jf \Big|_{x_0} (x - x_0) \dots$$

Dato x_0 calcolo, per $k = 0, 1, \dots, x^{k+1}$ tale che:

$$f(x^k) = f(x^k) + Jf \Big|_{x^k} (x^{k+1} - x^k) = 0$$

dove $f(x^k)$ è un vettore in $\mathbb{R}^n$ noto perchè conosco x^k , mentre $Jf \Big|_x$ è una matrice in $\mathbb{R}^{n \times n}$ nota, sempre perchè conosciamo x^k . Definisco un sistema lineare $n \times n$ del tipo:

$$Jf \Big|_{x^k} \delta^k = -f(x^k) \quad \delta^k \in \mathbb{R}^n$$

Esempio 6.3.

$$f(\underline{x}) = \begin{bmatrix} x^2 + y^2 - 1 \\ (x-1)^2 + y^2 - 1 \end{bmatrix}$$

Calcolo la matrice Jacobiana:

$$(Jf)_{i,j} = \frac{\partial f_i}{\partial x_j} = \begin{bmatrix} 2x & 2y \\ 2x-2 & 2y \end{bmatrix}$$

Supponiamo di partire da $x^0 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$:

$$Jf \Big|_{x^0} = \begin{bmatrix} 2 & 2 \\ 0 & 2 \end{bmatrix} \quad f(x^0) = f\left(\begin{bmatrix} 1 \\ 1 \end{bmatrix}\right) = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Per trovare x^1 devo risolvere

$$Jf \Big|_{x^0} \cdot \delta = -f(x^0) \Rightarrow \begin{bmatrix} 2 & 2 \\ 0 & 2 \end{bmatrix} \delta^0 = -\begin{bmatrix} 1 \\ 0 \end{bmatrix} \Rightarrow \delta^0 = \begin{bmatrix} -\frac{1}{2} & 0 \end{bmatrix}$$

Quindi $x^1 = x^0 + \delta^1 = \begin{bmatrix} \frac{1}{2} \\ 1 \end{bmatrix}$. All'iterazione successiva ricalcolo $f(x^k)$, $Jf \Big|_{x^k}$ e risolvo il nuovo sistema fino a soddisfare il test di arresto sul residuo $\|f(x^k)\| \leq c_1 \cdot \text{tol}_1$. Sull'iterazione successiva $\|x^{k+1} - x^k\| \leq c_2 \cdot \text{tol}_2$.

Teorema 6.5 (Convergenza locale per Newton). *Sia $f \in C^1$ in $B(x_0, \delta)$ (palla di centro x_0 e raggio δ). Necessitiamo inoltre che $(Jf)^{-1}$ esista in $B(x_0, \delta)$ (dunque Jf è invertibile) e che Jf sia limitata, ovvero esista una costante $C > 0$ tale che:*

$$\|(Jf(x))^{-1}\| \leq C \quad \forall x \in B(x_0, \delta)$$

Jf deve essere Lipschitziana, ovvero esista una costante $L > 0$ tale che:

$$\|Jf(y) - Jf(z)\| \leq L\|y - z\|$$

Allora esiste una palla $B(x_0, r)$ $r < \delta$ tale che il metodo di Newton converge per ogni dato iniziale $y^0 \in B(x_0, r)$.

Riprendendo l'esempio precedente:

$$\det J = 4xy - (2x-2)2y = 4y \neq 0 \text{ per } y \neq 0$$

Si verifica facilmente che se il punto di partenza è lontano da $y = 0$, le ipotesi del teorema sinì verificate e Newton converge a una delle due soluzioni a seconda della scelta iniziale (punto di partenza).

PseudoNewton A ogni iterazione il metodo di Newton richiede il calcolo della matrice Jacobiana, il che può essere molto costoso. Posso dunque tenere Jf fisso per un certo numero di iterazioni p . Dunque parto da x^0 e calcolo Jf in x^0 , che chiamiamo $\tilde{J}f$.

```

1: procedure PSEUDONEWTON( $f, x^0, p$ )
2:    $x^0 \leftarrow x^0$ 
3:    $Jf \leftarrow \tilde{J}f$ 
4:   for  $k = 0, 1, \dots$  do
5:     Risolvo  $\tilde{J}f \delta^k = -f(x^k)$ 
6:      $x^{k+1} \leftarrow x^k + \delta^k$ 
7:   end for
8: end procedure

```

Aggiorno $\tilde{J}f = Jf|_{x^{k+1}}$ e ripeto fino a convergenza.

Metodo di punto fisso Trasformo il problema di trovare y tale che $f(y) = 0$ in trovare y tale che $g(y) = y$. Uso dunque l'iterazione di punto fisso: dato x^0 , per $k = 0, 1, \dots$, calcolo x^{k+1} tale che:

$$x^{k+1} = x^k + g(x^k)$$

Questo metodo converge per ogni $x^0 \in B(y, r)$ se $\|g'(y)\| < 1$ in un intorno $B(y, r)$.

$$\|x^{k+1} - y\| \rightarrow 0$$

Finora abbiamo risolto sistemi $Ax = b$ con $A \in \mathbb{R}^{n \times n}$ invertibile, $b \in \mathbb{R}^n$, per cui cerchiamo $x \in \mathbb{R}^n$ che lo risolve. Abbiamo utilizzato:

- Fattorizzazione LU (metodo di Gauss), per cui $A = LU$ con L triangolare inferiore e U triangolare superiore, e quindi si risolvono due sistemi triangolari.
- Varianti di LU (e.g. pivoting)

Tipicamente abbiamo molte più equazioni che incognite (più dati che parametri da determinare). In questo caso si parla di **sistemi sovradeterminati**.

6.5 Sistemi sovradeterminati

Risolviamo dunque $Ax = b$ con $A \in \mathbb{R}^{m \times n}$ ($m > n$), $b \in \mathbb{R}^m$ e $x \in \mathbb{R}^n$.

$$\begin{bmatrix} A \end{bmatrix} \begin{bmatrix} x \end{bmatrix} = \begin{bmatrix} b \end{bmatrix}$$

In queste condizioni, generalmente, non esiste una soluzione esatta x tale che $Ax - b = 0$. Combinando la nozione di “soluzione” con la nozione di “indebolimento”, ovvero cercare x tale che:

$$\|Ax - b\|$$

sia più piccolo possibile.

Osservazione 6.10. Questa misura dipende naturalmente dalla norma scelta. Di solito usiamo la norma 2, in quanto è una rappresentazione intuitiva delle distanze ed è differenziabile.

Posto $r = Ax - b$, cerchiamo x che rende minima $\|r\|_2$. Questo problema è equivalente a minimizzare:

$$\|r\|_2^2 = \sum_{i=1}^m (Ax - b)_i^2 \quad (\text{problema ai minimi quadrati})$$

Chiamiamo $\varrho(x) = \|r\|_2^2 = \sum_{i=1}^m (Ax - b)_i^2$. Dunque $\varrho : \mathbb{R}^n \rightarrow \mathbb{R}$ è convessa. Per minimizzarla cerco i punti critici:

$$\nabla \varrho(x) = 0$$

Notiamo che $\varrho(x) = \langle Ax - b, Ax - b \rangle$.

$$\begin{aligned} \varphi(x) &= \langle Ax - b, Ax - b \rangle = \\ &= \langle Ax, Ax - b \rangle - \langle b, Ax - b \rangle = \\ &= \langle Ax, Ax \rangle - \langle Ax, b \rangle - \langle b, Ax \rangle + \langle b, b \rangle = \\ &= \langle x, A^T Ax \rangle - 2\langle x, A^T b \rangle + \langle b, b \rangle \end{aligned}$$

Dunque:

$$\nabla \varphi(x) = 2A^T Ax - 2A^T b = 0 \implies A^T Ax = A^T b$$

Dunque per minimizzare $\varrho(x)$ devo trovare x soluzione di $A^T Ax = A^T b$. Questo sistema è detto **sistema delle equazioni normali**.

$$\begin{bmatrix} & & & A^T & & \end{bmatrix} \begin{bmatrix} A \\ \end{bmatrix} \begin{bmatrix} x \\ \end{bmatrix} = \begin{bmatrix} & & & A^T & & \end{bmatrix} \begin{bmatrix} b \\ \end{bmatrix}$$

con $A^T \in \mathbb{R}^{n \times m}$. Definiamo dunque

$$\begin{bmatrix} A^T A \\ \end{bmatrix} \begin{bmatrix} x \\ \end{bmatrix} = \begin{bmatrix} \tilde{b} \\ \end{bmatrix}$$

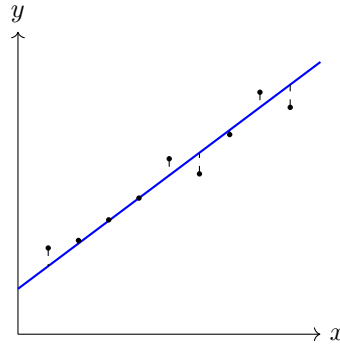
Quindi $\tilde{b} = A^T b \in \mathbb{R}^n$. Ho dunque un sistema $n \times n$ se $\text{rank}(A) = n$ (tutte le colonne di A sono linearmente indipendenti).

$$\implies \text{rank}(A^T A) = n \implies \det(A^T A) \neq 0$$

Risolvero il sistema delle equazioni normali (che non è più sovradeterminato) e ottengo un'unica soluzione x che coincide con la soluzione del problema dei minimi quadrati.

Esempio 6.4 (Retta di regressione lineare). Supponiamo di avere m dati (x_i, y_i) con $i = 1, \dots, m$ e vogliamo trovare la retta $y = px + q$ tale che

$$px_i + q = y_i$$



Quelle tratteggiate sono le distanze che noi siamo interessati a minimizzare. Cerchiamo dunque la retta che minimizza:

$$px_i + q - y_i \quad \forall i = 1, \dots, m$$

ovvero voglio $\sum_i (px_i + q - y_i)^2$ minimo. Vogliamo dunque risolvere:

$$A \begin{bmatrix} p \\ q \end{bmatrix} - b = 0 \quad \text{nel senso dei minimi quadrati}$$

con

$$A = \begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_m & 1 \end{bmatrix} \quad b = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} \quad x = \begin{bmatrix} p \\ q \end{bmatrix}$$

Il sistema sarebbe chiaramente sovradeterminato ($A \in \mathbb{R}^{m \times 2}$, $x \in \mathbb{R}^2$, $b \in \mathbb{R}^m$). Ricaviamo dunque il sistema delle equazioni normali:

$$A^T A x = \begin{bmatrix} x_1 & x_2 & \cdots & x_m \\ 1 & 1 & \cdots & 1 \end{bmatrix} \begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_m & 1 \end{bmatrix} \begin{bmatrix} p \\ q \end{bmatrix} = \begin{bmatrix} x_1 & x_2 & \cdots & x_m \\ 1 & 1 & \cdots & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}$$

$$\begin{bmatrix} \sum_{i=1}^m x_i^2 & \sum_{i=1}^m x_i \\ \sum_{i=1}^m x_i & m \end{bmatrix} \begin{bmatrix} p \\ q \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^m x_i y_i \\ \sum_{i=1}^m y_i \end{bmatrix}$$

Quella a sinistra è invertibile purché gli x_i non siano tutti uguali. Si intuisce imponendo che:

$$\begin{bmatrix} x_1 & x_2 & \cdots & x_m \\ 1 & 1 & \cdots & 1 \end{bmatrix} \begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_m & 1 \end{bmatrix}$$

abbia **rango massimo**. Risolvendo il sistema si trovano le stesse formule per p e q della regressione lineare.

6.6 Fattorizzazione QR

Sia $A \in \mathbb{R}^{m \times n}$. Allora esiste una matrice $Q \in \mathbb{R}^{m \times m}$ ortogonale e una matrice $R \in \mathbb{R}^{m \times n}$ triangolare superiore tali che:

$$A = QR$$

Graficamente

$$\begin{array}{c} \left[\begin{array}{c} \\ \\ \\ \end{array} \right] \\ A \\ m \times n \end{array} = \begin{array}{c} \left[\begin{array}{c} \\ \\ \\ \end{array} \right] \\ Q \\ m \times m \end{array} \times \begin{array}{c} \left[\begin{array}{c} \triangle \\ \\ \\ 0 \end{array} \right] \\ R \\ m \times n \end{array} \left. \begin{array}{l} \left. \vphantom{\begin{array}{c} \left[\begin{array}{c} \triangle \\ \\ \\ 0 \end{array} \right]} \right\} n \\ \left. \vphantom{\begin{array}{c} \left[\begin{array}{c} \triangle \\ \\ \\ 0 \end{array} \right]} \right\} m - n \end{array} \right\}
 \end{array}$$

Osserviamo che le “ultime” colonne sono inutili (moltiplicano per degli 0). Possiamo partizionare Q come:

$$\begin{array}{c} \overbrace{\hspace{1.5cm}}^n \quad \overbrace{\hspace{1.5cm}}^{m-n} \\ \left[\begin{array}{c|c} & \\ \hline Q_1 & Q_2 \\ \hline & \end{array} \right] \\ m \times m \end{array}$$

$$\begin{array}{c} \left[\begin{array}{c} \triangle \\ \hline \\ \\ \end{array} \right] \\ R_1 \\ \left. \vphantom{\begin{array}{c} \triangle \\ \hline \\ \\ \end{array}} \right\} n \\ \left. \vphantom{\begin{array}{c} \triangle \\ \hline \\ \\ \end{array}} \right\} m - n \\ \left[\begin{array}{c} \\ \\ R_2 \\ \\ \end{array} \right] \\ m \times n \end{array}$$

Le colonne di Q_2 sono ininfluenti (colonne **silenti**). In q moltiplichiamo elementi di R_2 (tutti gli elementi uguali a 0 per costruzione). Posso dunque riscrivere la fattorizzazione QR come:

$$A = Q_1 R_1$$

$$\begin{array}{ccc} \left[\begin{array}{c} \\ \\ \\ \end{array} \right] & \times & \left[\begin{array}{c} \triangle \\ \\ \\ \end{array} \right] \\ Q_1 & & R_1 \\ m \times n & & n \times n \end{array}$$

Dimostrazione. Nel caso $\text{rank}(A) = n$, $\exists Q, R$ tali che $A = QR$. Le colonne $\underline{a}_1, \underline{a}_2, \dots, \underline{a}_n$ sono linearmente indipendenti, quindi $\text{span}(\underline{a}_1, \underline{a}_2, \dots, \underline{a}_n) = \text{range}(A)$.

Posso usare come base di $\text{range}(A)$ dei vettori ortonormali $\underline{q}_1, \underline{q}_2, \dots, \underline{q}_n$ ossia $\exists c_1, c_2, \dots, c_n \in \mathbb{R}$ tali che:

$$\underline{a}_j = \sum_{k=1}^n c_{kj} \underline{q}_k \quad (\text{i coefficienti dipendono da quale colonna di } A \text{ sto considerando})$$

Per componenti:

$$(\underline{a}_j)_i = \sum_{k=1}^n c_{kj} (\underline{q}_k)_i = QC$$

dove $Q = \begin{bmatrix} | & | & & | \\ \underline{q}_1 & \underline{q}_2 & \dots & \underline{q}_n \\ | & | & & | \end{bmatrix}$. Si osservi che l'indice k si riferisce alla colonna di Q e i si riferisce alla riga di Q .

Le colonne di Q formano una base ortonormale di $\mathbb{R}^n$, mentre C è la matrice dei coefficienti del cambiamento di base da $\underline{a}_1, \underline{a}_2, \dots, \underline{a}_n$ a $\underline{q}_1, \underline{q}_2, \dots, \underline{q}_n$.

Per trovare C triangolare superiore proviamo a costruire un'opportuna base di $\mathbb{R}^2$.

Consideriamo $\underline{q}_1$ lungo $\underline{a}_1$:

$$\underline{q}_1 = \frac{\underline{a}_1}{\|\underline{a}_1\|}$$

Dopodiché consideriamo $\underline{q}_2$ ortogonale a $\underline{q}_1$, ma dipendente da $\underline{a}_2$:

$$\tilde{\underline{q}}_2 = \underline{a}_2 - \text{PROJ}_{\underline{a}_1} \underline{a}_2$$

ossia

$$\tilde{\underline{q}}_2 = \underline{a}_2 - \langle \underline{a}_2, \underline{q}_1 \rangle \underline{q}_1 \implies \underline{q}_2 = \frac{\tilde{\underline{q}}_2}{\|\tilde{\underline{q}}_2\|}$$

Si verifica che $\langle \underline{q}_1, \underline{q}_2 \rangle = 0$. Inoltre la matrice di cambiamento di base a $[\underline{q}_2 \quad \underline{q}_1]$ è triangolare superiore ossia:

$$\begin{bmatrix} \underline{a}_2 & \underline{a}_1 \end{bmatrix} = \begin{bmatrix} \underline{q}_2 & \underline{q}_1 \end{bmatrix} \begin{bmatrix} x & x \\ 0 & x \end{bmatrix}$$

dove x è un elemento diverso da 0. Veniamo ora al caso generale:

$A = [\underline{a}_1 \quad \underline{a}_2 \quad \dots \quad \underline{a}_n]$ con $\underline{a}_j \in \mathbb{R}^m$, $\text{rank}(A) = n$.

- 1: $\underline{q}_1 = \frac{\underline{a}_1}{\|\underline{a}_1\|}$
- 2: **for** $j = 2, \dots, n$ **do**
- 3: $\tilde{\underline{q}}_j = \underline{a}_j - \sum_{k=1}^{j-1} \langle \underline{a}_j, \underline{q}_k \rangle \underline{q}_k$
- 4: $\underline{q}_j = \frac{\tilde{\underline{q}}_j}{\|\tilde{\underline{q}}_j\|}$
- 5: **end for**

$\tilde{q}_j$ lo calcolo sottraendo alla j -esima colonna di A tutte le sue proiezioni sulle colonne ortogonali $\underline{q}_1, \underline{q}_2, \dots, \underline{q}_{j-1}$ già costruite. La matrice che raccoglie i coefficienti delle proiezioni $\langle \underline{a}_j, \underline{q}_k \rangle$ è triangolare superiore.

$$A = QR$$

□

Osservazione 6.11. Se A non ha rango pieno, per un certo $j = i$ otterrò $\tilde{q}_i = 0$. Impongo dunque $R_{ii} = 0$. Cerco $\underline{q}_i$ che sia diverso da 0 e ortogonale a tutti i vettori $\underline{q}_1, \underline{q}_2, \dots, \underline{q}_{i-1}$. Costruendolo come prima otteniamo

$$\text{rank}(A) = \#R_{ii} \neq 0$$

Osservazione 6.12. Se $m = n$ (A è quadrata) in $A = QR$ le matrici Q e R sono quadrate. Risolviamo

$$Ax = b \Leftrightarrow QRx = b \Leftrightarrow Rx = Q^T b \quad Q^T Q = \mathbb{I}$$

dato che Q è ortogonale. Trovo dunque x risolvendo il sistema triangolare superiore che ne risulta.

Osservazione 6.13. QR come metodo risolutivo di sistemi è del tutto equivalente (in aritmetica esatta) a LU , ma poiché R è triangolare superiore (per costruzione), QR è numericamente più stabile, ma più costoso a livello computazionale. Inoltre LU vale solo per matrici quadrate.

Per dimostrare l'esistenza della fattorizzazione QR , costruisco Q_1 e R_1 con la ortogonalizzazione di Gram-Schmidt.

Gram-Schmidt: parte dai vettori $\underline{a}_1, \underline{a}_2, \dots, \underline{a}_n$ colonne di A . Se $\text{rank}(A) = n$, allora $\underline{a}_1, \underline{a}_2, \dots, \underline{a}_n$ sono linearmente indipendenti

```

1:  $\underline{q}_1 = \frac{\underline{a}_1}{\|\underline{a}_1\|}$ 
2: for  $j = 2, \dots, n$  do
3:    $\tilde{q}_j = \underline{a}_j - \sum_{k=1}^{j-1} \langle \underline{a}_j, \underline{q}_k \rangle \underline{q}_k$  ( $\tilde{q}_j$  è ortogonale a  $\underline{q}_1, \underline{q}_2, \dots, \underline{q}_{j-1}$ )
4:    $\underline{q}_j = \frac{\tilde{q}_j}{\|\tilde{q}_j\|}$  (Normalizzazione)
5: end for

```

Osserviamo che $\tilde{q}_j = 0$ non è possibile se $\underline{a}_1, \underline{a}_2, \dots, \underline{a}_{j-1}$ sono linearmente indipendenti. In questo modo ho costruito Q_1 , la parte ridotta della matrice Q . Per Q_2 calcolo i vettori $\underline{q}_{n+1}, \underline{q}_{n+2}, \dots, \underline{q}_m$ ortonormali a $\underline{q}_1, \underline{q}_2, \dots, \underline{q}_n$.

R_1 è formata dai coefficienti $\langle \underline{a}_j, \underline{q}_k \rangle$. Infatti posso riscrivere

$$\underline{a}_j = \tilde{q}_j + \sum_{k=1}^{j-1} \langle \underline{a}_j, \underline{q}_k \rangle \underline{q}_k \quad \tilde{q}_j = \|\tilde{q}_j\| \underline{q}_j$$

Dunque la costruzione di R_1 e Q_1 diventa:

Algorithm 8 Fattorizzazione $A = Q_1 R_1$ con Gram-Schmidt

```

1: for  $j = 1, \dots, n$  do
2:    $\tilde{q}_j = \underline{a}_j$ 
3:   for  $k = 1, \dots, j-1$  do
4:      $r_{kj} = \langle \underline{a}_j, \underline{q}_k \rangle$ 
5:      $\tilde{q}_j = \tilde{q}_j - r_{kj} \underline{q}_k$ 
6:   end for
7:    $r_{jj} = \|\tilde{q}_j\|$ 
8:    $\underline{q}_j = \frac{\tilde{q}_j}{r_{jj}}$ 
9: end for

```

$$\begin{bmatrix} \text{---} & q_1^T & \text{---} \\ \text{---} & q_2^T & \text{---} \\ \vdots & \vdots & \vdots \\ \text{---} & q_n^T & \text{---} \end{bmatrix} \begin{bmatrix} q_1 & q_2 & \dots & q_n \end{bmatrix} = \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix}$$

Se $\text{rank}(A) < n$, allora ad un certo $j = i$ avrò $a_i \in \text{span}(\underline{a}_1, \underline{a}_2, \dots, \underline{a}_{i-1})$. In questo caso per $j = i$ trovo $\tilde{q}_i = 0$ e quindi $r_{ii} = 0$. Calcolo $\underline{q}_i$ come un qualsiasi vettore ortogonale a $\underline{q}_1, \underline{q}_2, \dots, \underline{q}_{i-1}$. Dunque l'algoritmo prosegue.

L'algoritmo di Gram-Schmidt in matematica floating point è instabile, nel senso che se le colonne $\underline{a}_1, \underline{a}_2, \dots, \underline{a}_n$ sono "quasi" linearmente dipendenti, le colonne di Q_1 sono solo approssimativamente ortogonali.

$$Q_1^T Q_1$$

Dato che, ad esempio

$$P_{11} = q_1^T q_1 = 1$$

$$P_{12} = q_1^T q_2 = 0$$

$$P_{21} = q_2^T q_1 = 0$$

$$P_{22} = q_2^T q_2 = 1$$

$$\vdots$$

In matematica esatta $P = \mathbb{I}$. In aritmetica floating point $P \approx \mathbb{I}$. Nella pratica si utilizzano le matrici di Householder.

Dato un vettore v si definisce

$$H = \mathbb{I} - 2 \frac{vv^T}{\|v\|^2} \quad \text{dove } v^T v \text{ è una matrice di rango 1}$$

Troviamo H^2 :

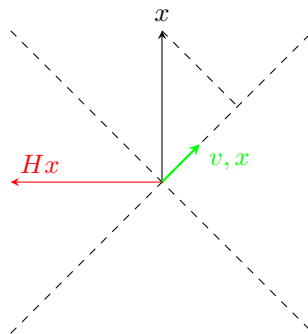
$$H^2 = \left(\mathbb{I} - 2 \frac{vv^T}{\|v\|^2} \right) \left(\mathbb{I} - 2 \frac{vv^T}{\|v\|^2} \right) = \mathbb{I} - 4 \frac{vv^T}{\|v\|^2} + 4 \frac{v(v^T v)v^T}{\|v\|^4}$$

Osserviamo che $v^T v = \|v\|^2$, quindi:

$$H^2 = \mathbb{I} - 4 \frac{vv^T}{\|v\|^2} + 4 \frac{vv^T}{\|v\|^2} = \mathbb{I}$$

Dunque H è invertibile e la sua inversa è $H^{-1} = H$. Troviamo inoltre che $H^T = H$. Dunque H è ortogonale. Se applichiamo H a un vettore x , otteniamo:

$$Hx = \left(\mathbb{I} - 2 \frac{vv^T}{\|v\|^2} \right) x = x - 2\langle w, x \rangle w \quad \text{dove } w = \frac{v}{\|v\|}$$



Scegliendo w accortamente riesco a ottenere che Hx punti in una direzione fissata. Si trasforma ciascuna colonna di A :

$$\begin{bmatrix} a_{11} \\ a_{21} \\ \vdots \\ a_{n1} \end{bmatrix} \xrightarrow{H} \begin{bmatrix} 0 \\ \vdots \\ 1 \\ 0 \end{bmatrix}$$

Voglio H tale che $Ha_1 = \alpha e_1$. In generale dato x voglio v tale che:

$$Hx = \underline{x} - \frac{2\langle v, x \rangle}{\|v\|^2} \underline{v} = \alpha \underline{e}_1$$

per cui $v \in \text{span}(\underline{x}, \underline{e}_1)$ ovvero $v = x + c\underline{e}_1$ e si trova c . Si ottiene $v = x \pm \|x\|_2 \underline{e}_1$.

Esempio 6.5. Sia $x = [3 \ 1 \ 5 \ 1]^T$. Trovare v tale che Hx sia parallelo a e_1 . Si ottiene

$$Hx = [-6 \ 0 \ 0 \ 0]^T$$

Data la matrice $A = \begin{bmatrix} x & x & x \\ x & x & x \\ x & x & x \\ x & x & x \end{bmatrix}$, determino $Q_1 = H_1$ tale che:

$$Q_1 A = \begin{bmatrix} \otimes & \otimes & \otimes \\ 0 & \otimes & \otimes \\ 0 & \otimes & \otimes \\ 0 & \otimes & \otimes \end{bmatrix}$$

Pongo Q_2

$$Q_2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & & \\ 0 & H_2 & \\ 0 & & \end{bmatrix}$$

dunque otterrò $Q_2 Q_1 A$.

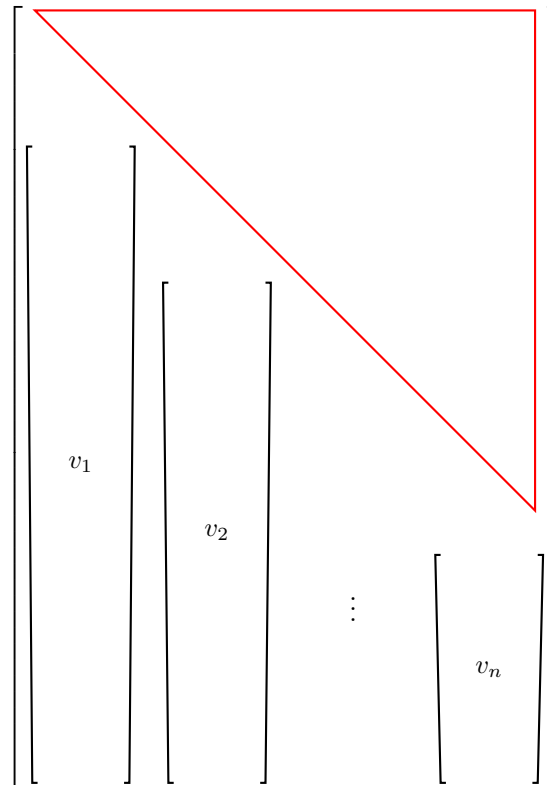
$$Q_2 Q_1 A = \begin{bmatrix} \otimes & \otimes & \otimes \\ 0 & \otimes & \otimes \\ 0 & 0 & \otimes \\ 0 & 0 & \otimes \end{bmatrix}$$

Costruisco QR applicando una successione di matrici di Householder sempre più piccole. Ottengo

$$\underbrace{Q_n Q_{n-1} \dots Q_1 A}_{\text{ortogonale}} = R$$

$$\begin{aligned} A &= (Q_n Q_{n-1} \dots Q_1)^T R \\ &= Q_1^T Q_2^T \dots Q_n^T R \quad H^T = H \\ &= Q_1 Q_2 \dots Q_n R \\ &= QR \end{aligned}$$

Trovo la fattorizzazione QR piena. Nella pratica una fattorizzazione QR produce una matrice con le stesse dimensioni di A



In Matlab, $[q,r]=qr(A)$. Per tornare al problema dei minimi quadrati:

$$r = \|Ax - b\|_2^2$$

calcolo dunque $A = QR$

$$r = \|QRx - b\|_2^2$$

ricordo che $\|Qy\|_2 = \|y\|_2 \forall y$. Quindi:

$$r = \|Q(Rx - Q^T b)\|_2^2 = \|Rx - Q^T b\|_2^2 \quad \text{ricordo } \|x\|^2 = \sum_i x_i^2$$

$$r = \|R_1 x - Q_1^T b\|_2^2 + \underbrace{\|R_2 x - Q_2^T b\|_2^2}_{\rightarrow 0} = \|R_1 x - Q_1^T b\|_2^2 + \|Q_2^T b\|_2^2$$

trovo x tale che $R_1 x = Q_1^T b$ (sistema $n \times n$ invertibile se $\text{rank}(A) = n \Rightarrow r_{ii} \neq 0 \forall i$). Il sistema è triangolare superiore. Mentre il residuo minimo è:

$$\min \|Ax - b\| = \|Q_2^T b\|$$

7 Autovalori e autovettori

In matlab possiamo invocare:

- $v=\text{eig}(A)$ per ottenere il vettore v degli autovalori
- $[v,r]=\text{eig}(A)$ per ottenere la matrice diagonale v diagonale degli autovalori e la matrice r degli autovettori

Definizione 7.1. Si dice che λ è un autovalore di A se esiste un vettore $\underline{x} \neq 0$ tale che

$$A\underline{x} = \lambda\underline{x} \iff (A - \lambda\mathbb{I})\underline{x} = 0$$

Ossia se $A - \lambda\mathbb{I}$ non è invertibile, quindi ha nel kernel un vettore non nullo.

Chiamiamo **polinomio caratteristico** di A il polinomio di grado n dato da:

$$p(\lambda) = \det(A - \lambda\mathbb{I})$$

e siamo interessati agli zeri di $p(\lambda)$.

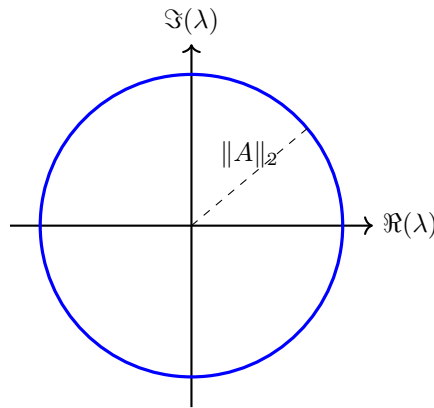
Teorema 7.1. *Il polinomio $p(\lambda)$ ha esattamente n radici complesse $\lambda_1, \lambda_2, \dots, \lambda_n$ che non sono necessariamente distinte.*

Se A è a coefficienti reali, $p(\lambda)$ può avere comunque coefficienti complessi. Se $\lambda \in \mathbb{C}$ è un autovalore di A ($\lambda \in \text{spec}(A)$), allora anche $\bar{\lambda}$ è un autovalore di A . Autovalori complessi di A si presentano in coppie coniugate.

7.1 Teoremi di localizzazione

Vogliamo stimare la posizione degli autovalori prima di calcolarli.

Teorema 7.2 (Teorema di Hirsch). *Se $\lambda \in \text{spec}(A)$, allora $|\lambda| \leq \|A\|_2$.*



Dimostrazione. Se $\lambda \in \text{spec}(A)$, con autovettore x , allora:

$$\begin{aligned} \|Ax\|_2^2 &= \langle Ax, Ax \rangle = \langle \lambda x, \lambda x \rangle = \\ &= \bar{\lambda}\lambda \langle x, x \rangle = |\lambda|^2 \|x\|_2^2 \end{aligned}$$

Dunque:

$$|\lambda| = \frac{\|Ax\|_2}{\|x\|_2} \leq \sup_{x \neq 0} \frac{\|Ax\|_2}{\|x\|_2} = \|A\|_2$$

□

Si può dimostrare che $|\lambda| \leq \|A\|$ per ogni norma matriciale.

Definizione 7.2. Chiamiamo C_i^R il **cerchio riga** relativo alla riga i , l'insieme:

$$\{z \in \mathbb{C} : |z - a_{ii}| \leq \sum_{j \neq i} |a_{ij}|\}$$

Teorema 7.3 (Teorema di Gershgorin). *Tutti gli autovalori di una matrice A sono contenuti nell'unione dei cerchi riga.*

$$\text{spec}(A) \subset \bigcup_{i=1}^n C_i^R$$

Dimostrazione. Sia $\lambda \in \text{spec}(A)$ con autovettore x . Dunque $Ax - \lambda x = 0 \iff \sum_{j=1}^n a_{ij}x_j - \lambda x_i = 0 \forall i = 1, \dots, n$.

$$a_{ii}x_i - \lambda x_i + \sum_{j \neq i} a_{ij}x_j = 0 \iff (a_{ii} - \lambda)x_i = - \sum_{j \neq i} a_{ij}x_j$$

Sia α l'indice tale che $|x_\alpha| = \max_i |x_i|$. Dunque $|x_\alpha| = \|x\|_\infty \neq 0$.

Dividiamo l'espressione precedente per x_α e consideriamo la riga $i = \alpha$:

$$(a_{\alpha\alpha} - \lambda) \frac{x_\alpha}{x_\alpha} = - \sum_{j \neq \alpha} \frac{a_{\alpha j}x_j}{x_\alpha}$$

Possiamo dunque applicare il modulo per ottenere la disuguaglianza triangolare:

$$|a_{\alpha\alpha} - \lambda| \leq \sum_{j \neq \alpha} |a_{\alpha j}| \underbrace{\left| \frac{x_j}{x_\alpha} \right|}_{<1} \leq \sum_{j \neq \alpha} |a_{\alpha j}|$$

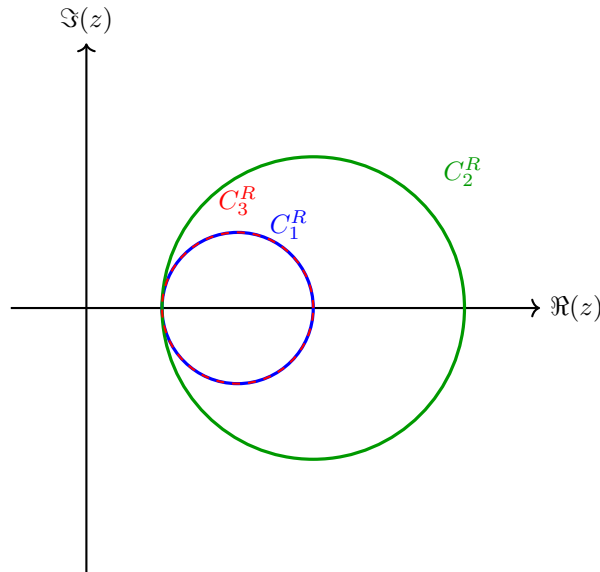
Ossia $\lambda \in C_\alpha^R \subset \bigcup_{i=1}^n C_i^R$. □

Esempio 7.1. Sia $A = \begin{bmatrix} 2 & -1 & 0 \\ 1 & 3 & -1 \\ 0 & 1 & 2 \end{bmatrix}$. Calcoliamo i cerchi riga:

$$C_1^R = \{|z - 2| \leq 1\}$$

$$C_2^R = \{|z - 3| \leq 2\}$$

$$C_3^R = \{|z - 2| \leq 1\}$$



Corollario 7.1. *Tutti gli autovalori di una matrice A sono contenuti nell'unione dei cerchi-colonna.*

$$\text{spec}(A) \subset \bigcup_{j=1}^n C_j^C$$

Infatti $\text{spec}(A^T) = \text{spec}(A)$, ossia una matrice e la sua trasposta hanno lo stesso spettro e ovviamente i cerchi-colonna di A sono i cerchi-riga di A^T .

Corollario 7.2.

$$\text{spec}(A) \subset \bigcup_{i=1}^n C_i^R \cap \bigcup_{j=1}^n C_j^C$$

Teorema 7.4. Sia $\mathcal{F}$ l'insieme di indici tale che $(\bigcup_{i \in \mathcal{F}} C_i^R) \cap (\bigcup_{j \notin \mathcal{F}} C_j^R) = \emptyset$ con $|\mathcal{F}| = k$. Allora la regione $\bigcup_{i \in \mathcal{F}} C_i^R$ contiene esattamente k autovalori di A .

Esempio 7.2. Sia $A = \begin{bmatrix} 2 & 1 & 0 \\ 1 & \frac{1}{2} & 1 \\ 0 & 1 & 5 \end{bmatrix}$. Calcoliamo i cerchi riga e colonna:

$$C_1^R = \{|z - 2| \leq 1\}$$

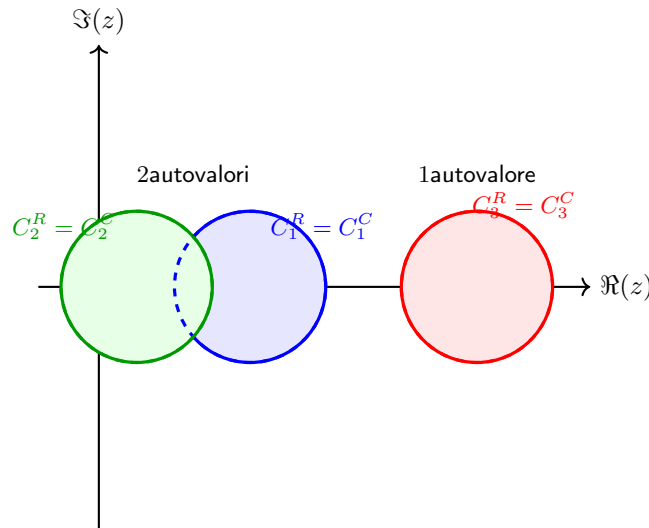
$$C_2^R = \{|z - \frac{1}{2}| \leq 1\}$$

$$C_3^R = \{|z - 5| \leq 1\}$$

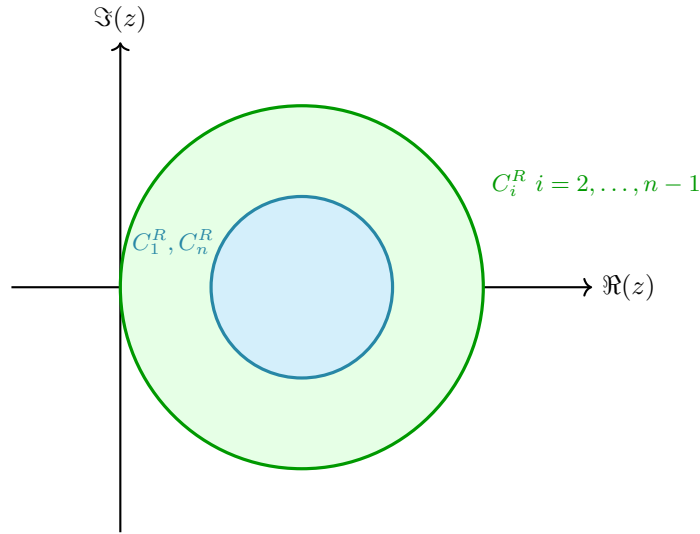
$$C_1^C = \{|z - 2| \leq 1\}$$

$$C_2^C = \{|z - \frac{1}{2}| \leq 1\}$$

$$C_3^C = \{|z - 5| \leq 1\}$$



Esempio 7.3. Sia $A = \begin{bmatrix} 2 & -1 & 0 & \dots & 0 \\ -1 & 2 & -1 & \dots & 0 \\ 0 & -1 & 2 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 2 \end{bmatrix}$. (la matrice del filo elastico). $C_1 = C_n = \{|z - 2| \leq 1\}$, mentre $C_i = \{|z - 2| \leq 2 \ \forall i = 2, n - 1\}$.



Teorema 7.5. *Un punto $z \in \mathbb{C}$ che appartiene alla frontiera dell'unione dei cerchi di Gershgorin ($z \in \partial \bigcup_{i=1}^n C_i^R$) può essere un autovalore solo se tutti i cerchi C_i^R che contengono z nella loro chiusura passano per z .*

Corollario 7.3. *$\lambda = 0$ non è un autovalore della matrice del filo elastico, che quindi non è singolare.*

7.2 Calcolo degli autovalori

Sia A una matrice $n \times n$ simmetrica. Allora $\exists \lambda_1, \lambda_2, \dots, \lambda_n \in \mathbb{R}$ autovalori reali con un sistema completo di autovettori ortogonali $w_1, w_2, \dots, w_n$.

Parto da $x^0 \in \mathbb{R}^n$, allora $\exists c_1, c_2, \dots, c_n : x^0 = \sum_{i=1}^n c_i w_i$. Proseguo dunque costruendo la successione che applica iterativamente A a x^k :

$$\begin{aligned} x^1 &= Ax^0 \\ x^2 &= Ax^1 \\ &\vdots \\ x^{k+1} &= Ax^k = A^{k+1}x^0 \end{aligned}$$

Ma

$$Ax^0 = A \sum_{i=1}^n c_i w_i = \sum_{i=1}^n c_i Aw_i = \sum_{i=1}^n c_i \lambda_i w_i$$

e dato che ciò vale per ogni k ottengo

$$x^k = \sum_{i=1}^n c_i \lambda_i^k w_i$$

supponiamo dunque che gli autovalori siano ordinati $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|$. Posso riscrivere:

$$x^k = c_1 \lambda_1^k w_1 + \sum_{i=2}^n c_i \lambda_i^k w_i = \lambda_1^k \left(c_1 w_1 + \sum_{i=2}^n c_i \underbrace{\left(\frac{\lambda_i}{\lambda_1} \right)^k}_{\rightarrow 0 \text{ per } k \rightarrow \infty} w_i \right)$$

per cui x^k tende a diventare parallelo a w_1 , e all'infinito si ha $x^k \parallel w_1$.

Osservazione 7.1. *L'algoritmo così scritto ha altissimo rischio di Overflow (se $|\lambda_1| > 1$) o Underflow (se $|\lambda_1| < 1$). È possibile mitigare il problema normalizzando opportunamente x^k .*

Algorithm 9 Metodo delle potenze

```

1:  $x^0 \leftarrow \frac{x^0}{\|x^0\|}$ 
2: for  $k = 0, 1, \dots$  do
3:    $x^{k+1} \leftarrow Ax^k$ 
4:    $\mu_k \leftarrow \langle x^{k+1}, x^k \rangle$ 
5:    $x^{k+1} \leftarrow \frac{x^{k+1}}{\|x^{k+1}\|}$ 
6: end for
7: return  $\mu_k$ 

```

Infatti

$$\langle x^{k+1}, x^k \rangle = \langle Ax^k, x^k \rangle \xrightarrow{k \rightarrow \infty} \langle Aw_1, w_1 \rangle = \lambda_1$$

ossia $\mu_k \rightarrow \lambda_1$ per $k \rightarrow \infty$ e abbiamo che l'algoritmo converge all'autovalore di modulo massimo.

Osservazione 7.2. Se $|\lambda_2| = |\lambda_1| > |\lambda_i| \forall i = 3, 4, \dots$ allora x^k converge ad un insieme $\subseteq \text{span}(w_1, w_2)$.

Non è dunque detto che l'algoritmo converga a un vettore per cui potrebbe oscillare, ma perde tutte le altre componenti, venendo proiettato su $\text{span}(w_1, w_2)$.

Introduciamo ora il metodo delle potenze inverse, per trovare l'autovalore di modulo minimo. Supponiamo dunque $|\lambda_1| \geq |\lambda_2| \geq \dots \geq |\lambda_{n-1}| > |\lambda_n|$. Per cui ($\lambda_n = \lambda_{\min}$)

Osservazione 7.3. $\frac{1}{\lambda_n}$ è l'autovalore di modulo massimo di A^{-1} . Applicando il metodo delle potenze a A^{-1} troviamo dunque l'autovalore di modulo minimo di A .

Algorithm 10 Metodo delle potenze inverse

```

1:  $x^0 \neq 0$  dato in input
2:  $x^0 \leftarrow \frac{x^0}{\|x^0\|}$ 
3: for  $k = 0, 1, \dots$  do
4:    $x^{k+1} \leftarrow A^{-1}x^k$ 
5:    $\mu_k \leftarrow \langle x^{k+1}, x^k \rangle$ 
6:    $x^{k+1} \leftarrow \frac{x^{k+1}}{\|x^{k+1}\|}$ 
7: end for
8: return  $\frac{1}{\mu_k}$ 

```

per il test di arresto possiamo, come al solito, usare la norma del residuo ($\|Ax^{k+1} - \mu_{k+1}x^{k+1}\| > c \cdot \text{Tol}$) o il test sulle iterazioni successive.

Osservazione 7.4. Mai calcolare A^{-1} esplicitamente, ma piuttosto si opta, sapendo $x^{k+1} = A^{-1}x^k$, per risolvere il sistema lineare $Ax^{k+1} = x^k$. Di seguito è riportato un algoritmo che implementa questo approccio.

Algorithm 11 Metodo delle potenze inverse

```

1:  $x^0 = \frac{x^0}{\|x^0\|}$ 
2:  $L, U = LU(A)$ 
3: for  $k = 0, 1, \dots$  do
4:    $LUx^{k+1} = x^k$ 
5:    $\mu^k \leftarrow \langle x^{k+1}, x^k \rangle$ 
6:    $x^{k+1} \leftarrow \frac{x^{k+1}}{\|x^{k+1}\|}$ 
7: end for
8: return  $\frac{1}{\mu_k}$ 

```

Metodo delle potenze

Ad ogni iterazione calcolo $x^{k+1} = Ax^k$. Il calcolo è rapido se A è sparsa e memorizzata in modo

sparso.

Metodo della potenza inversa

Ad ogni iterazione calcolo $Ax^{k+1} = x^k$ risolvo per x^{k+1} , è un sistema lineare e se A è sparsa, uso un metodo iterativo.

7.3 Calcolo di tutti gli autovalori

Condizionamento

Se invece di A calcolo gli autovalori di una matrice $A + E$ (con E matrice piccola, ovvero di norma piccola), siamo interessati a capire quanto cambiano gli autovalori.

Supponiamo che A sia diagonalizzabile, cioè $\exists X$ invertibile tale che:

$$X^{-1}AX = D \quad \text{diagonale} \quad D = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n) \quad \lambda_i \in \text{spec}(A)$$

e le colonne di X sono gli autovettori di A .

Teorema 7.6. *Sia $\mu \in \text{spec}(A + E)$ cioè un autovalore di $A + E$. Allora:*

$$\min_{\lambda \in \text{spec}(A)} |\mu - \lambda| \leq K_p(X) \|E\|_p \quad \forall \mu \in \text{spec}(A + E) \quad \forall p$$

dove $K_p(X)$ è il numero di condizionamento di X rispetto alla norma p

$$K_p(X) = \|X\|_p \|X^{-1}\|_p$$

In particolare se $K_p(X) \gg 1$ allora gli autovalori di $A + E$ possono essere molto lontani da quelli di A . Se $K_p(X) \sim 1$ allora gli autovalori di $A + E$ sono vicini a quelli di A , purché $\|E\|_p$ sia piccola.

Se A è simmetrica, il calcolo degli autovalori è sempre ben condizionato perché X è ortogonale e quindi $K_2(X) = 1$.

Se A non è simmetrica, X può avere condizionamento alto e anche se A non è diagonalizzabile può non essere possibile determinare X invertibile.

Possiamo dunque ricorrere a ottenere una matrice triangolare per il calcolo degli autovalori. Ovvero vedo se $\exists T \in \mathbb{C}$ triangolare tale che:

$$A = XTX^{-1}$$

Ricordo che se $\exists$ una matrice T invertibile tale che $A = XTX^{-1}$ allora A e T sono matrici simili e hanno gli stessi autovalori.

Fattorizzazione di Schur

Ogni matrice $A \in \mathbb{C}^{n \times n}$ può essere scritta come

$$A = UTU^{-1} \quad \text{con } U \text{ unitaria}$$

Matrici unitarie Una matrice $U \in \mathbb{C}^{n \times n}$ si dice unitaria se $U^*U = \mathbb{I}$ dove $U^* = \overline{U}^T$ è la matrice trasposta coniugata di U (complesso coniugato di U^T) e si dice complemento Hermetiano.

In matlab x' è $\bar{x}^T$ cioè $x' = x^*$. Osserviamo che se $U \in \mathbb{R}^{n \times n}$ allora $U^* = U^T$. Sia $z \in \mathbb{C}$:

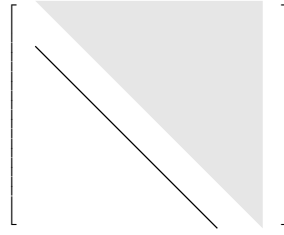
$$z = a + ib \quad a, b \in \mathbb{R} \quad \Rightarrow \quad \bar{z} = a - ib$$

Dove $a = \text{Re}(z) \in \mathbb{R}$ e $b = \text{Im}(z) \in \mathbb{R}$. Ricordiamo che un numero $z \in \mathbb{R}$ se $\text{Im}(z) = 0$ o $z = \bar{z}$.

Se volessimo lavorare nel campo reale, non possiamo ottenere la fattorizzazione di Schur, ma otteniamo:

$$A = U\tilde{T}U^{-1}$$

con U ortogonale e $\tilde{T}$ triangolare superiore con la sottodiagonale $t_{i,i-1} \neq 0$ per qualche i .



Se evidenziamo la sottodiagonale di $\tilde{T}$ abbiamo che questa sarà composta da qualche numero e qualche zero.

$$\begin{bmatrix} x & \cdots & \cdots & \cdots & \cdots \\ 0 & x & \cdots & \cdots & \cdots \\ & x & x & \cdots & \cdots \\ & & x & x & \cdots \\ & & & 0 & \ddots \\ & & & & \ddots \end{bmatrix}$$

Se $t_{i,i-1} = 0$ allora $t_{i-1,i-1} = \lambda \in \mathbb{R}$. Se invece $t_{i,i-1} \neq 0$

$$B_i = \begin{bmatrix} t_{i-1,i-1} & t_{i-1,i} \\ t_{i,i-1} & t_{i,i} \end{bmatrix}$$

è un blocchetto 2×2 sulla diagonale di $\tilde{T}$ e contiene 2 autovalori complessi coniugati λ e $\bar{\lambda}$ con

$$\text{Tr}(B_i) = \lambda + \bar{\lambda} = 2 \text{Re}(\lambda) \quad \det(B_i) = \lambda \bar{\lambda} = [\text{Re}(\lambda)]^2 + [\text{Im}(\lambda)]^2$$

Possiamo dunque calcolare **tutti** gli autovalori di una matrice.

Caso simmetrico ($\lambda \in \mathbb{R}$)

```

1:  $A^0 = A$ 
2: for  $k = 1, 2, \dots$  do
3:    $Q^k R^k = A^k$  ▷ ovvero la fattorizzazione QR di  $A^k$ 
4:    $A^{k+1} = R^k Q^k$ 
5:   TEST di arresto
6: end for

```

Noto che A^{k+1} ha gli stessi autovalori di A^k perchè è simile a A^k . Infatti:

$$A^{k+1} = R^k Q^k = (Q^k)^T A^k Q^k$$

dunque A^{k+1} è simile ad A . Si può dimostrare che

$$A^k \rightarrow T \text{ per } k \rightarrow \infty$$

più precisamente:

$$\begin{aligned} A \text{ simmetrica} &\implies A^k \text{ simmetrica} \\ &\implies T \text{ simmetrica} \\ &\implies T \text{ diagonale} \end{aligned}$$

dunque $T = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n)$. Nel **test di arresto** mi fermo quando:

$$\|T(A^k)\| < \text{Tol}$$

dove $T(A^k)$ è triangolo inferiore stretto di A^k . Questo algoritmo si chiama **algoritmo QR**.

Si dimostra che se gli autovalori di A simmetrica sono distinti si dimostra che l'algoritmo QR produce una successione di matrici A^k le quali convergono a $T = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n)$.

Caso non simmetrico

Applico l'algoritmo QR, per cui ottengo una successione di matrici $A^k \rightarrow \tilde{T}$ triangolare superiore con la sottodiagonale della fattorizzazione di Schur reale.

Devo modificare il **test di arresto** in modo da tener conto del fatto che la sottodiagonale di $\tilde{T}$ non è nulla.

7.4 SVD

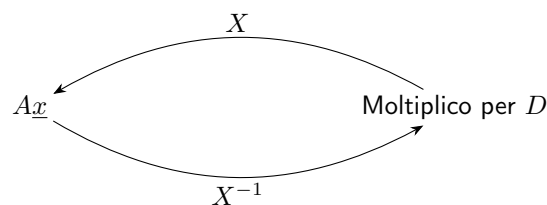
$\forall A$ matrice $m \times n \exists U, V$ ortogonali tale che:

$$A = U\Sigma V^T$$

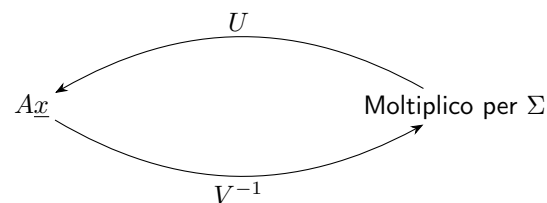
dove $U \in \mathbb{R}^{m \times m}$, $V \in \mathbb{R}^{n \times n}$ e $\Sigma \in \mathbb{R}^{m \times n}$ è una matrice diagonale. $\Sigma = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_n)$ con $\sigma_i \geq 0$ valori singolari di A .

U è $m \times m$, V è $n \times n$.

- **Diagonalizzazione:** se A $n \times n$ è diagonalizzabile $A = XDX^{-1}$



- **SVD** $A = U\Sigma V^T$



$$\begin{bmatrix} A \end{bmatrix} = \begin{bmatrix} U & \vdots \\ & \end{bmatrix}_m \begin{bmatrix} \Sigma \end{bmatrix}_m \begin{bmatrix} V^T \end{bmatrix}_n$$

colonne *silenti*

le colonne *silenti* moltiplicano degli zeri di Σ

Si definisce dunque la **SVD ridotta**:

$$\begin{bmatrix} A \end{bmatrix} = \begin{bmatrix} \tilde{U} \end{bmatrix}_m \begin{bmatrix} \tilde{\Sigma} \end{bmatrix}_n \begin{bmatrix} V^T \end{bmatrix}_n$$

con $\tilde{U}$ che contiene le prime n colonne di U . Vediamo dunque chi sono i σ :

$$\begin{aligned} A^T A &= (U \Sigma V^T)^T (U \Sigma V^T) \\ &= V \Sigma^T U^T U \Sigma V^T \\ &= V \Sigma^T \Sigma V^T \quad \Sigma^T \Sigma = \text{diag}(\sigma_1^2, \sigma_2^2, \dots, \sigma_n^2) \end{aligned}$$

$A^T A$ è simmetrica e diagonalizzabile con matrici ortogonali. Dunque $V \Sigma^T \Sigma V^T$ è la diagonalizzazione di $A^T A$, cioè i σ_i^2 sono gli autovalori di $A^T A$. Nella forma ridotta risparmiamo memoria.

$\text{rank}(A) = \#$ valori singolari non nulli

Supponiamo che i σ_i siano ordinati e che $\text{rank}(A) = k \leq n$. Allora:

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_k > 0$$

$$\sigma_{k+1} = \dots = \sigma_n = 0$$

Quindi se la matrice A non ha tutte le colonne linearmente indipendenti, allora esistono dei valori singolari nulli. Posso dunque ridurre le dimensioni ulteriormente, perchè le ultime $n - k$ colonne di $\tilde{U}$ sono silenti e le ultime $n - k$ righe di V^T sono silenti.

$$\begin{bmatrix} A \end{bmatrix} = \begin{bmatrix} \tilde{U} \end{bmatrix}_m \begin{bmatrix} \tilde{\Sigma} \end{bmatrix}_n \begin{bmatrix} V^T \end{bmatrix}_n$$

$\begin{matrix} n \\ \vdots \\ k \end{matrix}$
 $\begin{matrix} n \\ \vdots \\ n-k \end{matrix}$

Osservazione 7.5. Se D è una matrice diagonale, calcolo

$DA =$ la riga i -esima di A viene moltiplicata per d_{ii}

$AD =$ la colonna i -esima di A viene moltiplicata per d_{ii}

Otteniamo dunque una forma ancora più ridotta:

$$\begin{bmatrix} A \end{bmatrix} = \begin{bmatrix} \tilde{\tilde{U}} \end{bmatrix}_m \begin{bmatrix} \tilde{\tilde{\Sigma}} \end{bmatrix}_k \begin{bmatrix} \tilde{V}^T \end{bmatrix}_k$$

Osservo che posso scrivere A come:

$$A = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \cdots + \sigma_k u_k v_k^T = \sum_{i=1}^k \sigma_i \underline{u}_i \underline{v}_i^T$$

le $\sigma_i \underline{u}_i \underline{v}_i^T$ sono matrici di rango 1. Ricordiamo che $\underline{u}_i$ e $\underline{v}_i$ sono ortonormali tra loro.

Osservazione: norma 2 di matrici:

Teorema 7.7.

$$\|A\|_2 = \sigma_1$$

Dimostrazione. Ricordo che $\|A\|_2 = \sup_{x \neq 0} \frac{\|Ax\|_2}{\|x\|_2}$.

$$\begin{aligned} \|Ax\|_2^2 &= \langle Ax, Ax \rangle \\ &= \langle x, A^T A x \rangle \end{aligned}$$

$A^T A$ è simmetrica: chiamo $\lambda_1, \dots, \lambda_n$ i suoi autovalori $\lambda_i \in \mathbb{R}$ e dunque $|\lambda_M| = \max_i |\lambda_i|$. I suoi autovettori $\underline{w}_1, \dots, \underline{w}_n$ sono ortogonali e formano una base di $\mathbb{R}^n$.

$$\exists c_1, \dots, c_n : x = \sum_{i=1}^n c_i \underline{w}_i$$

$$\begin{aligned} \langle w_i, A^T A w_i \rangle &= \lambda_i \langle w_i, w_i \rangle \quad \forall i \\ &\leq \lambda_M \|w_M\|^2 \end{aligned}$$

Dunque:

$$\|Ax\|_2^2 \leq \lambda_M \|x\|^2$$

Gli autovalori di $A^T A$ sono $\lambda_i \in \mathbb{R}$ e $\lambda_i \geq 0$.

$$A^T A = (U \Sigma V^T)^T (U \Sigma V^T) = V \Sigma^T \underbrace{U^T U}_{\mathbb{I}} \Sigma V^T = V \Sigma^T \Sigma V^T = V \Sigma^2 V^T$$

dato che Σ è diagonale, dunque $\Sigma^T = \Sigma$ e $\Sigma^2 = \text{diag}(\sigma_1^2, \sigma_2^2, \dots, \sigma_n^2)$. Quindi gli autovalori λ_i di $A^T A$ sono $\lambda_i = \sigma_i^2$.

$$\frac{\|Ax\|}{\|x\|} \leq \sqrt{\lambda_M} = \sigma_1$$

□

Inoltre $A^T A V = V \Lambda$ dove $\Lambda = \Sigma^T \Sigma$ è una matrice diagonale con elementi $\omega_{ii} = \sigma_i^2$. Osserviamo che $A^T A$ è simmetrica, quadrata e definita positiva. Mentre:

$$A A^T = U \Sigma \underbrace{V^T V}_{\mathbb{I}} \Sigma^T U^T = U \Sigma^2 U^T$$

e

$$A A^T U = U \Lambda$$

Dunque i vettori singolari sinistri sono autovettori di $A A^T$ e i vettori singolari destri sono autovettori di $A^T A$.

Se $A \in \mathbb{R}^{n \times n}$ quadrata, $\det A \neq 0$ allora:

$$\|A^{-1}\|_2 = \frac{1}{\sigma_n} \quad A^{-1} = (U \Sigma V^T)^{-1} = V \Sigma^{-1} U^T$$

dove σ_n è il più piccolo autovalore non nullo di A e il condizionamento della matrice A che è dato da:

$$\kappa(A) = \|A\|_2 \|A^{-1}\|_2 = \frac{\sigma_1}{\sigma_n}$$

Sia ora $A \in \mathbb{R}^{n \times m}$, di cui abbiamo la SVD $A = U\Sigma V^T$. Dato $\text{rank}(A) = k$ e $\tilde{k} < k$ allora:

$$B = \sum_{i=1}^{\tilde{k}} \sigma_i \underline{u}_i \underline{v}_i^T \quad \text{troncamento SVD}$$

e posso trovare l'errore dato da

$$\|A - B\|_2 = \sigma_{\tilde{k}+1} \quad \|A - B\|_F = \sqrt{\sum_{i=\tilde{k}+1}^k \sigma_i^2}$$

Teorema 7.8. Sia $A \in \mathbb{R}^{n \times m}$, $A_{\tilde{k}} = \sum_{i=1}^{\tilde{k}} \sigma_i \underline{u}_i \underline{v}_i^T$ e $\text{rank}(A) = k, \tilde{k} < k$. Inoltre $\|A - A_{\tilde{k}}\|_2 = \sigma_{\tilde{k}+1}$. Allora la sua migliore approssimazione con una matrice più piccola è data da:

$$\min_{\substack{B \in \mathbb{R}^{n \times m} \\ \text{rank}(B) = \tilde{k}}} \|A - B\| = \|A - A_{\tilde{k}}\|_2 = \sigma_{\tilde{k}+1}$$

Questa è detta **best low rank approximation**.

Un'altra proprietà per $A \in \mathbb{R}^{n \times n}$ quadrata e invertibile

$$A = U\Sigma V^T \implies A^{-1} = V\Sigma^{-1}U^T$$

Se A è $n \times m$ posso costruire la sua pseudoinversa:

$$A^\dagger = V\Sigma^\dagger U^T$$

dove

$$\Sigma^\dagger = \text{diag}\left(\frac{1}{\sigma_1}, \frac{1}{\sigma_2}, \dots, \frac{1}{\sigma_k}\right) \quad \text{se } \text{rank}(A) = k$$

$A^\dagger$ è detta **pseudoinversa di Moore-Penrose**.

Dunque:

$$AA^\dagger = U\Sigma V^T V\Sigma^\dagger U^T = U\Sigma\Sigma^\dagger U^T = U \begin{pmatrix} \mathbb{I} \\ 0 \end{pmatrix} U^T$$

$\Sigma\Sigma^\dagger$ è dunque quadrata e ha nella diagonale 1 fino a $\tilde{k}$ e 0 altrove.

7.5 Calcolo della SVD

1. Diagonalizzazione di AA^T o $A^T A$.

- Si calcolano autovalori λ_i e autovettori di AA^T ($m \times m$) o di $A^T A$ ($n \times n$).
- I valori singolari sono $\sigma_i = \sqrt{\lambda_i}$.
- Se $m \gg n$ (o viceversa), la matrice da diagonalizzare è molto grande e l'operazione diventa costosa.
- Ricaviamo i vettori sinistri dai destri con

$$Av_i = \sigma_i u_i \implies u_i = \frac{Av_i}{\sigma_i}.$$

2. Controllo del condizionamento e instabilità numerica.

- Se $\sigma_i^2 = \lambda_i < \text{eps}$, allora $\sigma_i < \sqrt{\text{eps}}$ e la divisione per σ_i amplifica gli errori numerici.
- Bisogna quindi scartare o regolarizzare i valori singolari troppo piccoli per evitare instabilità.

3. Riduzione bidiagonale e QR (o proiezione).

- Si riduce A a forma bidiagonale B tramite trasformazioni ortogonali:

$$A = Q_1 B Q_2^T,$$

quindi si calcola la SVD di B , il che è molto più efficiente.

- Alternativamente, quando $n \gg m$, si sfrutta che

$$A^T A V = V \Sigma^2 \implies A V = U \Sigma,$$

e si ottiene U eseguendo un'ulteriore fattorizzazione QR di $A V$.

Osservazione 7.6. *Nessuno dei metodi presentati è efficiente.*

8 Interpolazione Integrati

Abbiamo precedentemente visto come approssimare le funzioni utilizzando i polinomi di Lagrange.

$$p(x) = \sum_{i=0}^n f(x_i) \ell_i(x) \quad \ell_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}$$

per cui abbiamo stimato l'errore:

$$e_n(x) = \omega_{n+1}(x) \frac{f^{(n+1)}(\xi)}{(n+1)!} \quad \omega_{n+1}(x) = \prod_{i=0}^n (x - x_i)$$

Esempio 8.1. Stimiamo l'errore del polinomio interpolante di grado 7 per la funzione $f(x) = \sin(x)^2$ in $[-5, 5]$. Osseviamo che $f \in C^8([-5, 5])$ (in realtà è C^∞), per ciò possiamo procedere alla stima:

$$\omega_8(x) \leq |b - a|^8 = 10^8$$

Mentre

$$f^{(8)}(x) = -128 \cos(2x)$$

e dunque:

$$|\omega_7(x)| \leq \frac{\|\omega_8\|_\infty \|f^{(8)}\|_\infty}{8!} \leq \frac{10^8 \cdot 128}{8!} \quad (\text{stima ancora troppo alta})$$

Vediamo dunque come controllare meglio l'errore.

$$|x - x_j| \leq (b - a) \quad x_j \in [a, b]$$

La prima idea potrebbe essere quella di considerare un intervallo più piccolo. Nell'esempio precedente, se avessimo considerato $[0, 1]$ avremmo avuto:

$$\omega_7(x) \leq \frac{128}{8!}$$

Possiamo dunque dividere l'intervallo in più sottointervalli e costruire il polinomio interpolante su ciascuno di essi.

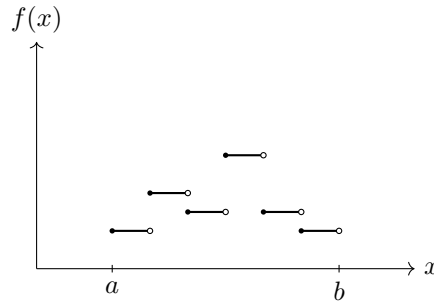
8.1 Interpolazione composita

Costruiamo la griglia di punti $a = x_0 < x_1 < \dots < x_n = b$ e chiamiamo $I_j = [x_j, x_{j+1}]$. Dunque $[a, b] = \bigcup_{j=0}^{n-1} I_j$. Definiamo una funzione

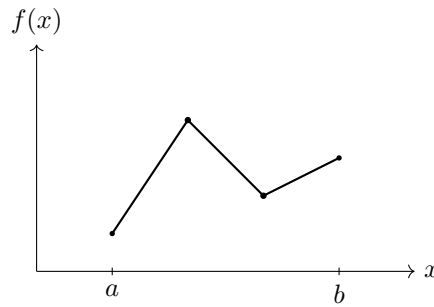
$$Q^d : [a, b] \rightarrow \mathbb{R} \\ x \mapsto Q^d(x) = p_j^d(x) \quad x \in I_j$$

Dove p_j^d è il polinomio interpolante di grado d su I_j e Q^d è **polinomiale a tratti**.

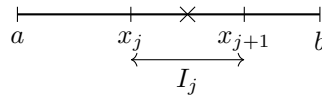
Se $d = 0$ allora Q^0 è **costante a tratti**.



Se $d = 1$ allora Q^1 è **lineare a tratti**.

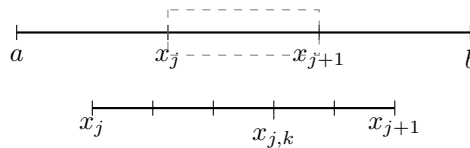


Se $d = 2$ allora Q^2 è **quadratica a tratti**, ma per costruire una parabola su ogni intervallo ci servono 3 punti.

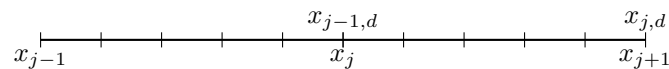


Osservazione 8.1. Potremmo utilizzare anche i nodi di I_{j+1} per costruire la parabola, ma perdiamo il controllo sull'ampiezza del dominio. Preferiamo dunque aggiungere punti extra in ogni I_j .

Definizione 8.1. L'insieme dei punti che determinano $p_j^d(x)$ costituiscono lo **stencil** di p_j .



Con $k = 0, \dots, d$: per $p_j^d(x)$ ci servono $d + 1$ punti/nodi. Osserviamo che $x_{j,0} = x_{j-1,d} = x_j$ e $x_{j,d} = x_{j+1,0}$



Vogliamo p_j^d tale che $p_j^d(x_{j,k}) = f(x_{j,k})$ per $k = 0, \dots, d$.

8.2 Errore approssimazione composita

$$e_j^d = f(x) - p_j^d(x) = \underbrace{\left(\prod_{k=0}^d (x - x_{j,k}) \right)}_{\omega_{d+1}(x)} \frac{f^{(d+1)}(\xi)}{(d+1)!}$$

Naturalmente f deve essere $d+1$ volte derivabile. Chiamo $h_j = x_{j+1} - x_j$ l'ampiezza dell'intervallo I_j .

$$\|e_j^d\| \leq h_j^{d+1} \frac{\|f^{(d+1)}\|_\infty}{(d+1)!} \leq h^{d+1} \frac{\|f^{(d+1)}\|_\infty}{(d+1)!} \quad \text{con } h = \max_j h_j$$

Se la griglia è uniforme, allora $h_j = h$ per ogni j .

Osservazioni

- Possiamo controllare l'errore facendo diminuire h
- Richiede meno regolarità di f rispetto all'approssimazione con polinomi interpolanti

$$d = 0 \implies |e_j^0| \leq h \|f'\|_\infty \quad f \in C^1([a, b])$$

$\vdots$

$$d = 2 \implies |e_j^2| \leq \frac{1}{6} h^3 \|f'''\|_\infty \quad f \in C^3([a, b])$$

Osservazione 8.2.

$$Q^d \in \left\{ v : v \Big|_{I_j} \in \mathbb{P}^d \wedge v(x_{j,k}) = f(x_{j,k}) \right. \\ \left. \forall j=0, \dots, n \quad k=0, \dots, d \right\}$$

Osservazione 8.3. $v(x_{j,k}) = f(x_{j,k})$ impone anche la continuità di v

$$\lim_{x \rightarrow x_{j,k}^-} Q^d(x) = p_{j-1}^d(x_j) = f(x_j)$$

$$\lim_{x \rightarrow x_{j,k}^+} Q^d(x) = p_j^d(x_j) = f(x_j)$$

Dunque $Q^d \in C^0([a, b])$

Struttura dello spazio delle funzioni polinomiali a tratti

$[a, b] = \bigcup_{j=0}^{n-1} I_j$ con $I_j = [x_j, x_{j+1}]$, $h_j = x_{j+1} - x_j$ e $I_j \cap I_{j+1} = \{x_{j+1}\}$. Chiamo $\bar{Q}^d := \left\{ v : v \Big|_{I_j} \in \mathbb{P}^d \right\}$.

$$\text{Se } v_1, v_2 \in \bar{Q}^d \longrightarrow v_1 + v_2 \in \bar{Q}^d$$

$$\text{Se } \alpha \in \mathbb{R}, v \in \bar{Q}^d \longrightarrow \alpha v \in \bar{Q}^d$$

Dunque $\bar{Q}^d$ è uno **spazio lineare**. Notiamo che $\dim \bar{Q}^d = (d+1)N$ dove $N = \# \text{sottointervalli}$ e $d = \# \text{nodi in } I_j$. Vediamo dunque come diventa la **base di Lagrange**:

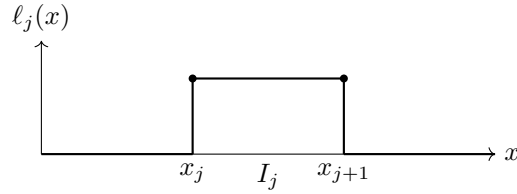
$$\ell_{j,k}(x_{i,z}) = \begin{cases} 1 & \text{se } i = j \text{ e } z = k \\ 0 & \text{altrimenti} \end{cases}$$

dove $i = j$ è la condizione sull'intervallo e $z = k$ è la condizione sui nodi.

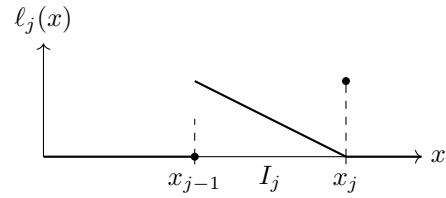
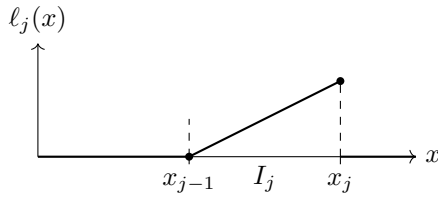
Ricordiamo infatti la condizione base che definisce la base di Lagrange:

$$\ell_i(x_j) = \delta_{ij} = \begin{cases} 1 & \text{se } i = j \\ 0 & \text{altrimenti} \end{cases}$$

Se $d = 0$, $\ell_j(x) = 1$ se $x \in I_j$



Se $d = 1$:



Consideriamo ora:

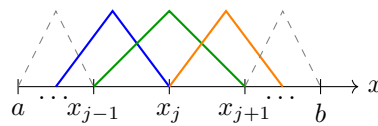
$$Q^d = \left\{ v : v|_{I_j} \in \mathbb{P}^d \wedge v \in C^0([a, b]) \right\}$$

e $\dim Q^d = \dim \bar{Q}^d - (n - 1)$, dove $n - 1$ è il numero di condizioni per la continuità di v . Vale inoltre la continuità dei nodi interni

$$\lim_{x \rightarrow x_j^-} v(x) = \lim_{x \rightarrow x_j^+} v(x) \quad \forall j = 1, \dots, n - 1$$

dunque $\dim Q^d = nd + 1$.

Se $d = 1$, $\dim Q^1 = n + 1$.



Vediamo che la base di Lagrange è formata da **funzioni a cappello**.

Applicazioni di interpolazioni polinomiali

Tutte le funzioni interpolanti Φ permettono

1. $\Phi \approx f$
2. $\int_a^b \Phi \approx \int_a^b f$
3. $\Phi' \approx f'$

8.3 Integrazione numerica

Siamo interessati ad approssimare la quantità:

$$I(f) := \int_a^b f(x) dx$$

ovvero ad approssimare l'area sottesa ad f in $[a, b]$. Approssimiamo f con f_n e otteniamo

$$I_n(f) = \int_a^b f_n(x) dx \quad \text{con l'idea che} \quad I_n(f) \approx I(f)$$

Se $f \in C^0([a, b])$, $E_n(f) = I(f) - I_n(f)$ è l'errore di interpolazione.

$$\begin{aligned} |E_n(f)| &= \left| \int_a^b f(x) - f_n(x) dx \right| \\ &\leq \int_a^b |f(x) - f_n(x)| \\ &\leq (b-a) \|f - f_n\|_\infty \end{aligned}$$

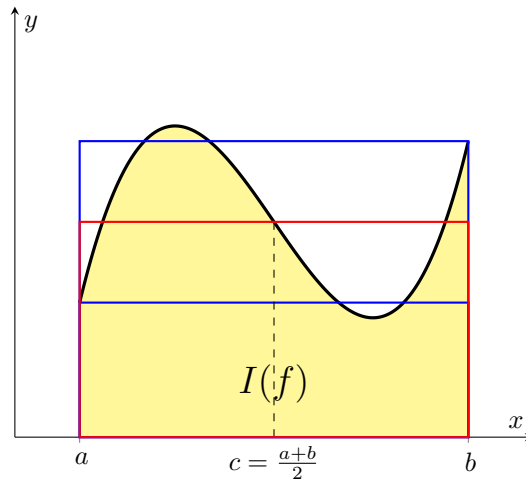
Ossia l'errore su $I(f)$ dipende dall'errore su f . Riprendendo l'idea dell'interpolazione di Lagrange, scegliendo $f_n(x) = \sum_{i=0}^n f(x_i) \ell_i(x)$ si ha:

$$\begin{aligned} I_n(f) &= \int_a^b f_n(x) dx \\ &= \sum_{i=0}^n \left(f(x_i) \underbrace{\int_a^b \ell_i(x) dx}_{\alpha_i} \right) \\ &= \sum_{i=0}^n \alpha_i f(x_i) \end{aligned}$$

Ci siamo dunque ricondotti a una combinazione lineare di f nei nodi.

8.4 Metodo dei rettangoli

Il metodo dei rettangoli si basa sull'approssimazione dell'integrale con la costruzione di un rettangolo che ha per base l'intervallo di integrazione e per altezza il valore della funzione in un punto dell'intervallo.



Si usa spesso il punto medio, per cui si approssima l'integrale con:

$$I(f) = (b-a)f\left(\frac{a+b}{2}\right)$$

Quando la funzione è costante il metodo dei rettangoli è esatto. Sia infatti $f(x) = k$ con $k \in \mathbb{R}$. Allora:

$$I(f) = (b-a)k$$

$$I_0(f) = k(b-a)$$

Prendiamo invece $f(x) = x$. Allora:

$$I(f) = \int_0^1 x dx = \frac{1}{2}$$

$$I_0(f) = 1 \cdot \frac{1}{2} = \frac{1}{2}$$

e dunque, anche per questa funzione $I(f) = I_0(f)$.

Sia $f(x) = x^2$. Allora:

$$I(f) = \int_0^1 x^2 dx = \frac{1}{3}$$

$$I_0(f) = 1 \left(\frac{1}{2}\right)^2 = \frac{1}{4} \neq \frac{1}{3}$$

ed è evidente che $I(f) \neq I_0(f)$.

Andiamo ad analizzare dunque l'esattezza, per cui si può determinare un'interpolazione che ha errore nullo.

8.4.1 Errore

Usiamo Taylor per la stima dell'errore.

$$f(x) = f(c) + f'(c)(x-c) + \frac{f''(\xi)}{2}(x-c)^2$$

Dunque genericamente:

$$I(f) - I_0(f) = \int_a^b (f(x) - f(c)) dx = \int_a^b \left(f'(c)(x-c) + \frac{f''(\xi)}{2}(x-c)^2 \right) dx$$

ci aspettiamo dunque che $\int_a^b f'(c)(x-c) dx = 0$ per le funzioni lineari. Risolviamo dunque l'integrale:

$$\begin{aligned} \int_a^b f'(c)(x-c) dx &= f'(c) \frac{(x-c)^2}{2} \Big|_a^b \\ &= f'(c) \frac{(b-c)^2 - (a-c)^2}{2} \\ &= \frac{f'(c)}{2} \left(\left(\frac{b-a}{2} \right)^2 - \left(\frac{a-b}{2} \right)^2 \right) \\ &= 0 \end{aligned}$$

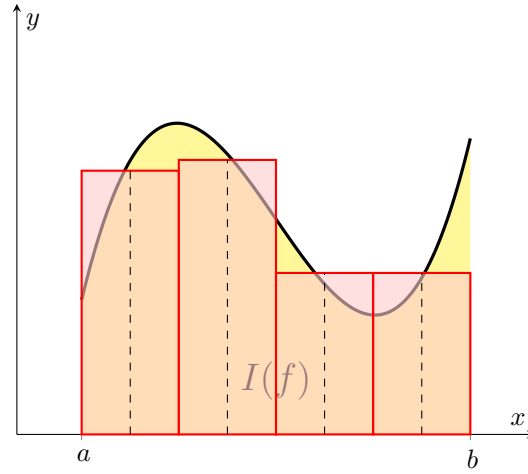
Dunque

$$E_0(f) = \int_a^b \frac{f''(\xi)}{2}(x-c)^2 dx$$

Grado di esattezza è 1 poichè $f''(x) = 0$ con $f(x) = ax + b$. Dunque:

$$E_0(f) = \frac{f''(\xi)}{2} \frac{(x-c)^3}{3} \Big|_a^b = \dots = \frac{h^3}{3} f''(\xi) \quad h = \frac{b-a}{2}$$

8.5 Interpolazione composta



Per un'interpolazione più precisa possiamo suddividere $[a, b]$ in m sottointervalli, per cui abbiamo una griglia di punti $a = x_0 < x_1 < \dots < x_m = b$.

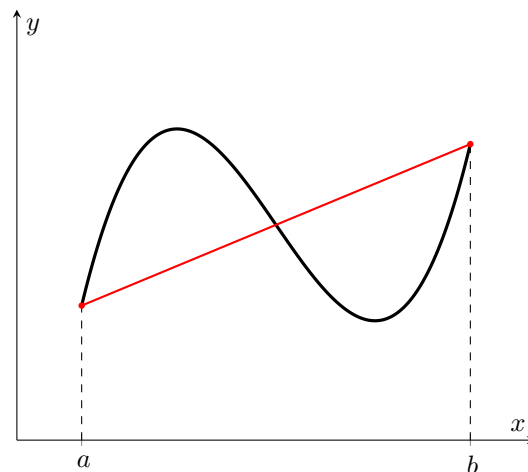
Vediamo il caso in cui la griglia è uniforme, cioè $H = \frac{b-a}{m}$. Dunque ci interessano i punti medi $x_i = a + (2i+1)\frac{H}{2}$ con $i = 0, 1, \dots, m-1$ (x_i sono la griglia dei punti medi della partizione dei sottointervalli).

$$I_{0,m}(f) = Hf(x_0) + Hf(x_1) + \dots + Hf(x_{m-1}) = H \sum_{i=0}^{m-1} f(x_i)$$

Osserviamo che l'area del generico sottointervallo vale $Hf(x_i)$. Dunque per l'errore si ha:

$$E_{0,m}(f) = \frac{(b-a)}{24} H^2 f''(\xi)$$

8.6 Metodo dei trapezi



Una migliore strategia per migliorare la precisione dell'interpolazione è quella di usare polinomi di grado maggiore.

$$I_1(f) = (b-a) \left(\frac{f(a) + f(b)}{2} \right)$$

Se $f(x) = k$ allora:

$$I(f) = k(b-a) \quad I_1(f) = \frac{(b-a)}{2} \cdot (k+k) = k(b-a)$$

Prendiamo invece $f(x) = x$. Allora:

$$I(f) = \int_0^1 x \, dx = \frac{1}{2}$$

$$I_1(f) = \frac{1}{2} (0+1) = \frac{1}{2}$$

Prendiamo invece $f(x) = x^2$. Allora:

$$I(f) = \int_0^1 x^2 \, dx = \frac{1}{3}$$

$$I_1(f) = \frac{1}{2} (0+1) = \frac{1}{2} \neq I(f)$$

Ed è evidente che il grado di esattezza del metodo dei trapezi è 1.

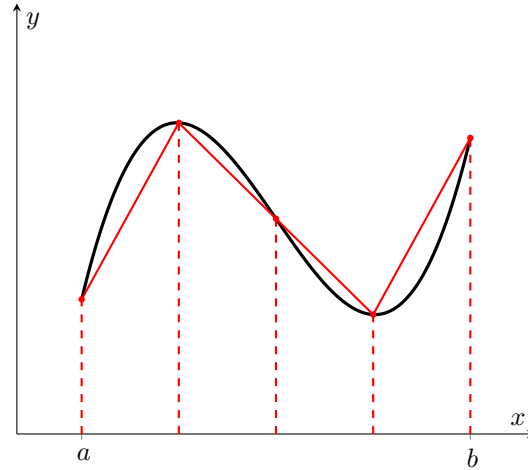
$$E_1(f) = \int_a^b f(x) - p_1(x) \, dx$$

dove $p_1(x)$ è il polinomio interpolante

$$\begin{aligned} E_1(f) &= \frac{1}{2} \int_a^b f''(\xi)(x-a)(x-b) \, dx \\ &= \frac{1}{2} f''(\xi) \int_a^b (x-a)(x-b) \, dx \\ &= \frac{1}{2} f''(\xi) \left[\frac{x^3}{3} - \frac{(a+b)x^2}{2} + abx \right] \Big|_a^b \\ &= \frac{1}{2} f''(\xi) \left[\frac{b^3}{3} - \frac{(a+b)b^2}{2} + ab^2 - \left(\frac{a^3}{3} - \frac{(a+b)a^2}{2} + a^2b \right) \right] \\ &= \frac{1}{2} f''(\xi) \left[\frac{b^3}{3} - \frac{(a+b)b^2}{2} + ab^2 - \frac{a^3}{3} + \frac{(a+b)a^2}{2} - a^2b \right] \\ &= \frac{1}{2} f''(\xi) \left[-\frac{b^3}{6} + \frac{ab^2}{2} + \frac{a^3}{6} - \frac{a^2b}{2} \right] \\ &= \frac{1}{12} f''(\xi) [a^3 - 3ab^2 + 3a^2b - b^3] \\ &= \frac{1}{12} f''(\xi)(a-b)^3 \end{aligned}$$

Dunque $E_1(f) = \frac{1}{12} f''(\xi)(a-b)^3$.

Possiamo dunque proporre anche in questo caso un **metodo composito**, per cui per ogni sottointervallo si applica il metodo dei trapezi e si sommano i risultati.



Abbiamo dunque la griglia equidistante con $H = \frac{b-a}{m}$. E i punti sono $x_i = a + iH$ con $i = 0, 1, \dots, m$.

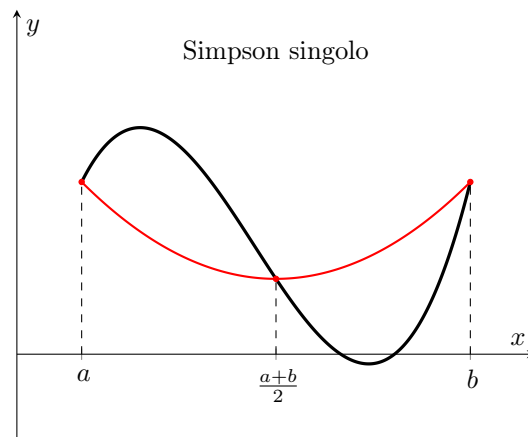
$$\begin{aligned} I_{1,m}(f) &= \frac{H}{2} (f(x_0) + f(x_1)) + \frac{H}{2} (f(x_1) + f(x_2)) + \dots + \frac{H}{2} (f(x_{m-1}) + f(x_m)) \\ &= H \left[\frac{1}{2} f(x_0) + \sum_{i=1}^{m-1} f(x_i) + \frac{1}{2} f(x_m) \right] \\ &= \frac{H}{2} \sum_{i=0}^{m-1} [f(x_i) + f(x_{i+1})] \end{aligned}$$

In questo caso l'errore è:

$$E_{1,m}(f) = -\frac{(b-a)}{12} H^2 f''(\xi) \quad f \in C^2([a, b]) \quad \xi \in (a, b)$$

Utilizziamo dunque un polinomio interpolante di grado 2.

8.7 Metodo di Cavalieri-Simpson



$$I_2(f) = \frac{b-a}{6} (f(a) + 4f(c) + f(b))$$

dove $c = \frac{a+b}{2}$.

Se $f(x) = k$ allora:

$$I(f) = k(b-a) \quad I_2(f) = \frac{b-a}{6} (k + 4k + k) = k(b-a)$$

Se $f(x) = x$ allora:

$$I(f) = \int_0^1 x \, dx = \frac{1}{2}$$

$$I_2(f) = \frac{1}{6} \left(0 + 4 \cdot \frac{1}{2} + 1 \right) = \frac{1}{2}$$

Se $f(x) = x^2$ allora:

$$I(f) = \int_0^1 x^2 \, dx = \frac{1}{3}$$

$$I_2(f) = \frac{1}{6} \left(0 + 4 \cdot \frac{1}{4} + 1 \right) = \frac{1}{3}$$

Proviamo dunque $f(x) = x^3$. Allora:

$$I(f) = \int_0^1 x^3 \, dx = \frac{1}{4}$$

$$I_2(f) = \frac{1}{6} \left(0 + 4 \cdot \frac{1}{8} + 1 \right) = \frac{1}{4}$$

Per $f(x) = x^4$ allora:

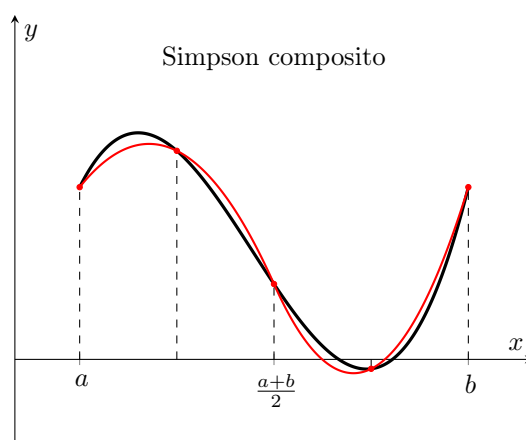
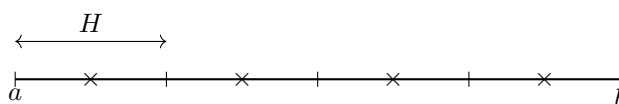
$$I(f) = \int_0^1 x^4 \, dx = \frac{1}{5}$$

$$I_2(f) = \frac{1}{6} \left(0 + 4 \cdot \frac{1}{16} + 1 \right) = \frac{5}{24} \neq \frac{1}{5}$$

Dunque il grado di esattezza è 3 (non dimostrato).

$$E_2(f) = -\frac{h^5}{90} f^{(4)}(\xi) \quad h = \frac{b-a}{2}$$

E chiaramente possiamo applicarlo in maniera composita. $H = \frac{b-a}{m}$. Dunque il generico punto è $x_i = a + i \frac{H}{2}$ con $i = 0, 1, \dots, 2m$.



$$\begin{aligned}
I_{2,m}(f) &= \frac{H}{6} (f(x_0) + 4f(x_1) + f(x_2)) + \frac{H}{6} (f(x_2) + 4f(x_3) + f(x_4)) + \cdots + \frac{H}{6} (f(x_{2m-2}) + 4f(x_{2m-1}) + f(x_{2m})) \\
&= \frac{H}{6} \left(f(x_0) + 4 \sum_{k=0}^{m-1} f(x_{2k+1}) + 2 \sum_{s=1}^{m-1} f(x_{2s}) + f(x_{2m}) \right)
\end{aligned}$$

e l'errore è dato da:

$$E_{2,m}(f) = \frac{(b-a)}{180} \left(\frac{h}{2} \right)^4 f^{(4)}(\xi) \quad f \in C^4([a, b]) \quad \xi \in (a, b)$$

9 Quadratura

L'idea è di avere un metodo numerico per approssimare $I(f)$ raggiungendo un errore prestabilito $\varepsilon > 0$.

$$\int_a^b f(x) dx$$

e posso considerare l'intervallo $[\alpha, \beta] \subseteq [a, b]$.



Considero:

$$I_f(\alpha, \beta) = \int_{\alpha}^{\beta} f(x) dx \quad \text{integrale esatto in } [\alpha, \beta]$$

e posso usare il metodo Cavalieri-Simpson partendo da $h_0 = \frac{\beta - \alpha}{2}$

$$\int_f(\alpha, \beta) = \frac{h_0}{3} [f(\alpha) + 4f(\alpha + h_0) + f(\beta)]$$

e l'errore è dato da

$$I_f(\alpha, \beta) - \int_f(\alpha, \beta) = -\frac{h_0^5}{90} f^{(4)}(\xi)$$

e vogliamo stimarlo senza calcolare $f^{(4)}$. Sappiamo che su intervalli più piccoli abbiamo un errore più "basso", dunque:

$$[\alpha, \beta] = [\alpha, \alpha + h_0] \cup [\alpha + h_0, \beta] \quad \text{entrambi di lunghezza } h_0$$

per cui si applica il metodo di Simpson sui due intervalli:

$$I_f(\alpha, \beta) - \int_{f,2}(\alpha, \beta) = -\frac{h_0^5}{2880} (f^{(4)}(\xi) + f^{(4)}(\eta))$$

Dove $\xi \in (\alpha, \alpha + h_0)$ e $\eta \in (\alpha + h_0, \beta)$. L'ipotesi è che $f^{(4)}(\xi) \approx f^{(4)}(\eta)$.

$$I_f(\alpha, \beta) - \int_{f,2}(\alpha, \beta) \simeq -\frac{1}{16} \frac{h_0^5}{90} f^{(4)}(\xi)$$

Definiamo

$$\begin{aligned}
 \varepsilon_f(\alpha, \beta) &= \int_f(\alpha, \beta) - \int_{f,2}(\alpha, \beta) \\
 &= I_f(\alpha, \beta) - \int_{f,2}(\alpha, \beta) - \left(I_f(\alpha, \beta) - \int_f(\alpha, \beta) \right) \\
 &\simeq -\frac{1}{16} \frac{h_0^5}{90} f^{(4)}(\xi) + \frac{h_0^5}{90} f^{(4)}(\xi) \\
 &= \frac{h_0^5}{90} f^{(4)}(\xi) \left(1 - \frac{1}{16} \right) \\
 &\implies \frac{16}{15} \varepsilon_f(\alpha, \beta) \approx \frac{h_0^5}{90} f^{(4)}(\xi)
 \end{aligned}$$

Dunque abbiamo che

$$|I_f(\alpha, \beta) - S_{f,12}(\alpha, \beta)| \simeq \frac{1}{16} \frac{h_0^5}{90} f^{(4)}(\xi)$$

$\varepsilon_f(\alpha, \beta)$ si chiama criterio a posteriori dato che ε_f dipende esclusivamente dal metodo numerico.

Usiamo per ciò un **metodo adattivo**: calcoliamo $\int_f(a, b)$ e $\int_{f,12}(a, b)$ e troviamo $\varepsilon_f(a, b)$. Per cui se $|\varepsilon_f(a, b)| < \varepsilon$ allora non devo migliorare il metodo. Altrimenti poniamo $a = a$ e $b = a + \frac{b-a}{2}$. Possiamo usare il seguente **criterio di arresto**

$$\frac{|\varepsilon_f(\alpha, \beta)|}{15} \leq \varepsilon \frac{(\beta - \alpha)}{(b - a)}$$

il 15 a denominatore deriva dalla stima

$$\frac{1}{16} \frac{h_0^5}{90} f^{(4)}(\xi) \approx \frac{1}{16} \frac{16}{15} \varepsilon_f.$$

10 Metodi numerici per ODE

Consideriamo il problema di Cauchy:

$$\begin{cases} \dot{y}(t) = f(t, y(t)) & t \in [t_0, T] \\ y(t_0) = y_0 \end{cases}$$

con $y_0 \in \mathbb{R}^n$, $f : I \times \mathbb{R}^n \rightarrow \mathbb{R}^n$. Il problema di Cauchy descrive un'evoluzione temporale.

Se $n > 1$ il sistema è equivalente a una ODE di ordine n . Per $n = 2$, ad esempio, abbiamo l'oscillatore armonico e $F = ma$.

Osservazione 10.1. Se $f(y(t))$ non dipende esplicitamente dal tempo la ODE si dice **autonoma**.

$$\begin{cases} \dot{y}(t) = f(y(t)) \\ y(t_0) = y_0 \end{cases}$$

è invariante nel tempo.

Vediamo dunque come approssimare la soluzione del problema di Cauchy:

$$\begin{cases} \dot{y}(t) = f(t, y(t)) \\ y(t_0) = y_0 \end{cases} \quad y(t) = y_0 + \int_{t_0}^t f(s, y(s)) \, ds$$

A noi interessa innanzitutto assicurarci che la soluzione esista e sia unica. f deve dunque essere continua e lipschitziana in $\mathbb{R}$.

$$|f(x) - f(y)| \leq L|x - y| \quad L > 0 \quad L \in \mathbb{R}$$

Possiamo avere esistenza locale oppure globale. Dipende dall'insieme in cui f è di tipo lipschitz. A noi interessa pure che la soluzione sia "stabile", ovvero che piccole perturbazioni dei dati iniziali producano piccole variazioni della soluzione (ci serve per tenere conto dell'errore numerico).

Definizione 10.1 (Stabilità secondo Lyapunov). Sia I un intervallo chiuso e limitato. Il problema di Cauchy è stabile secondo Lyapunov su I se, per $(\delta_0, \delta(t))$ tali che $|\delta_0| < \varepsilon$, $|\delta(t)| < \varepsilon \forall t \in I$, con $\varepsilon > 0$ piccolo, allora esiste $c > 0$ tale che $|y(t) - z(t)| < c\varepsilon \forall t \in I$.

$z(t)$ è la soluzione del **problema perturbato**

$$\begin{cases} \dot{z} = f(t, z(t)) + \delta(t) \\ z(t_0) = y_0 + \delta_0 \end{cases}$$

Definizione 10.2 (Stabilità asintotica). Il problema di Cauchy è **asintoticamente stabile** se

$$\lim_{t \rightarrow \infty} |y(t) - z(t)| \rightarrow 0$$

avendo $|\delta_0| \rightarrow 0$ per $t \rightarrow \infty$.

Lemma 10.1 (Gronwall). Sia p una funzione integrabile e non negativa su $[t_0, t_0 + \tau]$. Siano g e φ due funzioni continue su $[t_0, t_0 + \tau]$ tale che g non sia decrescente.

Se $\varphi(t)$ è tale che:

$$\varphi(t) \leq g(t) + \int_{t_0}^t p(s)\varphi(s) ds$$

allora:

$$\varphi(t) \leq g(t) \exp\left(\int_{t_0}^t p(s) ds\right) \quad \forall t \in [t_0, t_0 + \tau]$$

$z(t)$ è soluzione del problema perturbato, $y(t)$ è soluzione del problema di Cauchy. Mi interessa stimare $w(t) = z(t) - y(t)$.

$$\begin{cases} \dot{w}(t) = \dot{z}(t) - \dot{y}(t) = f(t, z(t)) + \delta(t) - f(t, y(t)) \\ w(t_0) = \delta_0 \end{cases}$$

Dunque questa è una **ODE** con condizione iniziale data dalla perturbazione.

$$\begin{aligned} w(t) &= \delta_0 + \int_{t_0}^t f(s, z(s)) - f(s, y(s)) ds + \int_{t_0}^t \delta(s) ds \\ &\leq |\delta_0| + L \int_{t_0}^t |w(s)| ds + \varepsilon(t - t_0) \quad \text{per la lipschitzianità} \\ &\leq (1 + (t - t_0))\varepsilon + L \int_{t_0}^t |w(s)| ds \\ &\leq (1 + (t - t_0))\varepsilon e^{L(t-t_0)} \quad \text{per il lemma di Gronwall } p(t) = L, g(t) = (1 + (t - t_0))\varepsilon, \varphi(t) = w(t) \\ &= C\varepsilon \quad \text{con } C = (1 + (t - t_0))e^{L(t-t_0)} \end{aligned}$$

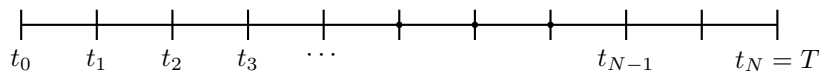
Vediamo dunque che C non dipende da ε (ovvero la perturbazione).

10.1 Metodi numerici ad un passo

$$\begin{cases} \dot{y}(t) = f(t, y(t)) & t \in [t_0, T] \\ y(t_0) = y_0 \end{cases} \iff y(t) = y_0 + \int_{t_0}^t f(s, y(s)) ds$$

Costruiamo una griglia di tempi:

$$t_0 < t_1 < \dots < t_N = T$$



Se la partizione è equidistante definiamo il passo temporale $\Delta t = t_j - t_{j-1} > 0$.

Notazione: $y_j \approx y(t_j)$ $f_j = f(t_j, y_j)$. Dove $y(t_j)$ è la soluzione esatta al tempo t_j , mentre y_j è la soluzione approssimata.

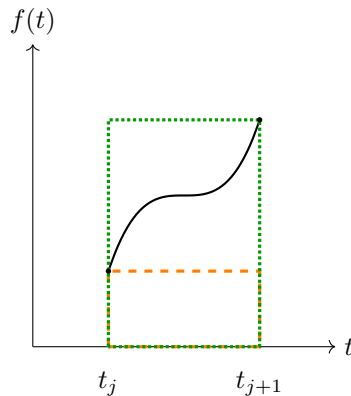
$$\begin{aligned} y(t) &= y_0 + \int_{t_0}^t f(s, y(s)) \, ds \\ y(t_1) &= y_0 + \int_{t_0}^{t_1} f(s, y(s)) \, ds \\ y(t_2) &= y_0 + \int_{t_0}^{t_2} f(s, y(s)) \, ds = \\ &= y_0 + \underbrace{\int_{t_0}^{t_1} f(s, y(s)) \, ds}_{y(t_1)} + \int_{t_1}^{t_2} f(s, y(s)) \, ds \\ &= y(t_1) + \int_{t_1}^{t_2} f(s, y(s)) \, ds \end{aligned}$$

In generale:

$$y(t_{j+1}) = y(t_j) + \int_{t_j}^{t_{j+1}} f(s, y(s)) \, ds$$

Per costruire il metodo numerico utilizzo formule di quadratura.

Vediamo il **Metodo dei rettangoli**. Possiamo approssimare l'integrale scegliendo il valore di f al punto iniziale dell'intervallo.



Approssimo $\Delta t f(t_j, y(t_j))$. Avremo dunque:

$$y_{j+1} = y_j + \Delta t f_j \quad j = 0, 1, \dots, N-1$$

Questo metodo è detto di **Eulero in avanti** (forward). Questo metodo è **esplicito** (utilizzo solo di y_j per calcolare y_{j+1}).

In alternativa posso approssimare l'integrale scegliendo il valore di f al punto finale dell'intervallo, per cui si ottiene:

$$y_{j+1} = y_j + \Delta t f_{j+1} \quad j = 0, 1, \dots, N-1$$

Questo metodo è detto di **Eulero all'indietro** (backward). Questo metodo è **implicito** (utilizzo di y_{j+1} per calcolare f_{j+1}).

Portando tutti i termini a sinistra:

$$y_{j+1} - y_j - \Delta t f_{j+1} = 0$$

ottenendo dunque un **sistema non lineare** (da risolversi con il metodo di Newton).

Vediamo il **Metodo di Crank-Nicolson** (o metodo dei trapezi).

$$y_{j+1} = y_j + \frac{\Delta t}{2}(f_j + f_{j+1}) \quad j = 0, 1, \dots, N-1$$

Dal metodo dei trapezi posso ulteriormente approssimare f_{j+1} usando eulero esplicito:

$$f(t_{j+1}, y_{j+1}) \approx f(t_{j+1}, y_j + \Delta t f_j)$$

lo schema numerico diventa:

$$y_{j+1} = y_j + \frac{\Delta t}{2}(f_j + f(t_{j+1}, y_j + \Delta t f_j)) \quad j = 0, 1, \dots, N-1$$

Ottenendo il **Metodo di Heun** (esplicito).

Definizione 10.3 (Metodo esplicito). Un metodo è detto **esplicito** se y_{j+1} può essere calcolata direttamente conoscendo le soluzioni approssimate y_k con $k \leq j$.

Definizione 10.4 (Metodo implicito). Un metodo è detto **implicito** se la soluzione approssimata dipende implicitamente da se stessa attraverso f .

Definizione 10.5 (Metodo a un passo). Un metodo è detto **a un passo** se $\forall j \geq 0$ allora y_{j+1} dipende solo da y_j . Altrimenti il metodo è detto **multi-step**.

Osservazione 10.2. *I metodi di Heun e di Eulero in avanti sono espliciti e a un passo, mentre il metodo di Eulero all'indietro e di Crank-Nicolson sono impliciti e a un passo.*

10.2 Differenze finite

Posso utilizzare il polinomio di Taylor per approssimare $f(t, y(t))$ in un intorno di t_j .

$$y(t_{j+1}) = y(t_j) + (t_{j+1} - t_j)y'(t_j) + \frac{(t_{j+1} - t_j)^2}{2}y''(\varepsilon)$$

dal problema di Cauchy abbiamo

$$y(t_{j+1}) = y(t_j) + \Delta t f(t_j, y(t_j)) + \frac{\Delta t^2}{2}y''(\varepsilon)$$

Dall'espansione di Taylor otteniamo

$$y_{j+1} = y_j + \Delta t f_j \quad \text{Eulero esplicito}$$

torniamo dunque all'espansione di Taylor

$$y_{j+1} = y_j + \Delta t y'(t_j) + \frac{\Delta t^2}{2}y''(\varepsilon)$$

da cui otteniamo

$$\frac{y_{j+1} - y_j}{\Delta t} = y'(t_j) + \underbrace{\frac{\Delta t}{2}y''(\varepsilon)}_{\mathcal{O}(\Delta t)}$$

che è un'approssimazione della derivata di ordine 1.

$$\begin{aligned} y(t_{j-1}) &= y(t_j) - \Delta t y'(t_j) + \frac{\Delta t^2}{2}y''(\varepsilon) \\ \frac{y(t_{j-1}) - y(t_j)}{\Delta t} &= -y'(t_j) + \frac{\Delta t}{2}y''(\varepsilon) \end{aligned}$$

Se trascuro l'errore:

$$y_{j-1} - y_j = -\Delta t f_j$$

ovvero $y_j = y_{j-1} + \Delta t f_j$ (Eulero implicito).

$$y(t_{j+1}) = y(t_j) + \Delta t y'(t_j) + \frac{\Delta t^2}{2} y''(t_j) + \mathcal{O}(\Delta t^3)$$

e

$$y(t_{j-1}) = y(t_j) - \Delta t y'(t_j) + \frac{\Delta t^2}{2} y''(t_j) + \mathcal{O}(\Delta t^3)$$

e possiamo sottrarre le due equazioni:

$$y(t_{j+1}) - y(t_{j-1}) = 2\Delta t y'(t_j) + \mathcal{O}(\Delta t^3)$$

ovvero

$$\frac{y(t_{j+1}) - y(t_{j-1})}{2\Delta t} = y'(t_j) + \mathcal{O}(\Delta t^2)$$

che è un'approssimazione della derivata di ordine 2. Se trascuro l'errore:

$$y_{j+1} = y_{j-1} + 2\Delta t f_j$$

è un metodo **esplicito** a due passi. Per calcolare il primo valore y_1 uso un metodo a un passo.

Metodi impliciti

Partiamo da eulero all'indietro:

$$y_{j+1} = y_j + \Delta t f_{j+1} \quad j = 0, 1, \dots, N-1$$

per $j = 0$

$$\begin{aligned} y_1 &= y_0 + \Delta t f_1 = \\ &= y_0 + \Delta t f(t_1, y_1) \end{aligned}$$

Occorre dunque trovare la soluzione di un sistema non lineare.

$$F(y_{j+1}) = y_{j+1} - y_j - \Delta t f(t_{j+1}, y_{j+1}) = 0$$

se $n = 1$

$$\frac{dF}{dy_{j+1}} = 1 - \Delta t \frac{\partial f}{\partial y_{j+1}}$$

(Metodo di Newton in una dimensione).

$$\begin{aligned} F(x) &= x - y_j - \Delta t f(t_{j+1}, x) \\ x_{k+1} &= x_k - \frac{F(x_k)}{F'(x_k)} \end{aligned}$$

finchè $|x_{k+1} - x_k| > \varepsilon$. Una volta raggiunta la convergenza, $y_{j+1} = x_k$.

Dato iniziale per il metodo di Newton: posso scegliere, per esempio,

$$x_0 = y_j$$

con l'idea che y_{j+1} sia vicino a y_j . Se $n > 1$ si prosegue nel modo seguente:

1. $J_F(x_k)\delta_k = F(x_k)$ (sistema lineare)
2. $x_{k+1} = x_k - \delta_k$

$$J_F(x_k) = \mathbb{I} - \Delta t J_f(x_k)$$

dove $J_f(x)$ è lo Jacobiano di $f(t_{j+1}, x)$.

10.3 Ordine di convergenza

$$|y(t_j) - y_j| < c\Delta t^p$$

dove p è l'ordine di convergenza del metodo. Dunque:

$$\begin{aligned} \overbrace{|y(t_j) - y_j|}^{e(\Delta t)} &\approx \Delta t^p && \text{metodo con passo } \Delta t \\ \underbrace{|y(t_j) - y_j^*|}_{e(\Delta t^*)} &\approx (\Delta t^*)^p && \text{metodo con passo } \Delta t^* \end{aligned}$$

$$\frac{e(\Delta t)}{e(\Delta t^*)} \approx \left(\frac{\Delta t}{\Delta t^*} \right)^p$$

Voglio trovare p

$$\log_{\frac{\Delta t}{\Delta t^*}} \frac{e(\Delta t)}{e(\Delta t^*)} = p$$

Osservazione 10.3. Per calcolare p devo conoscere la soluzione esatta.

$$\Delta t = \{0.1, 0.05, 0.025, \dots\}$$

dunque

$$\Delta t_i = \frac{0.1}{2^i} \quad i = 0, 1, 2, \dots$$

per cui

$$\frac{\Delta t_{i+1}}{\Delta t_i} = \frac{\frac{0.1}{2^{i+1}}}{\frac{0.1}{2^i}} = \frac{1}{2}$$

Dunque $\frac{\Delta t_i}{\Delta t_{i+1}} = 2$, e $p_i = \log_2 \left(\frac{e(\Delta t_i)}{e(\Delta t_{i+1})} \right)$.

Eulero in avanti/indietro: $p \approx 1$, mentre con Heun e Crank-Nicolson $p \approx 2$.

I metodi espliciti presentato la seguente struttura:

$$y_{i+1} = y_i + \Delta t \Phi(t_i, y_i, f_i, \Delta t) \quad (\text{metodo numerico})$$

dunque

$$y(t_{i+1}) = y(t_i) + \Delta t \Phi(t_i, y(t_i), f(t_i, y(t_i)), \Delta t) + \varepsilon_{i+1}$$

dove ε_{i+1} è il residuo al tempo t_{i+1} .

Supponiamo che $y(t_i)$ soddisfi il metodo numerico. Possiamo scrivere $\varepsilon_{i+1} = \Delta t \tau_{i+1}(\Delta t)$ dove $\tau_{i+1}(\Delta t)$ è l'errore di troncamento locale (LTE).

Mentre **l'errore di troncamento globale** (GTE) è dato da:

$$\tau(\Delta t) = \max_i (\tau_i(\Delta t))$$

dove $\tau_i(\Delta t)$ dipende dal problema di Cauchy.

Osservazione 10.4 (Eulero esplicito).

$$\Phi(t_i, y_i, f_i, \Delta t) = f_i$$

Osservazione 10.5 (Heun).

$$\Phi(t_i, y_i, f_i, \Delta t) = \frac{1}{2}(f_i + f(t_i + \Delta t, y_i + \Delta t f_i))$$

Vogliamo che l'errore di troncamento tenda a zero per $\Delta t \rightarrow 0$.

$$\lim_{\Delta t \rightarrow 0} \tau_{i+1}(\Delta t) = 0 \quad (\text{consistenza del metodo numerico})$$

Definizione 10.6 (Consistenza). Un metodo numerico si dice **consistente** se

$$\lim_{\Delta t \rightarrow 0} \tau(\Delta t) = 0$$

Inoltre se $\tau(\Delta t) = \mathcal{O}(\Delta t^p)$ lo schema numerico è di ordine p .

Definizione 10.7 (Zero-stabilità). Un metodo numerico si dice **zero-stabile** se

$$\exists \Delta t_0 > 0, \exists c > 0 \text{ t.c. } \Delta t \in (0, \Delta t_0) \forall \varepsilon > 0, \text{ se } |\delta_0| \leq \varepsilon \implies |z_i^{(\Delta t)} - y_i^{(\Delta t)}| < c\varepsilon \quad 0 \leq i \leq N$$

dove

$$\begin{cases} z_{i+1}^{(\Delta t)} &= z_i^{(\Delta t)} + \Delta t [\Phi(t_i, z_i^{(\Delta t)}, f(t_i, z_i^{(\Delta t)}), \Delta t) + \delta_{i+1}] \\ z_0^{(\Delta t)} &= y_0 + \delta_0 \end{cases}$$

mentre

$$\begin{cases} y_{i+1}^{(\Delta t)} &= y_i + \Delta t [\Phi(t_i, y_i, f(t_i, y_i), \Delta t) + \varepsilon_{i+1}] \\ y_0^{(\Delta t)} &= y_0 \end{cases}$$

L'idea è che le piccole perturbazioni del metodo numerico non alterino la soluzione approssimata.

Teorema 10.1. Consideriamo metodi espliciti a un passo. Ipotizziamo che la funzione Φ sia di tipo Lipschitz rispetto alla seconda variabile con costante Λ .

$$\exists \Delta t_0 > 0, \exists \Lambda > 0 : \forall \Delta t \in (0, \Delta t_0),$$

$$\begin{aligned} & |\Phi(t_i, y_i^{(\Delta t)}, f(t_i, y_i^{(\Delta t)}), \Delta t) - \\ & \Phi(t_i, z_i^{(\Delta t)}, f(t_i, z_i^{(\Delta t)}), \Delta t)| \leq \Lambda |y_i^{(\Delta t)} - z_i^{(\Delta t)}| \end{aligned}$$

per $0 \leq i \leq N$ e con $|\delta| \leq \varepsilon$. Allora il metodo è zero-stabile.

Dimostrazione. Sia $w_i = z_i - y_i$.

$$w_{i+1} = w_i + \Delta t [\Phi(t_i, z_i, f(t_i, z_i), \Delta t) - \Phi(t_i, y_i, f(t_i, y_i), \Delta t)] + \Delta t \delta_{i+1}$$

sommando su tutti gli indici i :

$$w_N = w_0 + \Delta t \sum_{i=0}^{N-1} [\Phi(t_i, z_i, f(t_i, z_i), \Delta t) - \Phi(t_i, y_i, f(t_i, y_i), \Delta t)] + \Delta t \sum_{i=0}^{N-1} \delta_i$$

dunque

$$|w_N| \leq |w_0| + \Delta t \Lambda \sum_{i=0}^{N-1} |w_i| + \Delta t \sum_{i=0}^{N-1} |\delta_i|$$

utilizzando il lemma di Gronwall discreto:

$$|w_N| \leq (1 + \Delta t N) \varepsilon e^{N \Delta t \Lambda} = c \varepsilon \implies c = (1 + \Delta t N) e^{N \Delta t \Lambda}$$

dove $N \Delta t = T - t_0$. □

Convergenza

Definizione 10.8. Un metodo è convergente se $\forall i = 0, 1, \dots, N$ $|y_i - y(t_i)| < c(\Delta t)$ con $c(\Delta t) \rightarrow 0$ per $\Delta t \rightarrow 0$. Un metodo è convergente con ordine p se $c(\Delta t) = \mathcal{O}(\Delta t^p)$.

Teorema 10.2. Siano soddisfatte le ipotesi del teorema della zero-stabilità. Allora

$$|y_i - y(t_i)| \leq (|y_0 - y(t_0)| + i \Delta t \tau(\Delta t)) e^{i \Delta t \Lambda}$$

Se il metodo è consistente e $y_0 - y(t_0) \rightarrow 0$ per $\Delta t \rightarrow 0$ allora il metodo è convergente. Inoltre se $\tau(\Delta t) = \mathcal{O}(\Delta t^p)$ allora l'ordine di convergenza del metodo è p .

Dimostrazione.

$$w_j = y_j - y(t_j) \quad \delta_{j+1} = \tau_{j+1}(\Delta t)$$

La stima viene dalla zero-stabilità. L'ordine di convergenza viene dall'errore di troncamento. □

Dunque zero-stabilità + consistenza $\implies$ convergenza (Lax-Richtmyer).

Convergenza di Eulero esplicito Ricordiamo Eulero esplicito:

$$y_{i+1} = y_i + \Delta t f_i$$

e l'errore

$$e_{i+1} = y(t_{i+1}) - y_{i+1} = (y(t_{i+1}) - y_{i+1}^*) + (y_{i+1}^* - y_{i+1})$$

dove $y_{i+1}^* = y(t_i) + \Delta t f(t_i, y(t_i))$ per il primo termine $(y(t_{i+1}) - y_{i+1}^*)$ abbiamo

$$e_{i+1} = \Delta t \tau(\Delta t)$$

mentre per il secondo termine $(y_{i+1}^* - y_{i+1})$ abbiamo

$$\begin{aligned} y_{i+1}^* - y_{i+1} &= y(t_i) + \Delta t f(t_i, y(t_i)) - y_i - \Delta t f(t_i, y_i) = \\ &= e_i + \Delta t [f(t_i, y(t_i)) - f(t_i, y_i)] \end{aligned}$$

e

$$\begin{aligned} e_{i+1} &= \Delta t \tau_{i+1}(\Delta t) + e_i + \Delta t [f(t_i, y(t_i)) - f(t_i, y_i)] \\ e_{i+1} &\leq \Delta t |\tau(\Delta t)| + |e_i| + \Delta t L |e_i| = \Delta t (|\tau(\Delta t)| + (1 + \Delta t L) |e_i|) \end{aligned}$$

dove L è la costante di Lipschitz per f .

Si procede ricorsivamente su i e si ottiene

$$e_{i+1} \leq \Delta t (|\tau(\Delta t)| + (1 + \Delta t L) (\Delta t |\tau(\Delta t)| + (1 + \Delta t L) |e_{i-1}|))$$

quindi

$$|e_{i+1}| \leq [1 + (1 + \Delta t L) + \dots + (1 + \Delta t L)^i] \Delta t \tau(\Delta t) \leq \frac{e^{L(t_{i+1}-t_0)} - 1}{L} \tau(\Delta t)$$

Questo viene dallo sviluppo di Taylor:

$$\begin{aligned} y(t_{i+1}) &= y(t_i) + \Delta t y'(t_i) + \frac{\Delta t^2}{2} y''(\varepsilon) \\ \frac{y(t_{i+1}) - y(t_i)}{\Delta t} &= \underbrace{y'(t_i)}_{f(t_i, y(t_i))} + \frac{\Delta t}{2} y''(\varepsilon) \end{aligned}$$

Nel caso di Eulero esplicito:

$$\tau(\Delta t) = \frac{1}{2} \Delta t y''(\varepsilon) \implies \text{metodo convergente}$$

10.4 Stabilità assoluta

$$\begin{cases} y'(t) = \lambda y(t) \\ y(0) = 1 \end{cases}$$

con $\lambda \in \mathbb{C}$. La soluzione esatta è $y(t) = e^{\lambda t}$.

$$\lim_{t \rightarrow \infty} |y(t)| = 0 \iff \Re(\lambda) < 0$$

Definizione 10.9 (Assoluta stabilità). Un metodo numerico si dice **assolutamente stabile** se

$$|y_i| \rightarrow 0 \quad \text{quando} \quad t_i \rightarrow \infty \quad y_i \approx y(t_i)$$

Definizione 10.10 (Regione di assoluta stabilità). La regione di assoluta stabilità è l'insieme

$$A = \{z = \lambda \Delta t \in \mathbb{C} \mid |y_i| \rightarrow 0\}$$

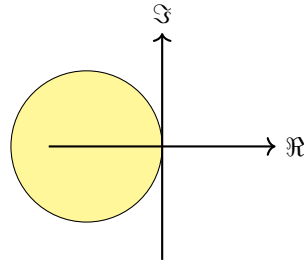
Eulero esplicito

$$y_{i+1} = y_i + \Delta t \lambda y_i = y_i(1 + \Delta t \lambda)$$

in generale $y_i = \underbrace{y_0}_{=1} (1 + \Delta t \lambda)^i$ e dunque la condizione di assoluta stabilità è verificata quando

$$|1 + \Delta t \lambda| < 1 \implies -2 < \Delta t \lambda < 0$$

Ovvero la palla unitaria centrata in $(-1, 0)$



Dunque $\lambda \Delta t \in \mathbb{C}^- \implies 0 < \Delta t < \frac{-2\Re(\lambda)}{|\lambda|^2}$.

Esempio 10.1. Consideriamo il sistema:

$$\begin{cases} y' = -5y \\ y(0) = 1 \end{cases}$$

Abbiamo $\lambda = -5$ e $|1 - 5\Delta t| < 1 \implies 0 < \Delta t < \frac{2}{5}$. Se si supera questo valore la soluzione potrebbe andare a zig-zag.

Convergenza di Eulero implicito

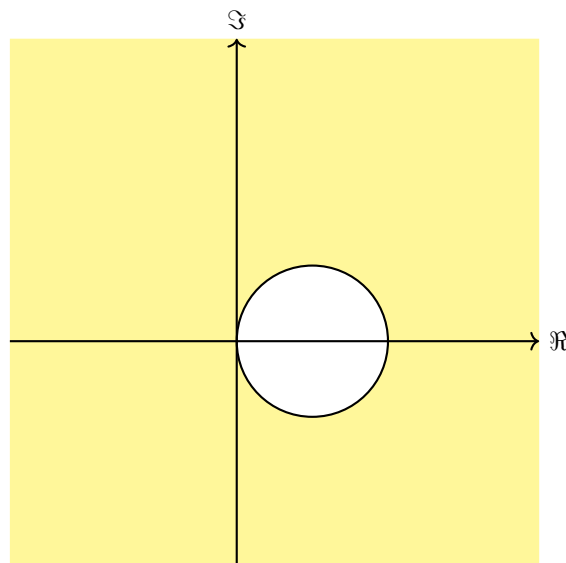
$$y_{i+1} = y_i + \Delta t \lambda y_{i+1} \implies y_{i+1} = \frac{y_i}{1 - \Delta t \lambda}$$

e dunque

$$y_i = \frac{1}{(1 - \Delta t \lambda)^i}$$

da cui deduciamo la regione di assoluta stabilità:

$$\frac{1}{1 - \Delta t \lambda} < 1 \iff |1 - \Delta t \lambda| > 1$$



e sapendo che $\Re(\lambda) < 0$ la regione di assoluta stabilità è $\mathbb{C}^- \Rightarrow$ non ci sono restrizioni su Δt , o meglio il metodo risulta sempre assolutamente stabile ($\Re(\lambda) < 0$).

Convergenza di Crank-Nicolson

$$\begin{aligned} y_{i+1} &= y_i + \frac{\Delta t}{2} [\lambda y_i + \lambda y_{i+1}] \\ y_{i+1} \left[1 - \frac{\Delta t}{2} \lambda \right] &= y_i \left[1 + \frac{\Delta t}{2} \lambda \right] \\ y_i &= \frac{y_{i+1} \left[1 - \frac{\Delta t}{2} \lambda \right]}{1 + \frac{\Delta t}{2} \lambda} \end{aligned}$$

e la regione di assoluta stabilità è data da

$$\left| \frac{1 + \lambda \frac{\Delta t}{2}}{1 - \lambda \frac{\Delta t}{2}} \right| < 1 \iff \left| \frac{2 + \lambda \Delta t}{2 - \lambda \Delta t} \right| < 1$$

e svolgendo i calcoli si ottiene che il metodo risulta assolutamente stabile sempre ($\Re(\lambda) < 0$).

Convergenza di Heun

$$\begin{aligned} y_{i+1} &= y_i + \frac{\Delta t}{2} [\lambda y_i + \lambda y_i + \Delta t \lambda^2 y_i] \\ &= y_i \left(1 + \Delta t \lambda + \frac{\Delta t^2 \lambda^2}{2} \right) \end{aligned}$$

e dunque

$$y_i = y_0 \left(1 + \Delta t \lambda + \frac{\Delta t^2 \lambda^2}{2} \right)^i$$

e la regione di assoluta stabilità è data da

$$\left| 1 + \Delta t \lambda + \frac{\Delta t^2 \lambda^2}{2} \right| < 1$$

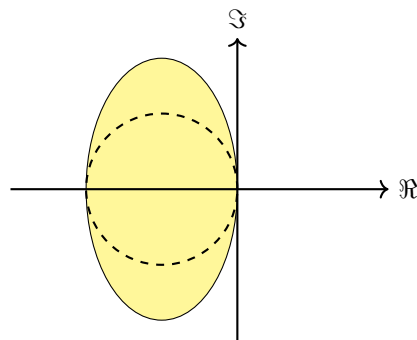


Figura 7: Regioni di stabilità: ellisse di Heun (riempita) e cerchio di Eulero esplicito (tratteggiato).

Osservazione 10.6. *I metodi impliciti a un passo sono incondizionatamente assolutamente stabili, mentre i metodi espliciti presentano restrizioni sulla scelta del passo temporale Δt .*

10.5 Metodo Runge-Kutta

È un metodo numerico a un passo ed è un metodo che richiede il calcolo di f varie volte. Di seguito presentiamo la struttura generale del metodo:

$$y_{n+1} = y_n + \Delta t F(t_n, y_n, \Delta t, f)$$

e

$$F(t_n, y_n, \Delta t, f) = \sum_{i=1}^s b_i k_i$$

dove

$$k_i = f(t_n + c_i \Delta t, y_n + \Delta t \sum_{j=1}^s a_{ij} k_j)$$

e s è il numero di **stages** del problema e osserviamo che il metodo viene caratterizzato dalla scelta di a_{ij}, b_i, c_i . Avremmo dunque:

$$\begin{array}{c|cccc} c_1 & a_{11} & a_{12} & \cdots & a_{1s} \\ c_2 & a_{21} & a_{22} & \cdots & a_{2s} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ c_s & a_{s1} & a_{s2} & \cdots & a_{ss} \\ \hline & b_1 & b_2 & \cdots & b_s \end{array}$$

$$c_i = \sum_{j=1}^s a_{ij} \quad \begin{array}{c|c} c & A \\ \hline & b^\top \end{array} \quad A \in \mathbb{R}^{s \times s} \quad b, c \in \mathbb{R}^s$$

Osservazione 10.7. Il metodo risulta esplicito se $a_{ij} = 0 \forall j \geq i$ (matrice A triangolare inferiore). Altrimenti il metodo risulta implicito. Notiamo anche che il metodo implicito ha un costo computazionale elevato poichè ad ogni iterazione risolve un sistema di s equazioni non lineari.

- Se $a_{ij} = 0 \forall j > i$ ho il caso semi-implicito.

$$k_i = f(t_n + c_i \Delta t, y_n + \Delta t a_{ii} k_i + \Delta t \sum_{j=1}^{i-1} a_{ij} k_j)$$

Esempio 10.2 ($s = 4$ esplicito).

$$y_{n+1} = y_n + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4)$$

$$k_1 = f_n$$

$$k_2 = f(t_n + \frac{\Delta t}{2}, y_n + \frac{\Delta t}{2} k_1)$$

$$k_3 = f(t_n + \frac{\Delta t}{2}, y_n + \frac{\Delta t}{2} k_2)$$

$$k_4 = f(t_{n+1}, y_n + \Delta t k_3)$$

In questo caso la tabella di Butcher è:

$$\begin{array}{c|cccc} 0 & 0 & 0 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ \hline & \frac{1}{6} & \frac{1}{3} & \frac{1}{3} & \frac{1}{6} \end{array}$$

Osserviamo che $\frac{1}{6} + \frac{1}{3} + \frac{1}{3} + \frac{1}{6} = 1$, e questo ci dice che il metodo di Runge-Kutta è consistente.

Vediamo un caso di metodo RK implicito per $s = 2$:

$$y_{n+1} = y_n + \Delta t(b_1 k_1 + b_2 k_2) \quad \begin{array}{c|cc} c_1 & a_{11} & a_{12} \\ c_2 & a_{21} & a_{22} \\ \hline & b_1 & b_2 \end{array}$$

$$k_1 = f(t_n + c_1 \Delta t, y_n + \Delta t a_{11} k_1 + \Delta t a_{12} k_2)$$

$$k_2 = f(t_n + c_2 \Delta t, y_n + \Delta t a_{21} k_1 + \Delta t a_{22} k_2)$$

Vediamo dunque delle **proprietà** per i metodi espliciti RL se p è l'ordine di convergenza e s è il numero di stadi. Possiamo scegliere $s \geq p$ o porre anche vincoli di minimo $\begin{cases} s > p \\ s \geq 5 \end{cases}$.

Costruiamo un metodo RK esplicito di ordine $s = 2$.

$$\begin{array}{c|cc} 0 & 0 & 0 \\ c_2 & c_2 & 0 \\ \hline & b_1 & b_2 \end{array}$$

Perché valga la consistenza deve essere $b_1 + b_2 = 1$. Ipotizziamo di conoscere $y(t_n)$:

$$y_{n+1} = y(t_n) + \Delta t(b_1 k_1 + b_2 k_2)$$

$$k_1 = f_n \quad k_2 = f(t_n + c_2 \Delta t, y(t_n) + \Delta t \overbrace{a_{21}}^{=c_2} k_1)$$

espandiamo dunque k_2 in un intorno di t_n :

$$\begin{aligned} y_{n+1} &= y(t_n) + \Delta t(b_1 f_n + b_2(f_n + \Delta t c_2(f_{n,t} + f_{n,y} f_n))) + \mathcal{O}(\Delta t^3) \\ &= y(t_n) + \Delta t f_n(b_1 + b_2) + \Delta t^2 c_2 b_2(f_{n,t} + f_{n,y} f_n) + \mathcal{O}(\Delta t^3) \end{aligned}$$

Se calcolo Taylor sulla soluzione esatta $y(t_n + 1)$ nell'intorno di t_n :

$$\begin{aligned} y(t_n + 1) &= y(t_n) + \Delta t y'(t_n) + \frac{\Delta t^2}{2} y''(t_n) + \mathcal{O}(\Delta t^3) \\ &= y(t_n) + \Delta t f_n + \frac{\Delta t^2}{2} (f_{n,t} + f_{n,y} f_n) + \mathcal{O}(\Delta t^3) \end{aligned}$$

e dunque:

$$\Rightarrow \begin{cases} b_1 + b_2 = 1 \\ c_2 b_2 = \frac{1}{2} \end{cases}$$

per cui le soluzioni di questo sistema restituiscono tutti i metodi RK espliciti con $s = 2$. Per Heun $b_1 = \frac{1}{2}, b_2 = \frac{1}{2}, c_2 = 1$

$$y_{n+1} = y_n + \Delta t \left(\frac{1}{2} f_n + \frac{1}{2} f(t_n + \Delta t, y_n + \Delta t f_n) \right)$$

In **Matlab** risolviamo RK esplicito con il comando **ode45**.

$$[t, y] = \text{ode45}(@f, [t_0, T], y_0);$$

Errore di troncamento locale

$$\Delta t \underbrace{\tau_{n+1}(\Delta t)}_{LTE} = y(t_{n+1}) - y(t_n) - \Delta t F(t_n, y_n, \Delta t, f)$$

Consistenza si ha se $\tau(\Delta t) \rightarrow 0$ quando $\Delta t \rightarrow 0$. Si dimostra che un metodo di Runge-Kutta è consistente se e solo se $\sum_{i=1}^s b_i = 1$ (non dimostrato). Inoltre il metodo è di ordine p se $\tau(\Delta t) = \mathcal{O}(\Delta t^p)$.

10.6 Runge-Kutta adattivo

Il metodo di Runge-Kutta adattivo permette di controllare automaticamente la dimensione del passo temporale Δt per mantenere l'errore entro una tolleranza prefissata.

Consideriamo un metodo RK di ordine p e supponiamo di conoscere $y_n = y(t_n)$, la soluzione esatta al tempo t_n .

Stima dell'errore con doppio passo

L'idea è di calcolare la stessa soluzione con due passi diversi e confrontare i risultati per stimare l'errore.

Passo singolo Δt : Calcoliamo y_{n+1} con passo Δt . L'errore di troncamento locale è:

$$y(t_{n+1}) - y_{n+1} = C\Delta t^{p+1} + \mathcal{O}(\Delta t^{p+2})$$

dove C è una costante che dipende dalla derivata $(p+1)$ -esima della soluzione esatta.

Passo doppio $2\Delta t$: Calcoliamo $\hat{y}_{n+1}$ partendo da y_{n-1} con un singolo passo di ampiezza $2\Delta t$. L'errore è:

$$y(t_{n+1}) - \hat{y}_{n+1} = C(2\Delta t)^{p+1} + \mathcal{O}((2\Delta t)^{p+2}) = C \cdot 2^{p+1} \Delta t^{p+1} + \mathcal{O}(\Delta t^{p+2})$$

Stima dell'errore: Sottraendo le due espressioni otteniamo:

$$\begin{aligned} y_{n+1} - \hat{y}_{n+1} &= (y(t_{n+1}) - \hat{y}_{n+1}) - (y(t_{n+1}) - y_{n+1}) \\ &= C \cdot 2^{p+1} \Delta t^{p+1} - C \Delta t^{p+1} + \mathcal{O}(\Delta t^{p+2}) \\ &= C \Delta t^{p+1} (2^{p+1} - 1) + \mathcal{O}(\Delta t^{p+2}) \end{aligned}$$

Quindi possiamo stimare l'errore del passo singolo come:

$$\varepsilon = y(t_{n+1}) - y_{n+1} \approx \frac{y_{n+1} - \hat{y}_{n+1}}{2^{p+1} - 1}$$

Questa stima ci permette di:

- **Accettare il passo** se $|\varepsilon| \leq$ tolleranza
- **Rifiutare il passo** e ridurre Δt se l'errore è troppo grande
- **Aumentare Δt** per il passo successivo se l'errore è molto piccolo

Controllo adattivo del passo: Il nuovo passo viene scelto secondo la formula:

$$\Delta t_{\text{nuovo}} = \Delta t \left(\frac{\text{tolleranza}}{|\varepsilon|} \right)^{1/(p+1)}$$

10.7 Runge-Kutta adattivo tramite *step-doubling*

Un integratore Runge-Kutta (RK) di ordine p permette di controllare automaticamente il passo temporale Δt stimando l'errore locale e confrontandolo con una tolleranza fissata.

Obiettivo. Dato $y_n \approx y(t_n)$, vogliamo ottenere $y_{n+1} \approx y(t_{n+1})$ con errore controllato.

Doppia integrazione sullo stesso intervallo

1. **Soluzione grossolana** $y_{n+1}^{[1]}$ un solo passo di ampiezza Δt .
2. **Soluzione fine** $y_{n+1}^{[2]}$ due passi consecutivi di ampiezza $\frac{\Delta t}{2}$.

Ordine degli errori. Per un metodo di ordine p vale, a meno di una costante C che dipende da $y^{(p+1)}$:

$$\begin{aligned} y(t_{n+1}) - y_{n+1}^{[1]} &= C \Delta t^{p+1} + \mathcal{O}(\Delta t^{p+2}), \\ y(t_{n+1}) - y_{n+1}^{[2]} &= 2C \left(\frac{\Delta t}{2} \right)^{p+1} + \mathcal{O}(\Delta t^{p+2}) = \frac{C \Delta t^{p+1}}{2^p} + \mathcal{O}(\Delta t^{p+2}). \end{aligned}$$

Stima dell'errore. La differenza tra le due approssimazioni è

$$\delta := y_{n+1}^{[2]} - y_{n+1}^{[1]} = C \Delta t^{p+1} \left(1 - \frac{1}{2^p}\right) + \mathcal{O}(\Delta t^{p+2}).$$

L'errore sulla soluzione *fine* (quella che vogliamo tenere) vale quindi

$$\varepsilon \approx \frac{\delta}{2^p - 1}.$$

Regole di accettazione e di scelta del nuovo passo.

- **Accetta** il passo se $|\varepsilon| \leq \text{tol}$ (conservi $y_{n+1}^{[2]}$).
- **Rifiuta** e ripeti con passo ridotto se $|\varepsilon| > \text{tol}$.
- **Aggiorna** il passo secondo

$$\Delta t_{\text{nuovo}} = \Delta t \left(\frac{\text{tol}}{|\varepsilon|} \right)^{1/(p+1)} \times \text{safety},$$

dove $\text{safety} \in (0, 1)$ (es. 0.9) evita oscillazioni.

Questo procedimento garantisce che l'errore locale resti entro la tolleranza prefissata con il minimo numero di valutazioni della funzione $f(t, y)$.